The OSI Model for System Design

L3, L4, L7, and where TLS really sits

Watch the 3-min video

In one line: the OSI model splits networking into seven layers, each doing one job; in system design you mostly care about L3 (IP), L4 (ports, TCP/UDP) and L7 (HTTP), because that is what "L4 load balancer" and "L7 firewall" refer to.

1 Seven layers, one job each

You type a URL and a page appears in milliseconds. Behind that, a stack of protocols is working. Each layer does one job and relies on the layer below it. The **OSI model** gives each layer a number, and engineers use those numbers as shorthand all the time.

From the bottom: **1 Physical** sends bits on wires and radio. **2 Data link** moves frames between neighbours on one network, using MAC addresses. **3 Network** routes packets between networks using IP addresses. **4 Transport** connects processes using ports, with TCP or UDP. **5 Session**, **6 Presentation** and **7 Application** sit on top; HTTP lives at layer 7. To remember the order from the top: *All People Seem To Need Data Processing*.

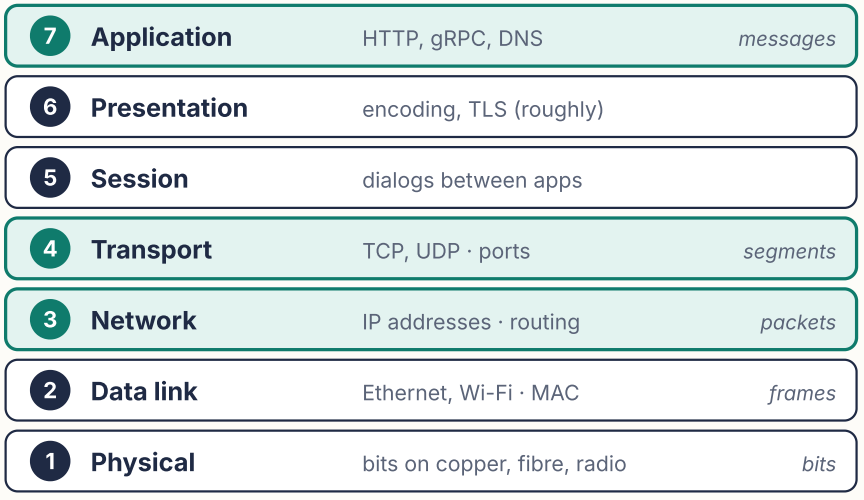
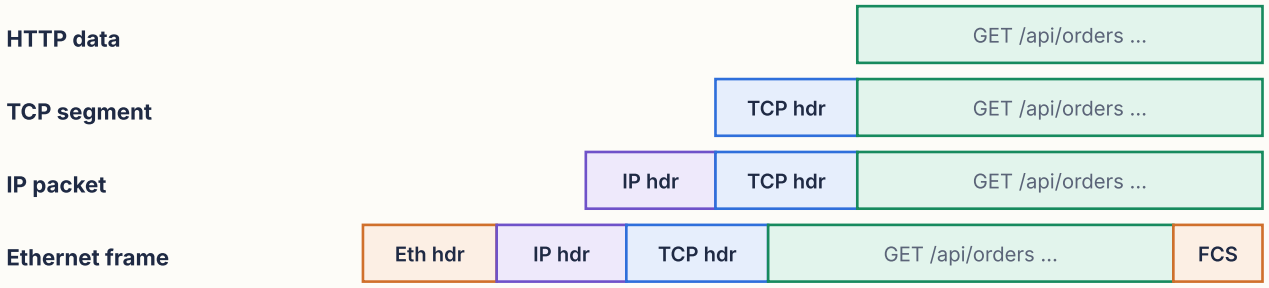


Figure 1. The OSI stack. Layers 3, 4 and 7 (highlighted) are the ones you will name in system design interviews.

Why only three? Layer 3 decides *which machine* gets a packet. Layer 4 decides *which process* on that machine, and whether delivery is reliable (TCP) or fast and best-effort (UDP). Layer 7 is the first layer that understands the actual request: the method, the path, the headers and the cookies.

2 Encapsulation: wrapping on the way down

As data goes down the stack, each layer wraps it in its own header. HTTP data becomes the payload of a **TCP segment**, which becomes the payload of an **IP packet**, which goes inside an **Ethernet frame**, which is sent as bits. The receiver unwraps it in reverse order. Each layer only reads its own header and treats the rest as opaque bytes. This is **encapsulation**.



sender: each layer adds its own header on the way down · receiver: strips them in reverse on the way up

Figure 2. The same HTTP request at each layer. Each header carries what that layer needs: ports (TCP), IP addresses (IP), MAC addresses and a checksum (Ethernet).

3 Every device works at a layer

A device's layer tells you what it can see, and so what it can decide on:

Device	Layer	What it looks at
Switch	L2	MAC addresses; forwards frames inside one network
Router	L3	Destination IP; forwards packets between networks
Basic firewall, cloud security group	L3 / L4	IPs, ports and protocol (allow TCP 443 from 10.0.0.0/16)
L4 load balancer	L4	IPs and ports of a connection
Reverse proxy, API gateway, WAF, L7 LB	L7	The HTTP request itself: method, path, host, headers, cookies, body

4 L4 vs L7 load balancers

This is the classic interview example. An **L4 load balancer**, such as the AWS Network Load Balancer, sees only IP addresses and ports. It picks a backend for each connection and forwards the bytes without reading them. That makes it very fast, and it works for any TCP or UDP protocol: databases, MQTT, game traffic, gRPC, anything.

An **L7 load balancer**, such as the AWS Application Load Balancer, reads HTTP. It can send `/api` to one service and `/images` to another, route by host name or header, add headers, and retry a failed request on another backend. The price is more work per request: it must parse every request, and usually decrypt it first.

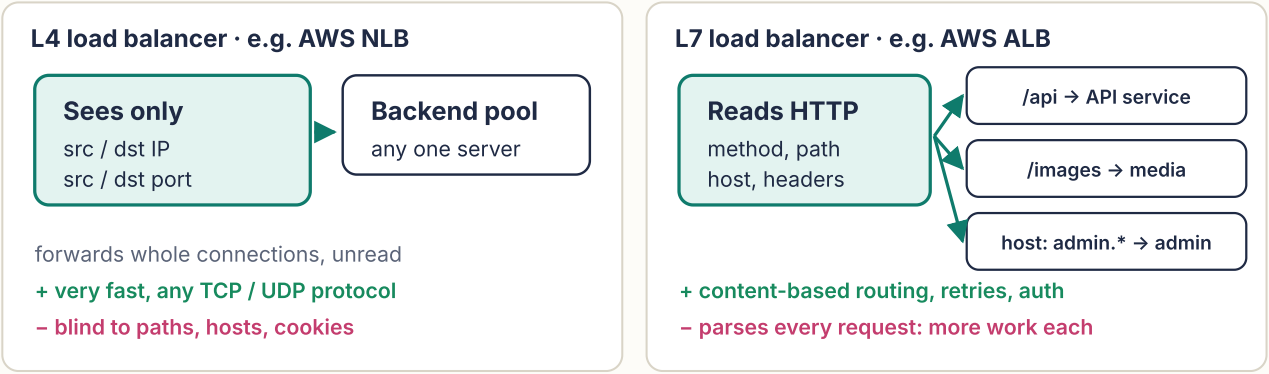


Figure 3. Left: an L4 balancer forwards connections by IP and port only. Right: an L7 balancer reads each HTTP request and routes it by path, host or header.

WORKED EXAMPLE: WHICH ONE?

A public website with /api, /images and /admin served by different services: L7. Only something that reads the path can split them, and you get one TLS certificate in one place.

A fleet of PostgreSQL read replicas, or a multiplayer game on UDP: L4. There is no HTTP to read, and you want the lowest added latency. Many real setups use both: an L4 balancer in front for raw scale and a static IP, L7 proxies behind it for routing.

5 Where does a TLS-terminating proxy operate?

TLS sits above TCP and below HTTP. In OSI terms that is roughly the presentation layer (6), and the TCP/IP model just counts it as part of the application. But once a proxy has decrypted the traffic, it can read the HTTP inside, so in practice we call a TLS-terminating proxy **layer 7**. The alternative is **TLS passthrough**: an L4 balancer forwards the encrypted bytes untouched and the backend decrypts them. The balancer then never sees the path or the headers.

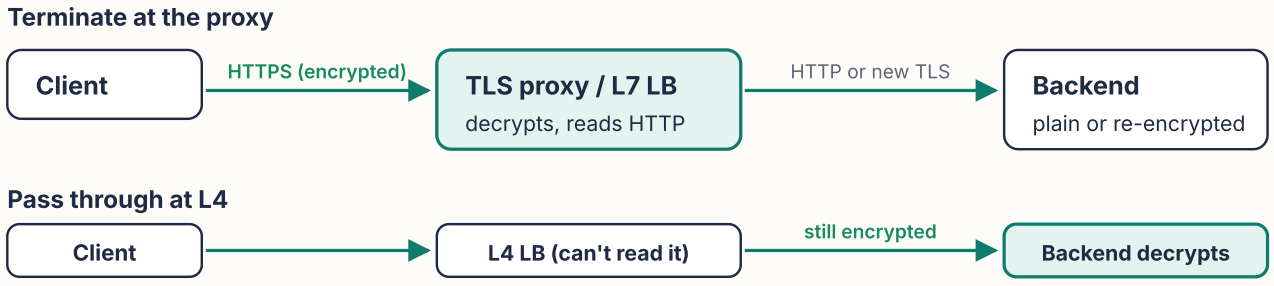


Figure 4. Terminating TLS at the proxy lets it route by HTTP content; passing it through keeps traffic encrypted end to end, but the balancer can only route by IP and port.

COMMON MISTAKES

- Saying an L4 balancer can route by URL path: it never sees the path.
- Calling TLS “layer 4” because it runs over TCP. It sits above TCP; a proxy that decrypts it acts at L7.
- Treating OSI as how the internet is built. It's a vocabulary; the internet runs on the 4-layer TCP/IP model (next lesson).
- Forgetting that L7 features need decryption, so the proxy needs the certificate and private key.

6 Key terms

OSI model: 7-layer reference model for networking

Port: number that picks a process on a host (L4)

TLS termination: proxy decrypts, then can read HTTP

Encapsulation: each layer wraps data in its own header

L4 load balancer: routes connections by IP and port

TLS passthrough: balancer forwards encrypted bytes; backend decrypts

MAC address: hardware address used at L2

L7 load balancer: routes requests by HTTP content

WAF: web application firewall, filters HTTP at L7

7 Interview questions

Q1. Name the OSI layers and the one or two that matter most in system design. BASIC

Physical, data link, network, transport, session, presentation, application. The ones that come up most are L3 (IP routing), L4 (TCP/UDP and ports) and L7 (HTTP: method, path, headers).

Q2. At which layer does a TLS-terminating proxy operate? BASIC

TLS itself sits between transport and application (roughly OSI layer 6). Once the proxy decrypts, it reads HTTP, so it operates at layer 7 in practice. An L4 balancer can instead pass the encrypted bytes through for the backend to decrypt.

Q3. Why does the L4 vs L7 distinction matter for load balancers? BASIC

L4 sees only IPs and ports: very fast, works for any TCP or UDP protocol, but blind to content. L7 reads HTTP, so it can route by path, host and headers, retry requests and terminate TLS, at a higher cost per request.

Q4. You need /api/* to go to the API service and /static/* to a file server. Which load balancer? SCENARIO

An L7 load balancer (ALB, NGINX, Envoy). Routing by path needs to read the HTTP request, which an L4 balancer never sees.

Q5. You must load-balance 100,000 long-lived MQTT or game connections with minimal added latency. Which type? SCENARIO

L4. The traffic isn't HTTP, connections are long-lived, and an L4 balancer forwards packets with almost no per-request work. A managed NLB scales to millions of connections and can keep the client's source IP.

Q6. Security wants traffic encrypted all the way to the app servers, but you also want path-based routing. What do you do? SCENARIO

Terminate TLS at the L7 proxy to read the path, then re-encrypt to the backends with a new TLS connection. Passthrough would keep one TLS session end to end but loses path routing.

Q7. A security group allows TCP 443. Can it block a request to /admin? TRICKY

No. Security groups filter at L3/L4 (IP, port, protocol) and never see the URL. Blocking /admin needs an L7 component: a WAF, API gateway or reverse proxy rule.

Q8. Is an L7 load balancer always better because it has more features? TRICKY

No. It parses (and usually decrypts) every request, adding latency and CPU, and only handles protocols it understands. For non-HTTP traffic or raw throughput, L4 is the better tool; many systems put L4 in front of L7.

Next: SD-B-002 · The TCP/IP Model backendroadmap.com

The TCP/IP Model

what really happens when you type a URL

Watch the 3-min video

In one line: the internet runs on the 4-layer TCP/IP model (link, internet, transport, application); opening a URL costs a DNS lookup, a TCP handshake and a TLS handshake before the first byte of HTTP, which is why caching and connection reuse matter.

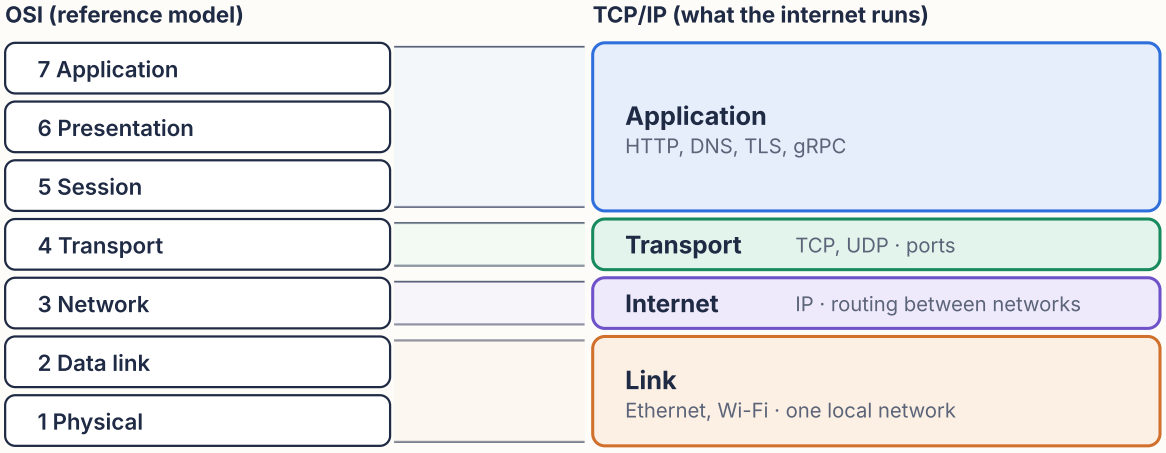
1 Four layers, not seven

The OSI model from the last lesson is a vocabulary. The internet actually runs on **TCP/IP**, a simpler four-layer model:

- **Link**: moves frames across one local network, such as Wi-Fi or Ethernet, using MAC addresses.
- **Internet**: moves packets between networks using IP addresses. Routers work here.
- **Transport**: connects processes using ports, with TCP (reliable stream) or UDP (fast datagrams).
- **Application**: everything your code speaks: HTTP, DNS, TLS, gRPC, SMTP.

2 Mapping TCP/IP to OSI

The mapping is simple. Link covers OSI layers 1 and 2. Internet is layer 3. Transport is layer 4. Application squashes layers 5, 6 and 7 into one, which is why TLS has no clean OSI number. That is also why engineers still say “L4” and “L7”: **the numbers come from OSI, but the protocols come from TCP/IP.**



“L4” and “L7” keep the OSI numbers, but the protocols are TCP/IP's

Figure 1. TCP/IP folds OSI's top three layers into one application layer and its bottom two into one link layer.

3 Opening shop.example.com, step by step

1. **DNS.** The browser asks a resolver for the server's IP address, usually over UDP port 53. If the answer is cached (browser, OS or resolver), this step costs almost nothing.
2. **TCP handshake** to port 443: SYN, SYN-ACK, ACK. One round trip before any data.
3. **TLS handshake.** The server proves who it is with a certificate signed by a trusted authority, and both sides agree on encryption keys. TLS 1.3 needs one round trip; the older TLS 1.2 needs two.
4. **HTTP.** Only now does the browser send `GET /`, inside the encrypted channel, and the server replies.

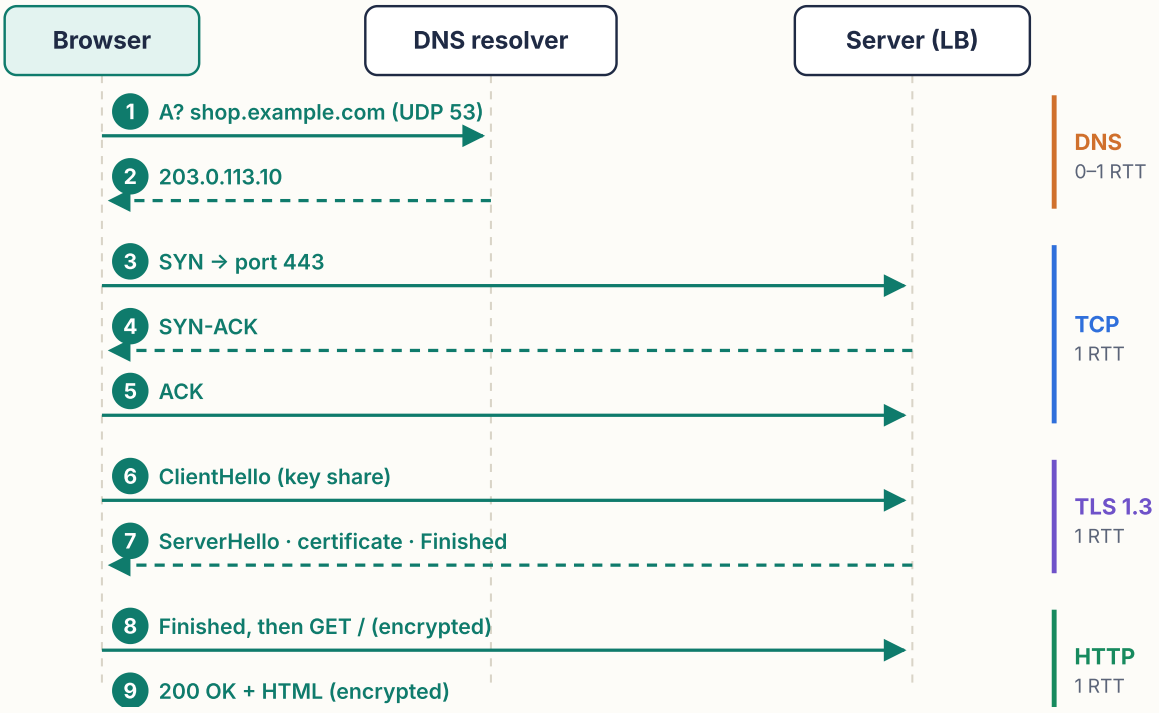


Figure 2. A cold first request: DNS, then TCP, then TLS, and only then HTTP. Solid arrows go to the server, dashed arrows come back.

4 Wrapped, then routed hop by hop

On the way out, the request is wrapped layer by layer: HTTP inside TLS, inside a TCP segment (ports 51514 → 443), inside an IP packet (to 203.0.113.10), inside a Wi-Fi frame. Then it hops: your home router, your ISP, several internet routers, the cloud load balancer, and finally the VM.

At every hop, a router strips the link frame, reads the destination IP, picks the next hop and builds a **new frame** for the next link. The **destination IP stays the same** all the way to the load balancer. (The source IP usually changes once: your home router's NAT swaps your private address for its public one.) The VM unwraps the request in reverse, and the response travels back the same way.

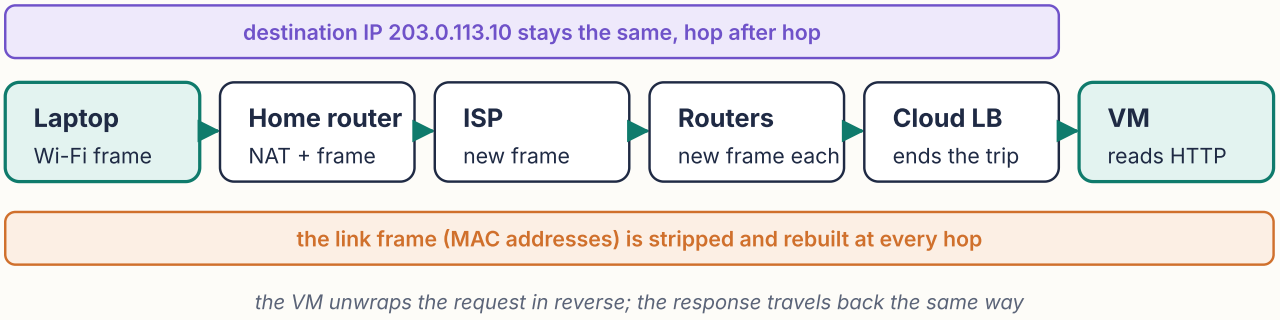


Figure 3. Frames are local to one link and are rebuilt at every hop; the IP packet inside travels end to end.

5 The hidden cost before the first byte

Count the round trips: a DNS lookup, a TCP handshake and a TLS handshake all happen *before a single byte of HTTP*. On a fresh connection, the setup can take longer than the request itself.

WORKED EXAMPLE

A user in Bengaluru opens a site hosted in Frankfurt. The round-trip time (RTT) is about **150 ms**.

- Cold: DNS (say 30 ms to a nearby resolver) + TCP 150 + TLS 1.3 150 + HTTP 150 ≈ **480 ms** to the first response byte.
- With TLS 1.2 instead: one more RTT, ≈ **630 ms**.
- Second request on the same kept-alive connection, DNS cached: just the HTTP round trip, ≈ **150 ms**.

Same server, same code: 3–4× faster just by reusing the connection. A CDN edge 10 ms away shrinks every handshake round trip too.

So the main levers are: **DNS caching** (sensible TTLs), **connection reuse** (HTTP keep-alive, connection pools, HTTP/2), **TLS 1.3 and session resumption**, and **moving the handshake closer** to users with a CDN or edge proxy.

COMMON MISTAKES

- Saying the internet "runs on OSI". It runs on TCP/IP; OSI supplies the layer numbers.
- Thinking the IP address changes at every router. The frame changes; the destination IP does not (until it reaches the load balancer).
- Forgetting DNS and TLS when estimating latency. A cold request costs several round trips, not one.
- Opening a new connection per request between services, paying TCP and TLS every time.

6 Key terms

TCP/IP model: link, internet, transport, application

Handshake: setup messages before data (TCP, TLS)

Certificate: proof of the server's identity, signed by a CA

Resolver: DNS server that looks up names for clients

Hop: one router-to-router step on the path

NAT: rewrites private source IPs to a public one

RTT: round-trip time between two hosts

Keep-alive: reusing a TCP connection for many requests

TTL (DNS): how long an answer may be cached

7 Interview questions

Q1. How does the TCP/IP model map to OSI? BASIC

Link covers OSI 1 and 2, internet is 3, transport is 4, and application covers 5, 6 and 7. That is why TLS has no clean OSI number: TCP/IP just counts it as application.

Q2. What happens when you type a URL and press Enter? BASIC

DNS resolves the name to an IP (usually over UDP). TCP opens a connection to port 443 (SYN, SYN-ACK, ACK). TLS authenticates the server with its certificate and agrees keys. Then the browser sends GET / over the encrypted channel, and IP routes every packet hop by hop to the server and back.

Q3. Which parts of a packet change at each router hop? BASIC

The link-layer frame (source and destination MAC addresses) is rebuilt at every hop, and the IP TTL drops by one. The destination IP stays the same until the packet reaches its destination, such as the load balancer.

Q4. Users far from your only region complain the first page load is slow, though server time is 20 ms. Why, and what helps? SCENARIO

The cold path pays several round trips (DNS, TCP, TLS, HTTP); at 150 ms RTT that is about 0.5 s before any server work matters. Use a CDN or edge proxy to terminate TCP and TLS near users, TLS 1.3, keep-alive and longer DNS TTLs.

Q5. A microservice calls another over HTTPS and opens a new connection per call. p50 is 40 ms with 1 ms of work. What do you change? SCENARIO

Reuse connections: an HTTP client with a keep-alive pool, or HTTP/2 or gRPC multiplexing. That removes the TCP and TLS handshakes from every call, leaving roughly one RTT plus work.

Q6. Where does DNS fit in the layers, and why is it usually UDP? SCENARIO

DNS is an application-layer protocol. A query and its answer usually fit in one small packet each, so UDP avoids a handshake. It falls back to TCP for large answers and zone transfers.

Q7. Why do we still say L4 and L7 if the internet uses TCP/IP? TRICKY

The numbers are a shared vocabulary from OSI. L4 means the transport layer (TCP/UDP, ports) and L7 means the application (HTTP), which map cleanly onto TCP/IP's transport and application layers.

Q8. Is the source IP your server sees the user's laptop address? TRICKY

Usually not. Home and mobile networks use NAT, so the server sees the router's or carrier's public IP. Behind a proxy or L7 load balancer, it sees the proxy's IP, and the client's IP arrives in a header such as X-Forwarded-For.

Next: SD-B-003 · TCP Explained backendroadmap.com

TCP Explained

ACKs, flow control, congestion control, head-of-line blocking

Watch the 2-min video

> **In one line:** TCP turns unreliable IP into a reliable, ordered byte stream using sequence numbers, ACKs and retransmission, adds flow and congestion control so it doesn't drown the receiver or the network, and pays for strict ordering with head-of-line blocking.

1 A reliable stream on an unreliable network

IP is best effort. Packets can be lost, duplicated, or arrive out of order. Yet your database results always arrive complete and in order. That is TCP's job.

TCP gives the application a **reliable, ordered byte stream**. The sender splits the stream into **segments** and gives every byte a **sequence number**. The receiver uses those numbers to put segments back in order and to drop duplicates. Your code just reads bytes; it never sees packets.

2 ACKs and retransmission

The receiver sends **acknowledgements**. An ACK is *cumulative*: "ACK 1001" means "I have every byte up to 1000; send me byte 1001 next." If a segment is lost, later segments still arrive, but the receiver can only repeat the same ACK. The sender retransmits the missing segment in one of two ways:

- **Timeout**: no ACK arrived within the retransmission timeout (RTO), which is computed from measured round-trip times (at least 200 ms on Linux, often around 1 s on long paths).
- **Fast retransmit**: three **duplicate ACKs** tell the sender something is missing while later data is still getting through, so it resends right away.

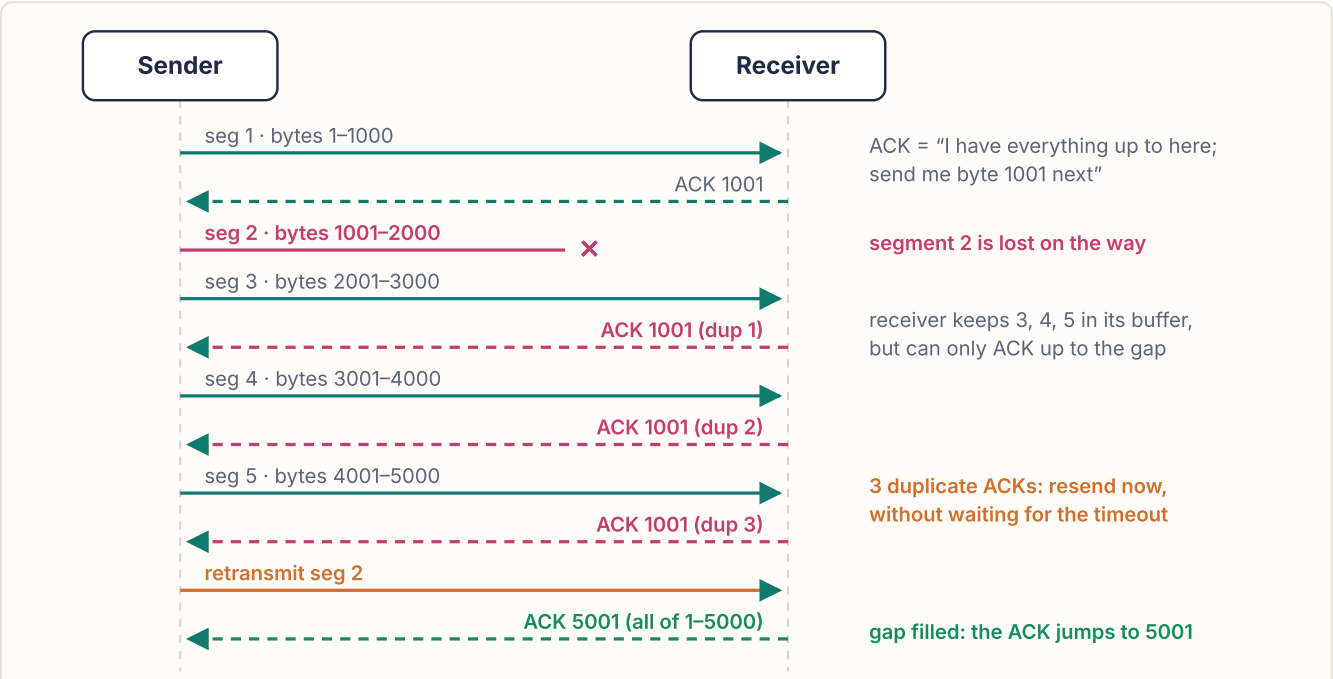


Figure 1. Segment 2 is lost. Segments 3, 4 and 5 each trigger a duplicate "ACK 1001"; on the third, the sender resends segment 2, and the ACK jumps to 5001.

3 Flow control: protect the receiver

Every ACK carries a **receive window**: how much free buffer space the receiver has. The sender may never have more unacknowledged data in flight than that. If the receiving app reads slowly, its buffer fills, the window shrinks and the sender must slow down. If the window reaches **zero**, the sender pauses and probes until the window opens again.

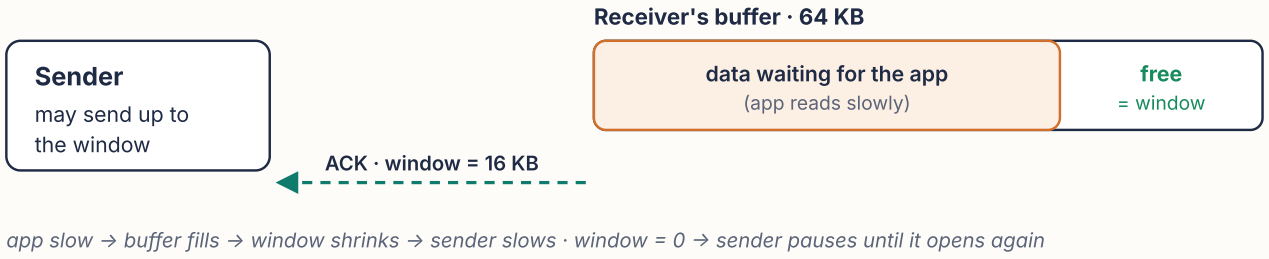


Figure 2. The receive window is the free part of the receiver's buffer. A slow consumer shrinks it, and that slows the sender.

4 Congestion control: protect the network

Flow control knows nothing about the routers in between. **Congestion control** does: the sender keeps a **congestion window** and sends no more than the smaller of the two windows.

- **Slow start.** A new connection starts small (Linux: 10 segments, about 14 KB) and roughly **doubles every round trip**.
- **Back off on loss.** When it detects loss, it cuts its window, then grows again more carefully.
- **CUBIC**, the Linux default, treats packet loss as the signal and cuts to about 70% on each loss. **BBR**, from Google, instead estimates the path's bandwidth and minimum round-trip time and paces sending to match, so random loss doesn't make it back off.

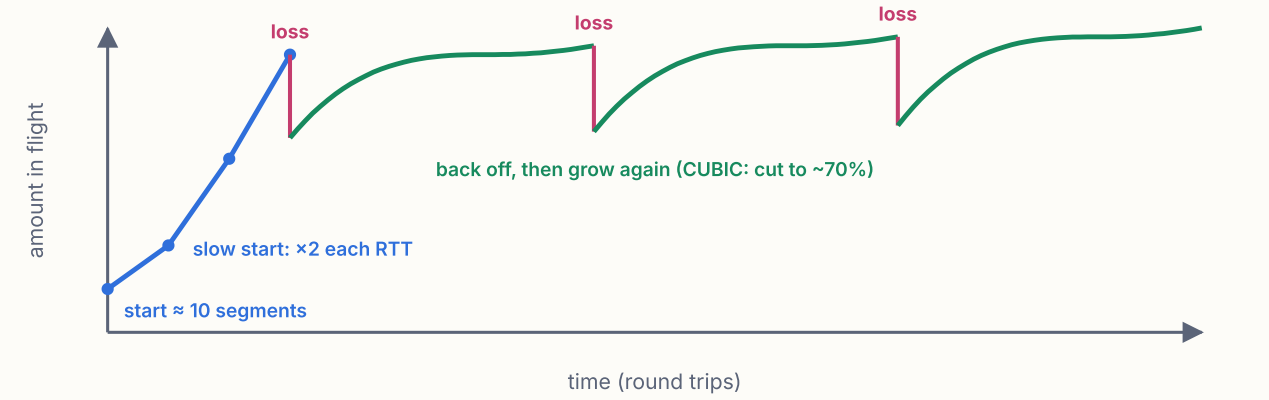


Figure 3. Slow start doubles the window every round trip; each loss cuts it, and it grows back. A short request may finish before slow start ever reaches full speed.

WORKED EXAMPLE: WHY A NEW CONNECTION IS SLOW FOR BIG RESPONSES

A 1 MB response on a fresh connection, 100 ms RTT, starting at 10 segments (~14 KB). Round trips carry about 14, 28, 56, 112, 224, 448 KB: after 6 round trips about 880 KB has been sent, so the 1 MB finishes in the 7th. That is roughly **0.7 s** of transfer time, even on a fast link, plus the handshake. On a warm, reused connection whose window is already large, the same response can take one or two round trips.

5 The price of order: head-of-line blocking

Strict ordering has a cost. Say segment 2 is lost, but 3 and 4 have already arrived. The app can't read them yet: TCP only hands over bytes in order. Everything waits until segment 2 is retransmitted, at least one more round trip. **One lost packet stalls the whole stream.**



TCP delivers bytes strictly in order, so one lost segment stalls everything behind it, even data already received.

Figure 4. Head-of-line blocking: segments 3 and 4 sit in the buffer, ready but unreadable, until segment 2 is resent.

Where you meet it	What happens
HTTP/1.1	One request at a time per connection, so browsers open about 6 connections per host in parallel.
HTTP/2	Many streams multiplexed over one TCP connection. Fixes HTTP-level blocking, but one lost packet still stalls every stream.
HTTP/3	Runs on **QUIC over UDP**, which orders each stream separately, so a loss only stalls its own stream.
Database drivers	Keep connection pools, so queries reuse open, warmed-up connections instead of paying handshakes and slow start.

COMMON MISTAKES

- Mixing up flow control (protects the receiver, receive window) and congestion control (protects the network, congestion window).
- Thinking TCP preserves message boundaries. It is a byte stream; your protocol must frame messages (length prefix, delimiter).
- Assuming HTTP/2 removes head-of-line blocking entirely. It removes it at the HTTP layer, not at the TCP layer.
- Benchmarking with fresh connections and blaming the server for slow-start time.

6 Key terms

Segment: one TCP chunk of the byte stream

Sequence number: position of a byte in the stream

ACK: next byte expected (cumulative)

Fast retransmit: resend after 3 duplicate ACKs

RTO: retransmission timeout

Receive window: receiver's free buffer: flow control

Congestion window: sender's estimate of what the network can take

Slow start: window doubles each RTT from a small start

HOL blocking: one loss holds back later data

7 Interview questions

Q1. How does TCP guarantee delivery? BASIC

Sequence numbers on every byte, cumulative ACKs from the receiver, and retransmission of anything not acknowledged (after a timeout or 3 duplicate ACKs). Flow and congestion control stop it from overwhelming the receiver or the network.

Q2. What is head-of-line blocking? BASIC

TCP delivers bytes strictly in order, so one lost segment holds back everything behind it, even data that has already arrived. The app waits at least one extra round trip for the retransmission.

Q3. Flow control vs congestion control? BASIC

Flow control protects the receiver: the receive window in each ACK says how much buffer is free. Congestion control protects the network: the sender's congestion window grows with slow start and shrinks on loss. The sender uses the smaller of the two.

Q4. A consumer service is slow and the producer's TCP sends stall, though the network is idle. Why? SCENARIO

Flow control. The consumer's receive buffer is full because the app isn't reading, so it advertises a zero window and the sender pauses. Fix the slow reader (more workers, async processing), not the network.

Q5. Why do database drivers and HTTP clients use connection pools? SCENARIO

A new connection pays a handshake (1 RTT, plus TLS) and starts in slow start at about 14 KB per round trip. A pooled connection is already open and its congestion window has grown, so queries start immediately.

Q6. Your HTTP/2 API is slow for mobile users on lossy networks. What could help? SCENARIO

All streams share one TCP connection, so each lost packet stalls every request. HTTP/3 (QUIC over UDP) recovers per stream, and BBR copes better with random loss than loss-based CUBIC.

Q7. Why retransmit after three duplicate ACKs instead of one? TRICKY

Packets can be reordered, which also produces a duplicate ACK or two. Three duplicates are strong evidence of a real loss while later data still flows, so the sender resends without waiting for the much longer timeout.

Q8. You send two 100-byte messages over TCP. The receiver's read() returns 200 bytes at once. Bug? TRICKY

No. TCP is a byte stream with no message boundaries; reads can merge or split writes. The application protocol must frame messages, for example with a length prefix, as HTTP/2, gRPC and database protocols do.

Next: SD-B-004 · TCP Handshake, FIN/RST & TIME_WAIT backendroadmap.com

TCP Handshake, FIN/RST & TIME_WAIT

how connections open, close, and run you out of ports

Watch the 3-min video

> **In one line:** a TCP connection costs a SYN, SYN-ACK, ACK round trip to open and a FIN/ACK exchange per direction to close, and the side that closes first holds its ports in TIME_WAIT for about 60 s, so many short-lived connections to one destination can exhaust the ephemeral ports.

1 The mystery: "cannot assign requested address"

Your service opens a fresh connection to a partner API for every request. Traffic grows, and calls start failing with `EADDRNOTAVAIL: cannot assign requested address`. CPU is fine, memory is fine. You have run out of **ports**. To see why, you need to know how TCP connections open and, more importantly, how they close.

2 Opening: the three-way handshake

Before any data flows, TCP does a **three-way handshake**:

1. The client sends **SYN** with its starting sequence number *x* (chosen at random).
2. The server replies **SYN-ACK**: its own starting number *y*, plus `ack = x + 1`, acknowledging the client's SYN.
3. The client sends **ACK** (`ack = y + 1`). The connection is open.

That costs **one full round trip** before the first byte of request data. At 80 ms RTT, every new connection adds 80 ms (and TLS adds more on top).

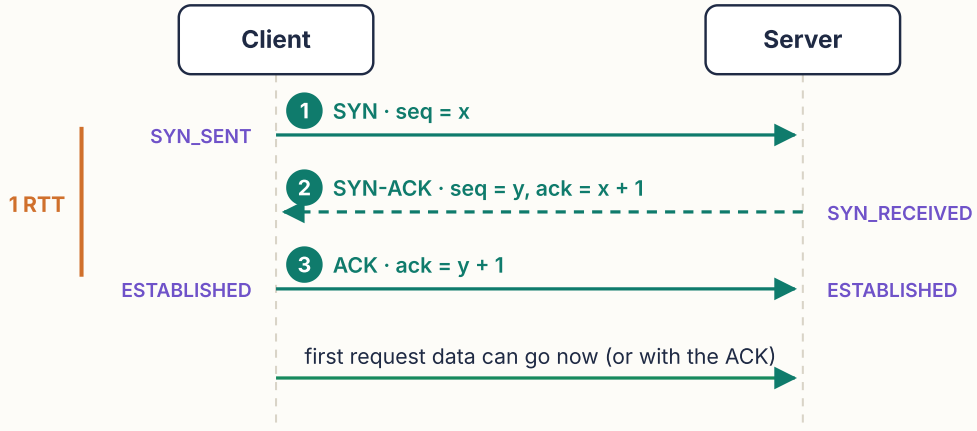
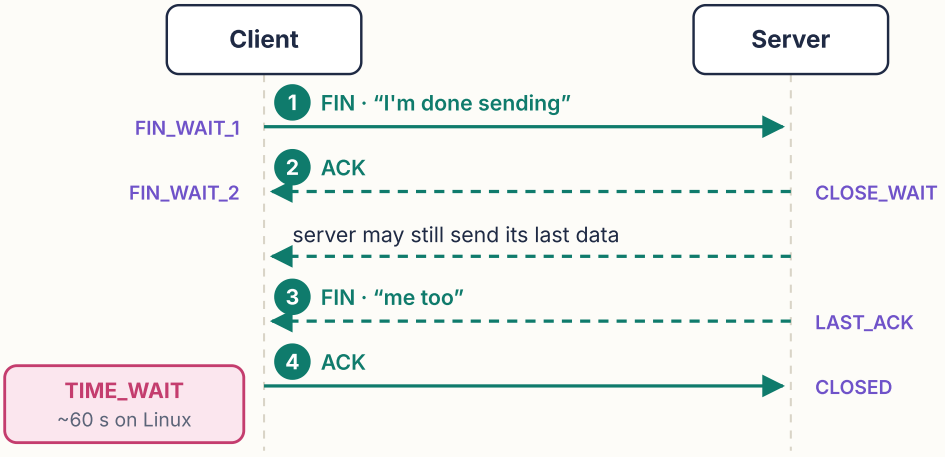


Figure 1. The three-way handshake. Both sides learn each other's starting sequence number; the client can send data right after its ACK.

3 Closing: FIN, ACK, and the abrupt RST

Closing takes four steps, because **each direction closes separately**. One side sends **FIN**: "I'm done sending." The other side ACKs it, finishes sending whatever it still has, then sends its own FIN, which is acknowledged too. (The middle two messages are often combined into one FIN-ACK.)

Sometimes there is no polite goodbye. A **RST** (reset) kills the connection instantly. You get one when you connect to a port nobody is listening on ("connection refused"), when a process crashes or closes a socket with unread data, or when a firewall or load balancer gives up on a connection.



No polite goodbye: a RST resets the connection at once (e.g. SYN to a port with no listener → "connection refused").

Figure 2. The four-way close, with the state each side passes through. The side that sent the first FIN ends in TIME_WAIT.

4 TIME_WAIT and the port math

The side that closes first enters **TIME_WAIT**. It keeps that connection's exact addresses and ports reserved for a while, **about 60 seconds on Linux**. Two reasons: a delayed packet from the old connection must not be mistaken for part of a new one, and if the final ACK is lost, the other side can resend its FIN and still get an answer.

Now the math. Every outgoing connection uses an **ephemeral port** on the client. Linux's default range is 32768–60999, about **28,000 ports**. A connection is identified by source IP, source port, destination IP and destination port, so the limit applies *per destination*:

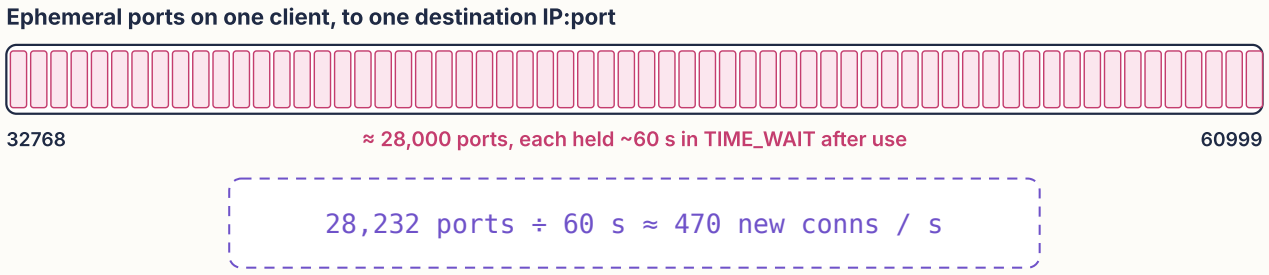


Figure 3. If every connection to the same server is closed by the client after one request, ports return to the pool only after 60 s, capping new connections at about 470 per second.

A **NAT gateway** hits the same wall: every instance behind it shares the gateway's ports for each destination, so a fleet of 50 servers calling one partner API through one NAT IP shares the same budget. (AWS NAT Gateway allows about 55,000 simultaneous connections per destination, per IP address.)

5 Half-open connections

Another failure mode: one side crashes, or a firewall or NAT silently drops an idle connection from its table. The other side still believes the connection is open. It only finds out when it sends data and gets a RST back, or no reply at all until its retransmissions give up, which can take many minutes.

TCP keepalives (an empty probe after a period of idleness; the Linux default waits 2 hours, so tune it to minutes) or **application heartbeats** (ping/pong messages) detect these dead connections early. Set the interval shorter than the idle timeout of any firewall, NAT or load balancer on the path, often 350 s for AWS NAT and NLB.

6 The fix: reuse connections

With **HTTP keep-alive** and **connection pools**, thousands of requests share a few long-lived connections. That removes the handshake cost and the TIME_WAIT pile-up together. If you still need more, spread traffic across **more IP addresses** (more source IPs, or more destination IPs), each with its own 28,000 ports. On Linux, `net.ipv4.tcp_tw_reuse` lets new *outgoing* connections safely reuse ports stuck in TIME_WAIT (it relies on TCP timestamps).

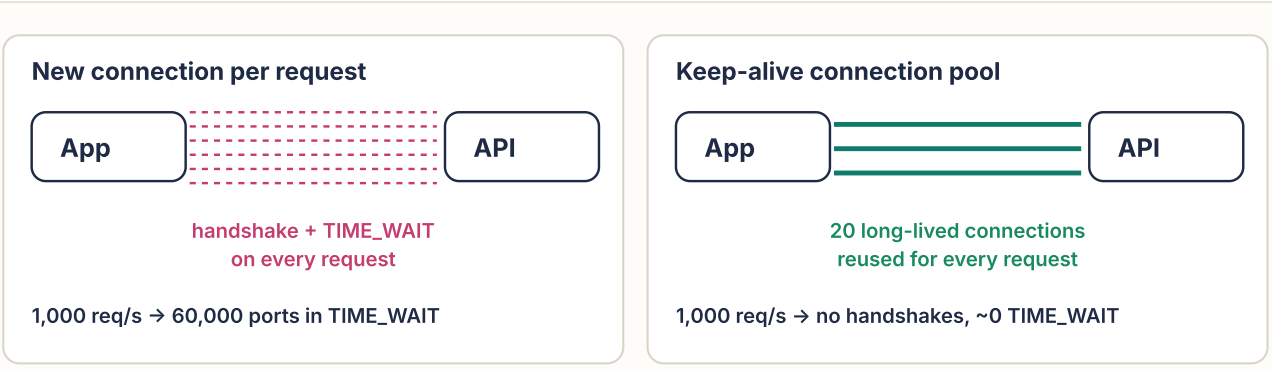


Figure 4. Left: a connection per request pays a handshake each time and leaves a port in TIME_WAIT. Right: a pool of warm connections serves the same load with none of that.

WORKED EXAMPLE

Your service makes **1,000 calls/s** to one partner endpoint, each on a new connection that your side closes. Ports in TIME_WAIT at steady state: 1,000 × 60 s = **60,000**, but only about 28,000 exist. Failures start at about 470 calls/s.

Switch to a keep-alive pool. By Little's Law, 1,000 calls/s at 20 ms each need about 1,000 × 0.02 = **20** connections in use at once. A pool of 30–50 covers bursts, with zero handshakes per call.

COMMON MISTAKES

- Creating a new HTTP client (and connection) per request instead of sharing one.
- Treating many TIME_WAIT sockets on a busy client as a leak. It's normal; it only hurts when it runs out the ports to one destination.
- Piles of **CLOSE_WAIT**, on the other hand, are a bug: the peer closed but your code never closed its socket.
- Turning on the old `tcp_tw_recycle`: it broke clients behind NAT and was removed from Linux in 4.12.

7 Key terms

SYN / SYN-ACK / ACK: the three-way handshake

FIN: “I'm done sending” for one direction

RST: abrupt reset of a connection

TIME_WAIT: closer holds the 4-tuple ~60 s

CLOSE_WAIT: peer closed; local app hasn't yet

Ephemeral port: temporary client-side source port

4-tuple: src IP, src port, dst IP, dst port

Half-open: one side gone, the other thinks it's open

Keepalive: idle probe that detects dead peers

8 Interview questions

Q1. Explain the TCP three-way handshake. BASIC

Client sends SYN with its starting sequence number; server replies SYN-ACK with its own number and an ACK of the client's; client sends ACK. It costs one round trip before data can flow, and both sides now know each other's sequence numbers.

Q2. Why does closing take four messages? BASIC

Each direction closes on its own. One side sends FIN and gets an ACK; the other side can keep sending, then sends its own FIN and gets an ACK. The middle ACK and FIN are often combined.

Q3. Why can TIME_WAIT cause port exhaustion? BASIC

The closing side keeps each connection's 4-tuple reserved for about 60 s. With about 28,000 ephemeral ports per destination, a client can open only about 470 new connections per second to the same server before ports run out.

Q4. Calls to a partner API fail with “cannot assign requested address” at peak, CPU is low. Diagnose and fix. SCENARIO

Port exhaustion: a new connection per call, closed by your side, leaves ports in TIME_WAIT. Check with ss -s or ss -tan state time-wait. Fix with a shared keep-alive pool; then add source IPs or tcp_tw_reuse if needed.

Q5. Users on a long-lived connection stop getting messages after 10 idle minutes, with no error. Why? SCENARIO

A firewall, NAT or load balancer dropped the idle connection (AWS NAT and NLB: 350 s), leaving it half-open. Send keepalives or app heartbeats every minute or so, under the shortest idle timeout on the path, and reconnect on failure.

Q6. A server shows thousands of sockets in CLOSE_WAIT. What's wrong? SCENARIO

The clients closed their side, but the application never called close(), usually a leaked connection on an error path. It's an application bug; restart buys time, the code needs fixing.

Q7. Which side ends up in TIME_WAIT, and how can a server avoid holding them? TRICKY

Whichever side sends the first FIN. Servers usually let clients close idle keep-alive connections, so TIME_WAIT spreads across many client machines. A busy server rarely runs out of ports anyway: every connection has a different client IP or port, so its 4-tuples stay unique; TIME_WAIT only costs it a little memory.

Q8. Why not just set TIME_WAIT to 1 second? TRICKY

It exists so stray delayed packets from an old connection can't corrupt a new one with the same 4-tuple, and so a lost final ACK can be recovered. On Linux it's fixed at 60 s; tcp_tw_reuse (with timestamps) is the safe way to reuse ports for outgoing connections.

Next: SD-B-005 · UDP vs TCP backendroadmap.com

UDP vs TCP

when speed beats reliability

Watch the 3-min video

In one line: UDP sends independent datagrams with no handshake, ACKs, retransmission or ordering; choose it when fresh data beats complete data (voice, video, games), for tiny exchanges like DNS, or as the base for protocols like QUIC, and choose TCP when losing a byte would be a bug.

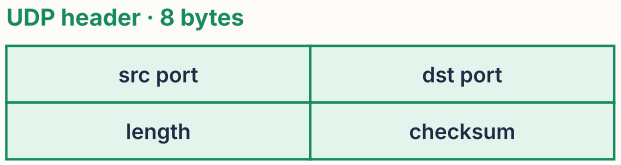
1 UDP: the simplest transport

You are on a video call and the connection hiccups for a split second. Do you want the call to freeze while every lost packet is resent? No. You would rather lose a few milliseconds of audio and keep talking. That is exactly what UDP gives you.

UDP sends independent messages called **datagrams**. There is no handshake, no connection, no acknowledgements, no retransmission and no ordering. Each datagram may arrive, arrive late, arrive out of order, arrive twice, or not arrive at all. Unlike TCP's byte stream, each datagram keeps its boundaries: one send is one receive.

2 What you get in return

- **Tiny overhead.** The UDP header is just **8 bytes**, against at least 20 for TCP.
- **No setup round trip.** The very first packet carries data.
- **No head-of-line blocking.** A lost datagram never holds up the next one.
- **No per-connection state** on the server, so one socket can serve huge numbers of clients.
- **Multicast and broadcast.** One packet to many receivers, which TCP cannot do.



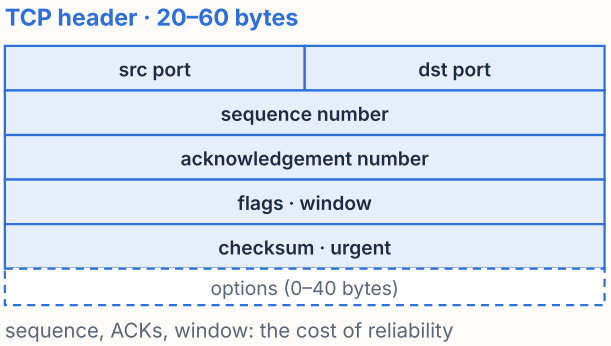


Figure 1. UDP adds only ports, a length and a checksum to IP. TCP's extra fields are what make reliability, ordering and flow control possible.



Figure 2. TCP waits a round trip before sending and holds later data behind a loss. UDP sends at once and hands every datagram that arrives straight to the app.

3 Real-time voice: why resending is pointless

Voice apps send audio in small packets, about one every **20 ms** (50 packets per second). If one is lost, resending it is pointless: by the time it arrives, at least one more round trip later, that moment of the conversation has already played.

So voice apps use UDP, keep a small **jitter buffer** (a few tens of milliseconds) to smooth out uneven arrival times, and simply **hide the gap** by repeating or blending the neighbouring audio. Losing 1–2% of 20 ms packets is barely noticeable; freezing for 150 ms on every loss is not.



Figure 3. A voice stream over UDP. The lost packet is concealed, not resent; the jitter buffer absorbs small variations in arrival time.

4 Choosing between them

Choose UDP when...	Choose TCP when...
Freshness beats completeness: voice and video calls, live broadcasts, online games	**Every byte must arrive, in order**: web pages and APIs, file transfers, database connections, payments
Tiny request/response: DNS (one packet out, one back), NTP, DHCP, metrics (StatsD)	**Large or long transfers** where you want congestion control and flow control for free
Building a new protocol with its own reliability: QUIC (HTTP/3), WebRTC media	**Losing data would be a bug**, and you don't want to rebuild reliability yourself

THE RULE OF THUMB

If losing data would be a bug, use TCP. If late data is worthless, use UDP. If you need reliability *and* UDP's freedoms (no head-of-line blocking, faster setup), use a protocol built on UDP such as QUIC rather than writing your own.

5 How a game handles lost UDP packets

A multiplayer game server sends the **whole relevant game state** many times per second (often 20–60 updates per second, up to 128 in competitive shooters). If one update is lost, the next one replaces it anyway, so there is nothing to resend.

The client **predicts** movement (its own player reacts instantly, other players are extrapolated) and **smooths** between updates, so a missing update is invisible. For the few things that must arrive, such as a purchase, a chat message or a match result, the game adds **its own acknowledgements and resends**, just for those messages, usually with an ID so a duplicate is ignored.

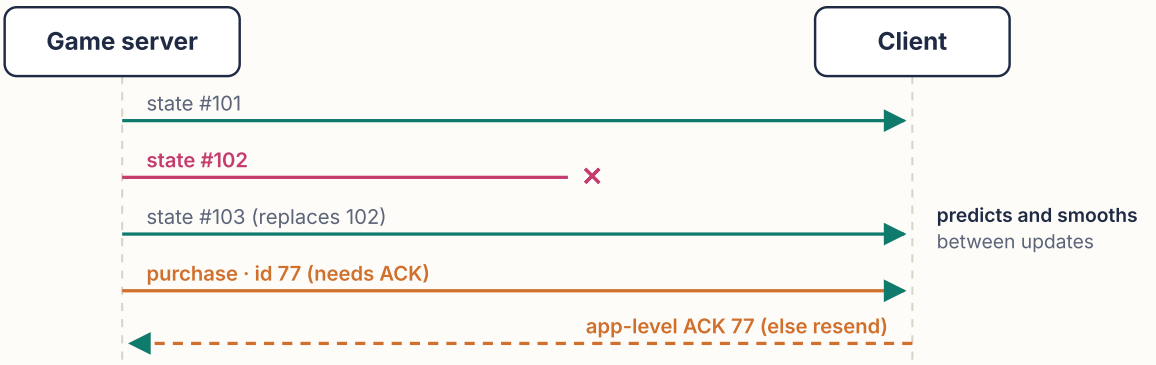


Figure 4. State updates are fire-and-forget: update 103 replaces the lost 102. Only the purchase is tracked with an application-level ACK and resent if needed.

WORKED EXAMPLE: WHAT DOES A LOSS COST?

Voice at 20 ms per packet, 150 ms round trip, 1% loss.

- **TCP:** each loss stalls the stream for at least one RTT plus detection time, about **150–300 ms** of silence, then a burst of stale audio. At 50 packets/s, that is roughly one stall every 2 seconds.
- **UDP:** each loss is a **20 ms** gap, concealed by the decoder. Total delay stays at the network delay plus a ~40 ms jitter buffer.

COMMON MISTAKES

- Saying UDP is “faster” in general. It is faster to start and never stalls on loss, but the bytes travel at the same speed.
- Sending large UDP payloads. Anything bigger than the path MTU (about 1,200–1,400 bytes to be safe) gets fragmented, and losing one fragment loses the whole datagram.
- Forgetting congestion: UDP has no congestion control, so a careless sender can flood a network. Rate-limit it yourself.
- Reinventing TCP on UDP for an API. Use TCP, or QUIC if you need what UDP offers.

6 Key terms

Datagram: one self-contained UDP message

Jitter buffer: short delay that evens out arrival

Tick rate: game state updates per second

Connectionless: no handshake, no shared state

Loss concealment: hiding a missing audio or video frame

Client prediction: simulating movement between updates

Jitter: variation in packet arrival times

Multicast: one packet delivered to many receivers

QUIC: reliable, encrypted transport on UDP

7 Interview questions

Q1. When would you choose UDP over TCP? BASIC

When low latency matters more than completeness (voice, video, games), for tiny exchanges like DNS where one packet each way is enough, or as the base for a protocol like QUIC that adds its own reliability. Otherwise, TCP.

Q2. What does UDP give up compared with TCP, and what does it gain? BASIC

It gives up the handshake, ACKs, retransmission, ordering, flow and congestion control. It gains an 8-byte header (vs 20+), no setup round trip, no head-of-line blocking, message boundaries and multicast.

Q3. How does a game handle lost UDP packets? BASIC

It sends frequent full state updates, so the next update replaces a lost one; the client predicts and smooths between updates; and only critical events (purchases, chat) get app-level ACKs and resends, with IDs to drop duplicates.

Q4. You're designing a video-calling service. TCP or UDP for the media, and what about signalling? SCENARIO

Media over UDP (RTP via WebRTC) with a jitter buffer and loss concealment, since late frames are worthless. Signalling (join, leave, chat) over a reliable channel such as WebSocket over TCP. Fall back to TURN over TCP or TLS only when UDP is blocked.

Q5. Your service pushes metrics to a collector 10,000 times a second. Would UDP be OK? SCENARIO

Yes, StatsD does exactly this: fire-and-forget over UDP, so a slow collector never blocks the app. Losing 0.1% of samples barely moves an aggregate. For billing-grade counts, use TCP or a queue instead.

Q6. Why does DNS use UDP, and when does it switch to TCP? SCENARIO

A query and its answer usually fit in one packet each, so UDP avoids a handshake and server state. If the answer is too large (truncated flag set), or for zone transfers, it retries over TCP.

Q7. If UDP is unreliable, why is HTTP/3 built on it? TRICKY

QUIC adds reliability, congestion control and TLS 1.3 on top of UDP, but tracks each stream separately, so one loss doesn't stall other streams. It also combines the transport and TLS handshakes into one round trip, and it runs in user space, so it can evolve without OS kernel changes.

Q8. A client sends one 6 KB UDP datagram and it often never arrives, while 1 KB datagrams do. Why? TRICKY

6 KB exceeds the typical 1,500-byte MTU, so IP fragments it into about 5 pieces. Lose any one and the whole datagram is dropped, and some firewalls drop fragments outright. Keep UDP payloads under about 1,200 bytes.

Next: SD-B-006 · IP Addresses & CIDR backendroadmap.com

IP Addresses, Subnets & CIDR

design the address space before you need it

Watch the 3-min video

In one line: an IPv4 address is 32 bits; CIDR (`/16`, `/24`) says how many of those bits are the fixed network part; you carve a VPC's block into subnets per zone and keep databases in private subnets with no route from the internet.

1 What an IP address is

Every machine on a network needs an address. In the cloud you design that address space yourself, and a bad plan is painful to change later, so it is worth getting right on day one.

An **IPv4** address is **32 bits**, written as four numbers from 0 to 255 (each one 8 bits, an *octet*), for example `10.0.1.25`. That gives 2^{32} ≈ **4.3 billion** addresses, and the internet ran out of fresh ones years ago. **IPv6** uses **128 bits**, written in hexadecimal (like `2001:db8::1`), so addresses are effectively unlimited.

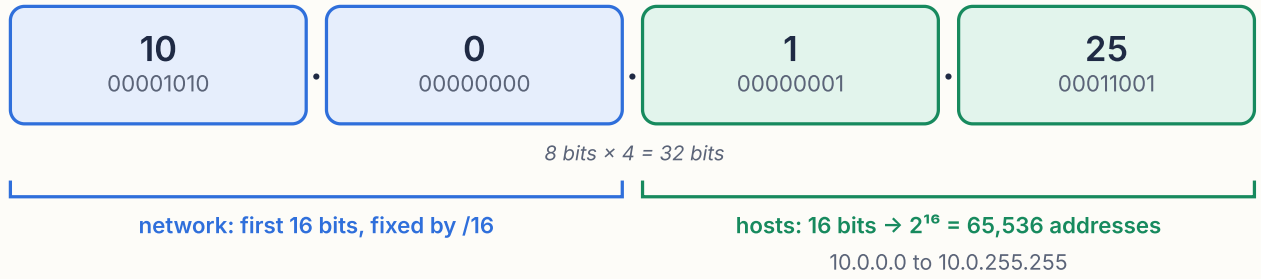


Figure 1. `10.0.1.25` is four 8-bit numbers. With `/16`, the first 16 bits are the network; the last 16 bits number the hosts.

2 Private ranges and NAT

Three ranges are reserved for internal networks (RFC 1918). They are not routed on the public internet, so every company can reuse them:

Range	CIDR	Size
`10.0.0.0 – 10.255.255.255`	`10.0.0.0/8`	16.7 million addresses: the usual choice for cloud VPCs
`172.16.0.0 – 172.31.255.255`	`172.16.0.0/12`	1 million. Only 172.**16** to 172.**31**; 172.15 and 172.32 are public
`192.168.0.0 – 192.168.255.255`	`192.168.0.0/16`	65,536: typical for home and office networks

To reach the internet, private machines go through **NAT** (network address translation). NAT swaps the private source address for a shared public one, remembers the mapping, and sends replies back to the right machine.

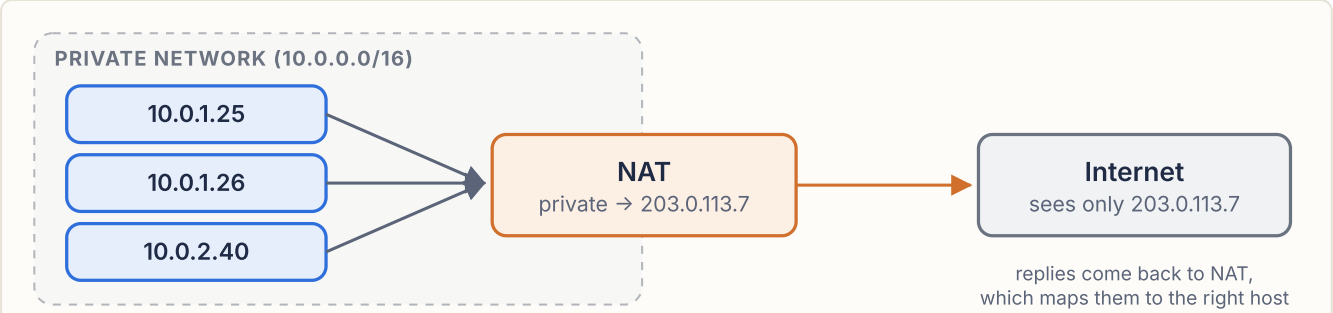


Figure 2. Many private hosts share one public address through NAT. The internet never sees the 10.x addresses.

3 CIDR: reading a block

CIDR notation (said "cider") describes a block of addresses as *start address / prefix length*. In `10.0.0.0/16`, the **/16** means the first 16 bits are the network part and are fixed. The other 32 − 16 = 16 bits are for hosts, so the block holds 2^{16} = **65,536** addresses, from `10.0.0.0` to `10.0.255.255`.

$$\text{addresses in a /n block} = 2^{(32 - n)}$$

A **bigger number after the slash means a smaller block**. Each extra bit halves it:

Prefix	Host bits	Addresses	Example	Usable on AWS (minus 5)
`/16`	16	65,536	`10.0.0.0 – 10.0.255.255`	a whole VPC
`/20`	12	4,096	`10.0.16.0 – 10.0.31.255`	4,091
`/24`	8	256	`10.0.1.0 – 10.0.1.255`	251
`/28`	4	16	`10.0.1.0 – 10.0.1.15`	11 (smallest AWS subnet)

4 Subnets: splitting the block

You split a big block into smaller **subnets**. Carve the `/16` into `/24`s and you get 2^8 = **256** of them: `10.0.0.0/24`, `10.0.1.0/24`, `10.0.2.0/24`, and so on up to `10.0.255.0/24`. Subnets don't have to be equal: you could use a few `/20`s for big app tiers and `/24`s for the rest, as long as they don't overlap.

In the cloud, **each subnet lives in exactly one availability zone**, and the provider reserves a few addresses in each. On AWS that is **five**: the network address, the next three (VPC router, DNS, reserved for future use) and the last (broadcast). So a `/24` gives 251 usable addresses.

5 Designing a real VPC

The example: a VPC with `10.0.0.0/16` across two availability zones, with three tiers of subnets in each zone.

- **Public subnets** hold only the load balancers (and the NAT gateways). Their route table sends `0.0.0.0/0` to the **internet gateway**.
- **Private app subnets** hold your services. Their default route goes to a **NAT gateway**: outbound only, for things like calling a payment API or pulling updates. Nobody on the internet can start a connection to them.
- **Private data subnets** hold the databases. Only the app tier can talk to them.

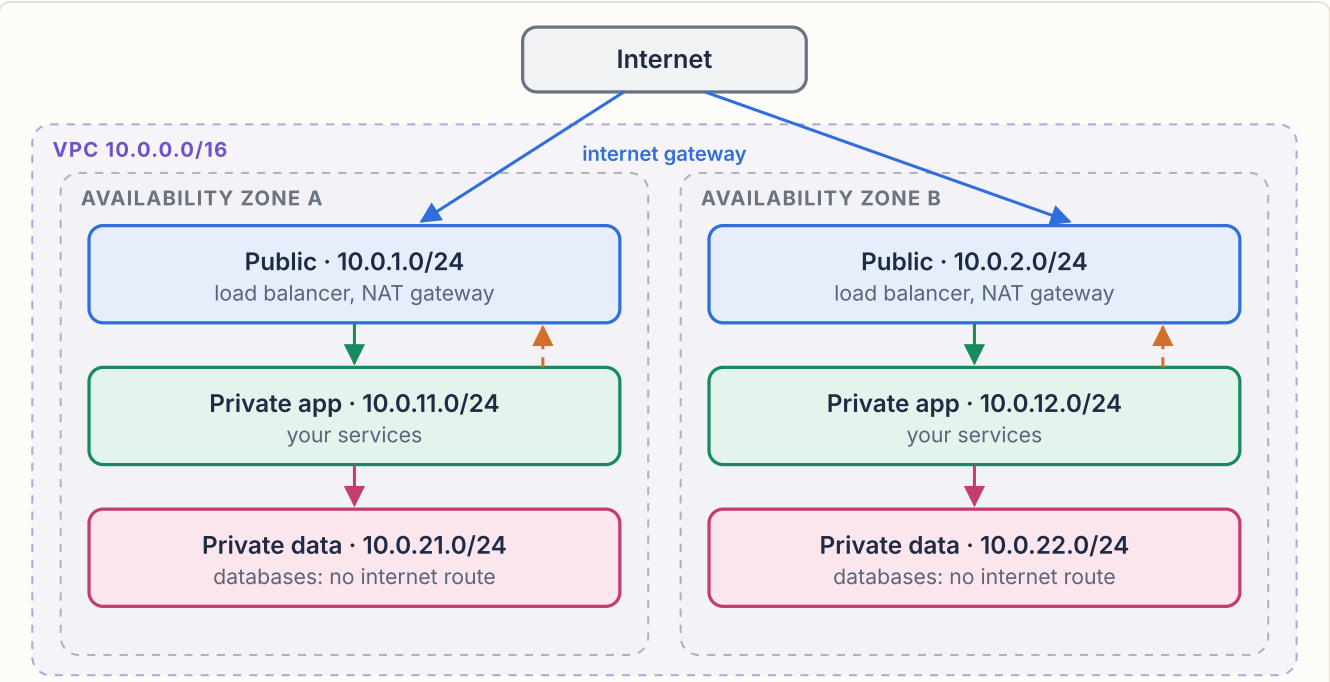


Figure 3. A two-zone VPC. Traffic enters only at the load balancers in the public subnets; apps go out through the NAT gateway (dashed orange arrows, outbound only); the data subnets have no internet route at all.

Why put databases in private subnets? Because there is simply no route from the internet to them. Even if a password leaks or a firewall rule is wrong, attackers cannot reach the database directly. That is **defense in depth**: routing is one layer, security groups are another, credentials a third.

WORKED EXAMPLE: THE ADDRESS PLAN

VPC `10.0.0.0/16` (65,536 addresses). Use `/24` subnets: public `10.0.1.0/24` and `10.0.2.0/24`, app `10.0.11.0/24` and `10.0.12.0/24`, data `10.0.21.0/24` and `10.0.22.0/24`. That is 6 × 256 = 1,536 addresses used, 6 × 251 = **1,506** usable on AWS, and over 97% of the block left for a third zone or new tiers. If the app tier will run thousands of containers that each take an IP, make the app subnets `/20` (4,091 usable each) instead.

PLAN AHEAD

Give the VPC a large block (a `/16`) and never overlap it with other VPCs or your office network. Peering, VPNs and transit gateways cannot route between two networks that both claim `10.0.0.0/16`, and renumbering a live VPC means rebuilding it. A simple fix: give each VPC its own slice, such as `10.0.0.0/16` for prod, `10.1.0.0/16` for staging, `10.2.0.0/16` for dev.

COMMON MISTAKES

- Reading `/24` as "24 hosts". It is 24 *network* bits, leaving 8 host bits: 256 addresses.
- Thinking all of 172.x is private. Only 172.16.0.0 to 172.31.255.255 is.
- Using the default small VPC or tiny subnets, then running out of IPs when containers or Lambda functions each take one.
- Overlapping VPC ranges with each other or with the office network.
- Putting databases in public subnets "just for now".

6 Key terms

IPv4 / IPv6: 32-bit / 128-bit addresses

Prefix length: number of fixed network bits

NAT: swaps private source IPs for a public one

Octet: one of the four 8-bit numbers in IPv4

Subnet: a slice of the VPC block, in one zone

Internet gateway: VPC route target for public subnets

CIDR: block = start address / prefix length

Private range: 10/8, 172.16/12, 192.168/16

Defense in depth: several independent layers of protection

7 Interview questions

Q1. What does `10.0.0.0/16` mean? BASIC

The first 16 bits (`10.0`) are fixed as the network, leaving 16 bits for hosts: 2^{16} = 65,536 addresses, from 10.0.0.0 to 10.0.255.255.

Q2. How many addresses are in a /24, and how many /24s fit in a /16? BASIC

A /24 has 32 − 24 = 8 host bits, so 256 addresses (251 usable on AWS, 254 on a classic network after network and broadcast). A /16 holds 2^8 = 256 of them.

Q3. What are the private IPv4 ranges, and why do they exist? BASIC

10.0.0.0/8, 172.16.0.0/12 and 192.168.0.0/16. They are not routed on the internet, so every organisation can reuse them internally; NAT translates them to a public address to go out.

Q4. Design the subnets for a three-tier app in two availability zones. SCENARIO

VPC 10.0.0.0/16. Per zone: a public /24 for load balancers and the NAT gateway (route 0.0.0.0/0 to the internet gateway), a private app subnet (default route to NAT), and a private data subnet with no internet route. Six subnets, each in one zone.

Q5. Why put databases in private subnets if they already have passwords and security groups? SCENARIO

Defense in depth. With no route from the internet, a leaked password or a wrong firewall rule is still not enough for an attacker to connect. Only the app tier can reach the database.

Q6. Your company wants to peer a new VPC with an existing one and the office VPN. What do you check first? SCENARIO

That the CIDR blocks don't overlap. If the new VPC uses 10.0.0.0/16 and so does the old one, routing between them is impossible without renumbering; pick, say, 10.1.0.0/16.

Q7. A /28 subnet on AWS: how many instances can you launch? TRICKY

A /28 has 2^4 = 16 addresses. AWS reserves 5, so 11 usable. It is the smallest subnet AWS allows, and it fills up fast.

Q8. Do instances in a private app subnet need public IPs to download updates? TRICKY

No. They send traffic to the NAT gateway in a public subnet, which uses its own public IP. Connections can go out and replies come back, but nothing on the internet can open a connection in.

Next: SD-B-007 · Ports, Sockets & Routing backendroadmap.com

Ports, Sockets & Routing

how one port serves 100,000 clients, and how packets find it

Watch the 3-min video

In one line: the IP finds the machine and the port finds the process; every TCP connection gets its own socket, named by a unique 5-tuple, so one port can hold thousands of connections; and routers forward packets hop by hop using longest prefix match.

1 Ports: which process on the machine

One server, one port: 443. Yet it holds **100,000 live WebSocket connections** at the same time. How? The answer is in how ports and sockets really work.

An **IP address** finds the machine. A **port** finds the process on that machine. Ports are **16-bit** numbers, from 0 to 65,535. Servers *listen* on well-known ports: **443** for HTTPS, **22** for SSH, **5432** for Postgres, **6379** for Redis (and 80 for HTTP, 53 for DNS). Clients use a random, temporary port on their side, called an **ephemeral port**, picked by the OS for each outgoing connection (on Linux, from 32768 to 60999 by default).

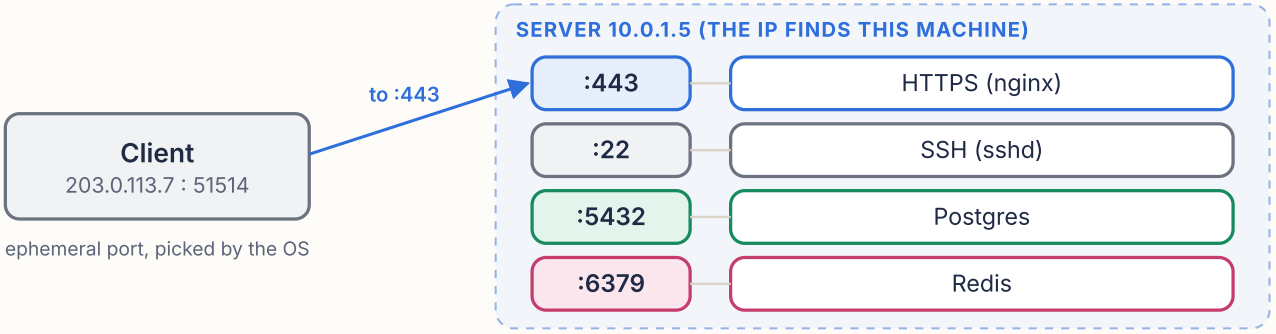


Figure 1. The client reaches 10.0.1.5 by IP; port 443 selects the web server process. The client's own port, 51514, was chosen by its OS.

2 Sockets and the 5-tuple

A **socket** is one endpoint of communication: an IP address, a port and a protocol, such as TCP. Your program asks the OS for a socket, then reads and writes through it, much like a file. On Linux it really is a file descriptor.

A TCP connection is identified by five values, the **5-tuple**:

```
( protocol, source IP, source port, destination IP, destination port )
```

Change any one of them and it is a different connection. That is why a TCP socket and a UDP socket can both use port 443 on the same machine, and why two tabs in your browser can open two separate connections to the same server: they get different ephemeral ports.

3 How one port holds 100,000 connections

The server has **one listening socket** on port 443. Each time a client connects, the OS creates a **brand new socket** for that connection and hands it to the program (the `accept()` call). All these sockets share local port 443, but each has a different client IP and port, so every 5-tuple is unique. Ports are not used up on the server side at all.

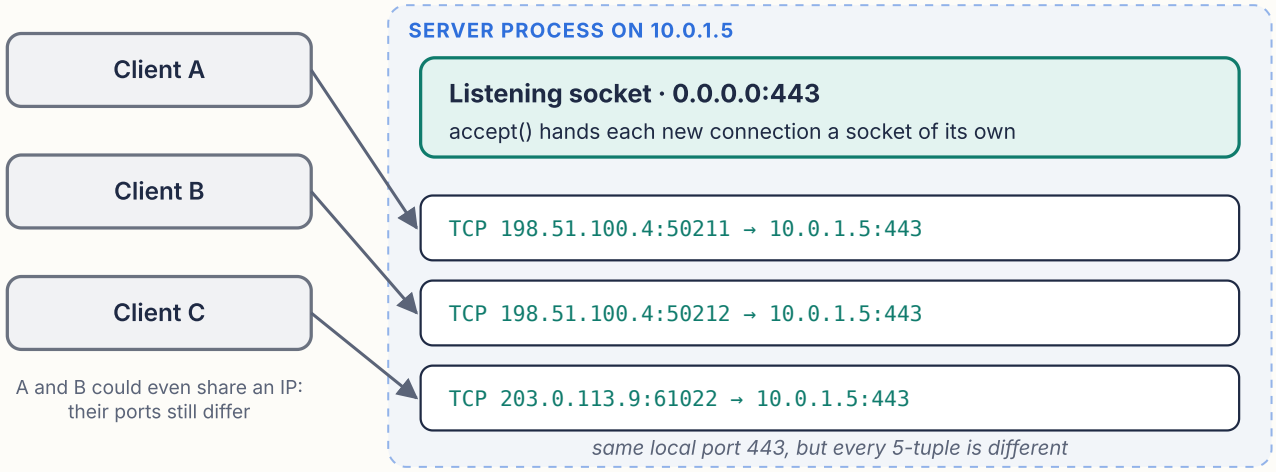


Figure 2. One listening socket on :443 accepts connections; each accepted connection becomes its own socket, told apart by the client's IP and port.

The real limits are **memory** and **file descriptors**, not ports. Every socket is a file descriptor, and many systems default to 1,024 open files per process. Every connection also costs kernel buffers plus your app's own state.

WORKED EXAMPLE: 100K WEBSOCKETS ON ONE BOX

File descriptors: 100,000 connections need 100,000 descriptors, plus some for logs and upstream connections. Raise the open-file limit (for example `ulimit -n` or systemd `LimitNOFILE`) to about 200,000.

Memory: budget per connection. If kernel buffers plus app state come to about 20 KB each, 100,000 × 20 KB ≈ **2 GB** of RAM. At 100 KB each it is 10 GB, which decides your instance size. Then load-test, because idle connections are cheap but busy ones also cost CPU.

4 Routing: how packets find the server

Every machine and router has a **route table**. For each packet, it picks the **most specific route** that matches the destination address. That rule is called **longest prefix match**: a `/24` beats a `/16`, which beats a `/0`. Addresses on the local network are delivered directly. Everything else goes to the **default gateway**, the `0.0.0.0/0` route, which matches any address.

Routers forward packets **hop by hop**. Each one looks only at the destination IP, checks its own table and passes the packet one step closer. No single router knows the full path. (Each hop also lowers the packet's TTL counter by one, so a routing loop can't circle forever; `traceroute` uses this to list the hops.)

Route table of a private app subnet

Destination	Target	Prefix bits
10.0.0.0/16	local (inside the VPC)	16
10.1.0.0/16	**peering link to VPC 2**	16
0.0.0.0/0	NAT gateway (default)	0

Packet to 10.1.4.20

matches /16 and /0: the /16 is more specific, so it goes over peering

8.8.8.8 matches only 0.0.0.0/0 → NAT gateway. 10.0.2.15 matches 10.0.0.0/16 → delivered locally. 0.0.0.0/0 matches every address, so it is the route of last resort: the default gateway.

Figure 3. Longest prefix match. The packet matches two routes; the one with more fixed bits wins.

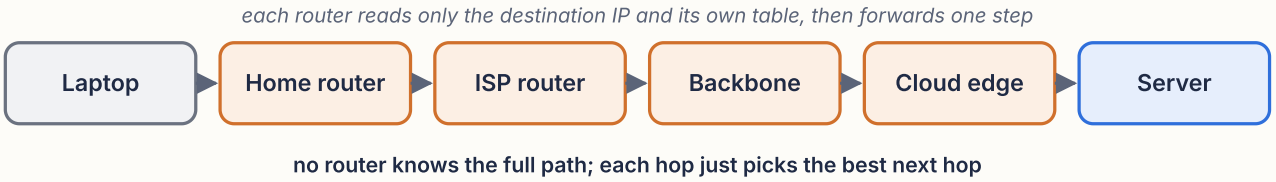


Figure 4. A request crosses several routers. Each makes a local decision; together they deliver the packet.

A **cloud VPC works the same way**. Each subnet's route table keeps local traffic (`10.0.0.0/16 → local`) inside the VPC, and sends `0.0.0.0/0` to an **internet gateway** (public subnets) or a **NAT gateway** (private subnets).

COMMON MISTAKES

- Thinking a server runs out of ports as clients connect. It doesn't: the client side picks the varying port.
- Leaving the default 1,024 open-file limit on a server meant for many connections: it fails with "too many open files".
- Sizing a WebSocket fleet by CPU only; memory per connection is usually the limit.
- Assuming the first matching route wins. The most specific one wins, whatever order the table is in.

5 Key terms

Port: 16-bit number that picks the process

5-tuple: protocol, src IP, src port, dst IP, dst port

Route table: destination prefix → next hop

Ephemeral port: temporary client-side port

Listening socket: waits on a port, accepts connections

Longest prefix match: the most specific route wins

Socket: endpoint: IP + port + protocol

File descriptor: OS handle for an open file or socket

Default gateway: the 0.0.0.0/0 route

6 Interview questions

Q1. What is a socket? BASIC

An endpoint defined by an IP address, a port and a protocol, which a program uses to send and receive data. The OS gives it to the program as a handle (a file descriptor) to read and write.

Q2. What's the difference between an IP address and a port? BASIC

The IP address finds the machine; the port (0 to 65,535) finds the process on it. 10.0.1.5:5432 is Postgres on that host, 10.0.1.5:6379 is Redis.

Q3. How can one server port accept thousands of connections? BASIC

Each connection gets its own socket, identified by the full 5-tuple. All share local port 443, but the client's IP and port make each one unique. The limits are memory and file descriptors.

Q4. You need one box to hold 100k WebSocket connections. What do you change? SCENARIO

Raise the open-file limit well above 100k (say 200k), budget memory per connection (100k × 20 KB ≈ 2 GB), tune kernel buffers if needed, and load-test. Ports are not the constraint.

Q5. A packet for 10.1.4.20 hits a table with 10.0.0.0/16 local, 10.1.0.0/16 peering, and 0.0.0.0/0 NAT. Where does it go? SCENARIO

Peering. It matches 10.1.0.0/16 and 0.0.0.0/0; longest prefix match picks the /16, the more specific route.

Q6. An app in a private subnet can't reach an external API. What do you check in routing? SCENARIO

That the subnet's route table has 0.0.0.0/0 pointing at a NAT gateway, that the NAT gateway sits in a public subnet whose 0.0.0.0/0 goes to the internet gateway, then security groups and network ACLs.

Q7. Where can ports actually run out? TRICKY

On the client side. One client IP talking to one server IP:port can only open as many connections as it has ephemeral ports, about 28,000 with Linux defaults. Proxies and load balancers hit this; fix it with more source IPs, more backend IPs, a wider port range, or connection reuse.

Q8. Do routers know the full path to a destination? TRICKY

No. Each router looks only at the destination IP, picks the best next hop from its own table and forwards the packet. The full path emerges from many local decisions, and it can change between packets.

Next: SD-B-008 · How DNS Resolution Works backendroadmap.com

How DNS Resolution Works: Resolvers, Records, TTL

why half your users still hit the old server

Watch the 3-min video

> **In one line:** your machine asks a recursive resolver, which walks root → TLD → authoritative on a cache miss; every answer is cached for its TTL, so lower the TTL before a migration and changes spread in minutes instead of hours.

1 The problem: DNS remembers

You move `api.example.com` to a new load balancer and update DNS. An hour later, half your users are still hitting the old one. Nothing is broken. It is DNS caching, doing exactly what you told it to. To control it, you need to know how a lookup really works.

DNS turns names into IP addresses. Your laptop does not search the internet itself. It asks a **recursive resolver**, usually run by your ISP, your cloud provider (inside a VPC, the provider's resolver), or a public service like `1.1.1.1` or `8.8.8.8`. The resolver does the hard work for you, and **caches** the answers so the next user gets them instantly.

2 The lookup, step by step

On a **cold cache**, the resolver walks down the naming tree, from right to left in the name:

1. It asks a **root server**: who handles `.com`? The root points it to the `.com` **TLD** (top-level domain) servers.
2. It asks a TLD server: who handles `example.com`? The TLD points it to example.com's **authoritative name servers**.
3. It asks the authoritative server, which gives the real answer: the IP address of `api.example.com`.

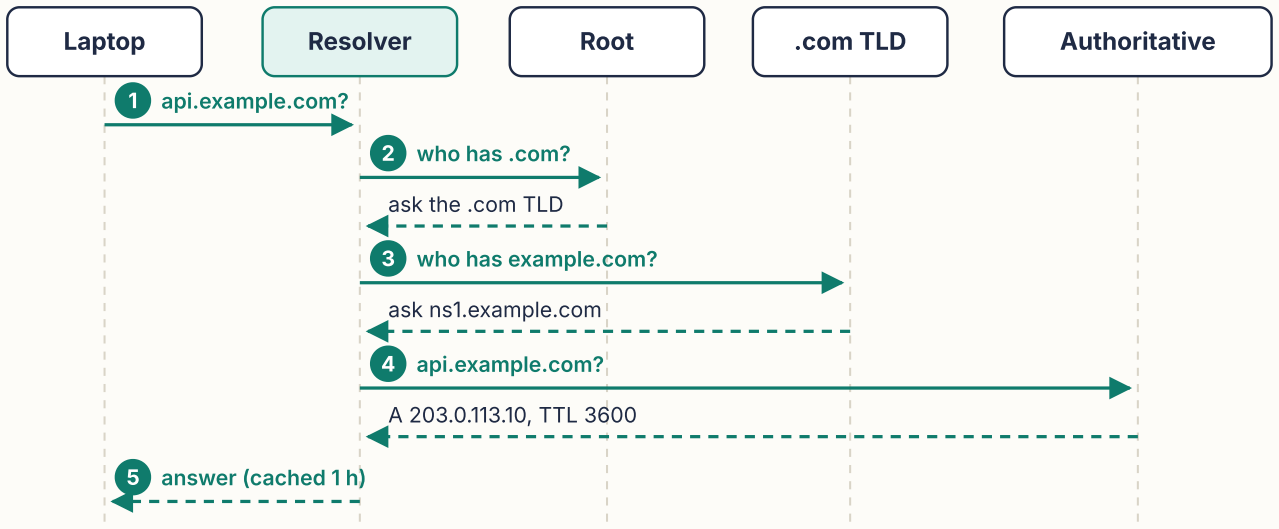


Figure 1. A cold-cache lookup. The laptop asks once; the resolver does every step and caches the answer for its TTL.

Root and TLD servers only *refer* the resolver onwards; only the authoritative server holds the records. On a warm cache most of this is skipped: the resolver already knows the `.com` and `example.com` servers, so a lookup is often one round trip, or zero.

3 Record types to know

Type	What it does	Example
A	Name → IPv4 address	`api.example.com A 203.0.113.10`
AAAA	Name → IPv6 address (“quad A”)	`api.example.com AAAA 2001:db8::10`
CNAME	Alias: points one name to another *name*, which is then resolved	`www CNAME my-lb.elb.amazonaws.com`
MX	Where email for the domain goes, with a priority	`example.com MX 10 mail.example.com`
TXT	Free text, often domain verification or email policy (SPF)	`example.com TXT "v=spf1 …"`
NS	Names the authoritative servers for a zone	`example.com NS ns1.example.com`

4 TTL and caching

Every record has a **TTL** (time to live) in **seconds**. Resolvers, operating systems and browsers all cache the answer until it expires. With a TTL of **3600**, a resolver may keep serving the old IP for **up to an hour** after you change it. (This DNS TTL is a cache lifetime; it has nothing to do with the hop counter called TTL in IP packets.)

Even failed lookups get cached. That is **negative caching**: if someone looked up a name before you created it, the “does not exist” answer is cached too (for a time set in the zone's SOA record), so the new name can keep failing for a while.

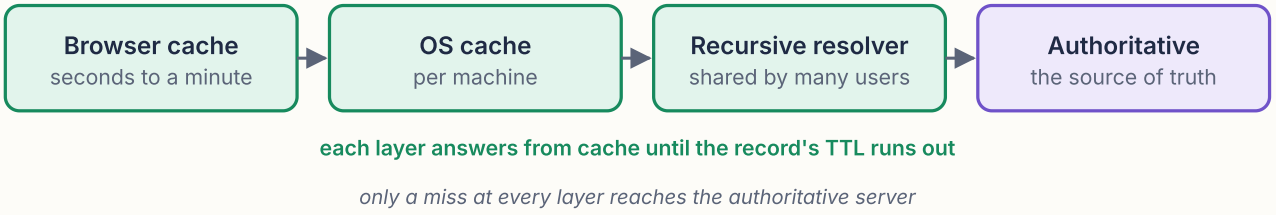


Figure 2. Answers are cached at several layers. A change reaches a user only when every layer between them and the authoritative server has expired its copy.

5 Migrating a record safely

Here is how to move `api.example.com` to a new load balancer with almost no stale traffic:

1. **A day or two ahead**, lower the TTL from 3600 to **60**.
2. **Wait at least one old TTL** (here, one hour), so every cache drops its 1-hour copy and picks up the short value.
3. **Switch the record** to the new load balancer. Within about a minute, most clients follow.
4. **Keep the old load balancer running** for a while (hours to days), because some clients and libraries ignore TTLs, and open connections stay where they are.
5. **Once stable, raise the TTL** again.

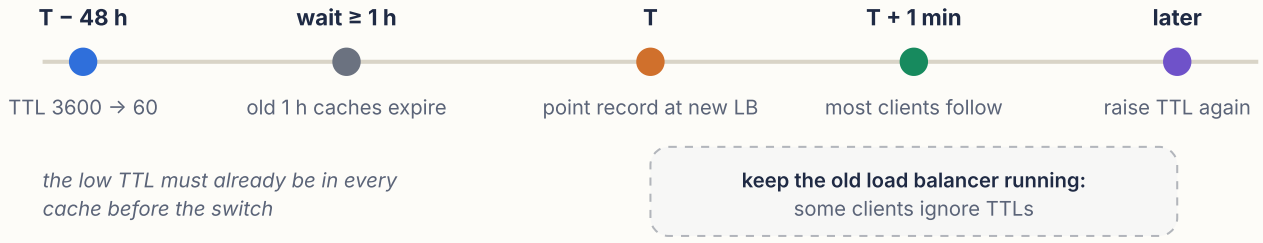


Figure 3. The safe migration. Lowering the TTL only helps once the old, long TTL has expired everywhere, so do it well before the switch.

WORKED EXAMPLE: WHAT GOES WRONG WITHOUT STEP 1

TTL is 3600. You switch the record at 10:00. A resolver that cached the answer at 09:59 keeps serving the old IP until **10:59**, and every user behind it hits the old load balancer for an hour. Lowering the TTL to 60 *at* 10:00 doesn't help them: they won't ask again until 10:59. Lower it to 60 the day before, and the worst case after the switch is about **60 seconds** for well-behaved caches.

WHY NOT KEEP TTLS LOW ALL THE TIME?

Short TTLs mean more lookups, more load (and cost) on DNS, and slightly slower first requests, because more of them miss the cache. Long TTLs are faster and cheaper, but changes take longer to spread. So use **long TTLs** (hours) for stable records and **short ones** (30–60 s) for records you may need to change quickly, such as a failover endpoint.

COMMON MISTAKES

- Lowering the TTL at the same moment as the change. The old, long TTL is still cached.
- Shutting down the old server as soon as DNS changes. Stale caches and long-lived connections still send traffic there.
- Testing from your own laptop and seeing a cached answer. Query the authoritative server directly (`dig @ns1.example.com api.example.com`).
- Treating a CNAME as an HTTP redirect. It is resolved inside DNS, before any connection is made.

6 Key terms

Recursive resolver: does the lookup for clients, caches answers

Root server: points to TLD servers

TLD server: points to a domain's name servers

Authoritative server: holds the real records for a zone

TTL: seconds an answer may be cached

Negative caching: caching "name does not exist"

A / AAAA: name → IPv4 / IPv6

CNAME: alias to another name

NS: which servers are authoritative

7 Interview questions

Q1. Walk through a DNS lookup. BASIC

The client asks a recursive resolver. On a cache miss, the resolver asks a root server (which refers it to the .com TLD), then the TLD server (which refers it to the domain's authoritative servers), then the authoritative server, which returns the record. The answer is cached for its TTL.

Q2. What's the difference between an A record and a CNAME? BASIC

An A record maps a name straight to an IPv4 address. A CNAME maps a name to another name, which must then be resolved; it's how you point `www` at a cloud load balancer whose IPs change.

Q3. What does a TTL of 3600 mean? BASIC

Any cache may keep serving that answer for up to 3,600 seconds (one hour) without asking again. A change can take up to an hour to reach users behind that cache, longer if a client ignores TTLs.

Q4. Why lower TTLs before a migration, and how far ahead? SCENARIO

So that when you switch the record, cached answers expire in minutes, not hours. Lower it at least one old TTL before the switch (a day or two ahead is common), or caches still hold the long value.

Q5. You updated DNS an hour ago and some users still hit the old server. Why? SCENARIO

Their resolver, OS or browser cached the old answer, either from a long TTL or because the client ignores TTLs; long-lived connections also stay put. Keep the old server up and draining until traffic to it stops.

Q6. You created a new subdomain, but it fails to resolve for some users for minutes. Why? SCENARIO

Negative caching. Someone queried it before it existed, and resolvers cached the “does not exist” answer for the time set in the zone's SOA record. It clears when that expires.

Q7. Why not set every TTL to 10 seconds? TRICKY

Far more lookups: more DNS load and cost, more cache misses, slower first requests, and more exposure if the DNS provider has an outage. Use short TTLs only for records you need to move fast.

Q8. Can you put a CNAME at the zone apex (example.com)? TRICKY

No. The apex must hold SOA and NS records, and a CNAME can't coexist with other records. Providers offer ALIAS or ANAME records (Route 53 alias records) that resolve the target and answer with A records instead.

Next: SD-B-009 · DNS Traffic Management backendroadmap.com

DNS Traffic Management: GeoDNS, Weighted, Failover & Anycast

send each user to the right region, and where DNS falls short

Watch the 3-min video

In one line: an authoritative DNS server can give different answers to different users (by location, measured latency, weights or health), which routes traffic between regions; but failover through DNS is slow because of health-check time and caching, so use an anycast global load balancer when you need seconds.

1 One name, different answers

Your app runs in two regions: **eu-west** in Ireland and **us-east** in Virginia. Users in Paris should land in Europe, users in New York in the US, and if Europe goes down, everyone should end up in us-east. DNS can do all of this.

The trick is that an authoritative DNS server doesn't have to give everyone the same answer. With **GeoDNS** (geolocation routing), it looks at where the query comes from and returns the IP of the closest region. A query from Paris gets the eu-west load balancer; a query from New York gets us-east.

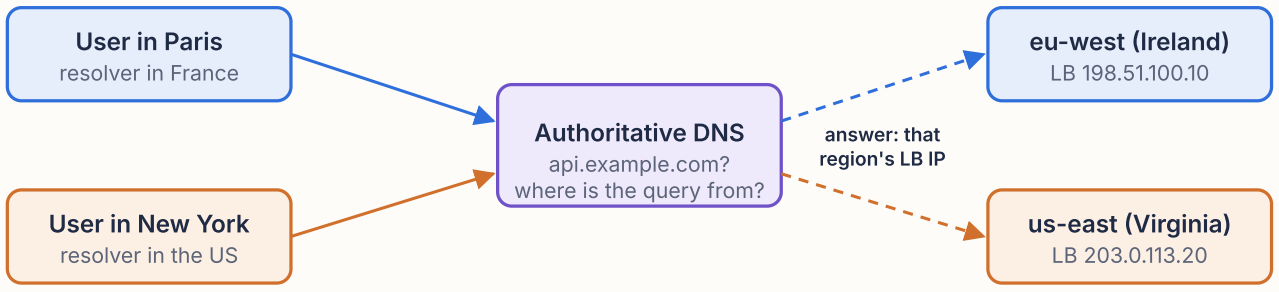


Figure 1. GeoDNS: the same question gets a different answer depending on where it comes from.

Latency-based routing goes one step further. Instead of guessing from geography, the DNS provider measures network latency from different networks to each region and returns the region that is actually fastest for that network. That matters where geography misleads: a user in one country may have a faster path to a region further away. **AWS Route 53** offers both, as well as the policies below.

2 Weighted routing

Weighted routing splits traffic by percentage. Give the current version weight 90 and a new one weight 10, and about 10% of answers point at the new one. It is handy for **canary releases**, **gradual migrations** (move a region or a provider in steps) and **balancing load** between regions of different size.

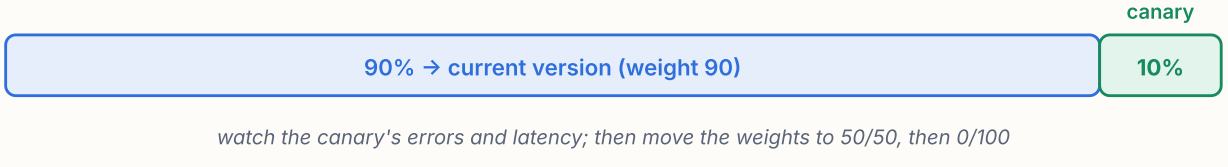


Figure 2. Weighted records send roughly 90% of lookups to the current stack and 10% to the canary.

The split is approximate. It applies to *DNS answers*, not to requests: one big resolver can serve thousands of users the same cached answer for a whole TTL. Over many resolvers it averages out, but don't expect an exact 10%.

3 Failover with health checks

The DNS provider runs **health checks** against each region every few seconds (for example an HTTP request to `/health` every 10 or 30 seconds, from several locations). If eu-west fails several checks in a row, it is marked **unhealthy**, and DNS starts answering with us-east instead. When eu-west recovers and passes its checks again, traffic moves back.

Policies combine: geolocation or latency routing picks the preferred region, and health checks remove a region from the answers when it fails.

4 Anycast: one IP in many places

Anycast is a different tool. The **same IP address** is announced from many locations around the world, and internet routing (BGP) delivers each packet to the nearest one in network terms. Nothing about the DNS answer changes: everyone gets the same IP.

DNS providers use anycast for their own name servers. That makes lookups fast (you reach a nearby server) and spreads out attacks: a DDoS flood is split across every location instead of hitting one. Global load balancers and CDNs use it too, which is the fix for slow DNS failover.

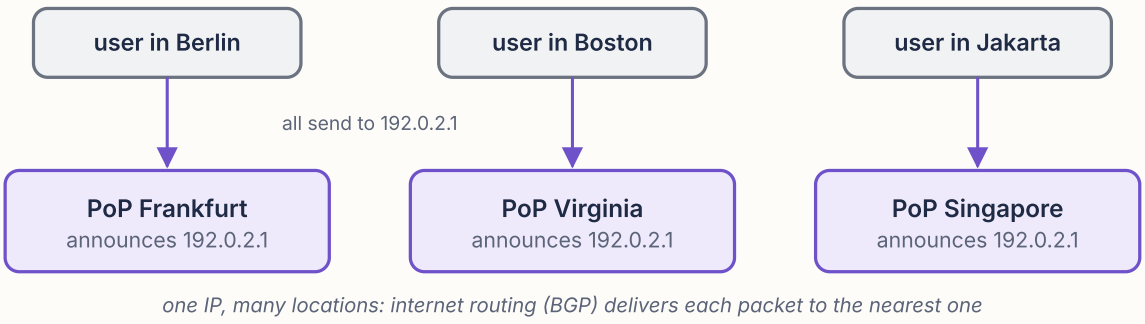
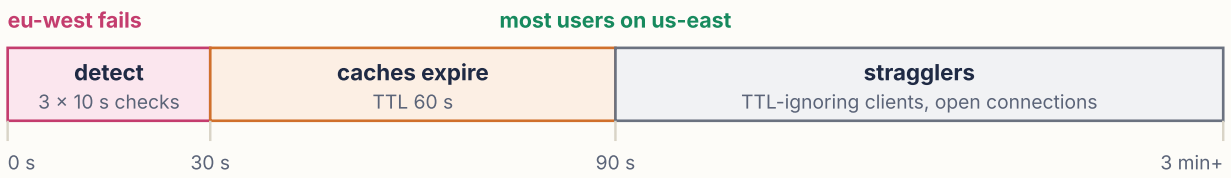


Figure 3. Anycast: three points of presence announce 192.0.2.1. Each user's packets reach the closest one; if one location fails, routing sends them to the next.

5 Why DNS failover is slow

- **Detection.** Several failed health checks are needed to avoid flapping: 3 checks at 10 s is 30 s; at 30 s it is a minute and a half.
- **Caching.** Resolvers keep the old answer until its TTL expires, and some ignore TTLs completely.
- **Connections.** Apps keep their existing connections open (keep-alive, connection pools, WebSockets) and won't look up DNS again until they reconnect.
- **Wrong location.** GeoDNS sees the *resolver's* location, not the user's. A user in Paris whose resolver is far away may land in the wrong region. (Some resolvers forward part of the client's subnet, called EDNS Client Subnet, which helps.)



a global load balancer with an anycast IP shifts traffic in seconds: no DNS answer has to change

Figure 4. A rough DNS failover timeline with fast checks and a 60 s TTL: about a minute and a half for most users, minutes or more for the rest.

WORKED EXAMPLE: THE FAILOVER BUDGET

Health checks every **10 s**, failure after **3** misses: detection takes about **30 s**. The record's TTL is **60 s**, so caches may serve the dead region for up to another minute: about **90 s** total for most users. With 30 s checks and a 300 s TTL it becomes 90 + 300 ≈ **6.5 minutes**. If the target is "under 10 seconds", DNS can't meet it. Put a **global load balancer with an anycast IP** in front (for example AWS Global Accelerator or Cloudflare): its health checks move traffic between regions behind the same IP, so no cache has to expire.

COMMON MISTAKES

- Promising near-instant failover with DNS alone.
- A long TTL on a failover record. The failover can only be as fast as the TTL allows.
- Assuming GeoDNS knows where the user is. It knows where the resolver is.
- Expecting weights to split traffic exactly.
- Health checks that only test "is the web server up", not whether the region can actually serve requests (database, dependencies).

6 Key terms

GeoDNS: answer by the query's location

Health check: probe that marks a target healthy or not

BGP: internet routing between networks

Latency-based routing: answer with the fastest measured region

DNS failover: stop answering with an unhealthy region

Canary: small share of traffic on a new version

Weighted routing: split answers by percentage

Anycast: one IP announced from many places

Global load balancer: anycast front door across regions

7 Interview questions

Q1. How can DNS route users to the nearest region? BASIC

GeoDNS answers based on where the query comes from; latency-based routing returns the region with the lowest measured latency from the user's network. Both are features of the authoritative DNS provider, such as Route 53.

Q2. What is anycast? BASIC

The same IP address announced from many locations. Internet routing delivers each packet to the nearest one. DNS providers use it for their name servers: lookups are fast and DDoS traffic is spread across every site.

Q3. Why is DNS failover slow? BASIC

Health checks need time to detect the failure (often 30–90 s), then cached answers live until their TTL expires, sometimes longer when clients ignore TTLs, and open connections stay on the old region.

Q4. Roll out a new version to 5% of users at the DNS level. How? SCENARIO

Weighted records: 95 for the current stack, 5 for the new one. Watch error rate and latency, then step up to 25, 50, 100. Keep the TTL short (60 s) so you can roll back quickly, and remember the split is approximate.

Q5. Users in Paris sometimes land in us-east. Why? SCENARIO

GeoDNS sees the resolver's location, not the user's. If they use a resolver located in the US, or one without EDNS Client Subnet, they get the US answer. Latency-based routing or an anycast front door reduces this.

Q6. The requirement is failover between regions in under 10 seconds. Design it. SCENARIO

Don't rely on DNS. Put a global load balancer with an anycast IP in front of both regions; it health-checks the regional backends and shifts traffic within seconds behind the same IP. Keep DNS as a slower backup layer.

Q7. You set TTL to 60 s, but after failover some traffic hits the dead region for 30 minutes. Why? TRICKY

Some resolvers and client runtimes cache longer than the TTL, and long-lived connections (pools, WebSockets, HTTP keep-alive) never re-resolve until they break. Close idle connections periodically and have clients retry against a fresh lookup.

Q8. Weighted 90/10, but the canary gets 30% of traffic. How? TRICKY

Weights apply to DNS answers, not requests. A few large resolvers (a big ISP, a corporate network) that happened to get the canary answer serve it to all their users for the whole TTL. It evens out over time, not per minute.

Next: SD-B-010 · HTTP Fundamentals backendroadmap.com

HTTP Fundamentals: Methods, Status Codes, 401 vs 403 & ETags

the details interviewers love

Watch the 3-min video

In one line: HTTP is stateless request → response; idempotent methods are safe to retry, 401 means “who are you?” and 403 “not allowed”, and ETags turn a re-download into a tiny 304.

1 Requests and responses

Every API call you make is HTTP: a request goes out and a response comes back. A **request** has a **method**, a **path**, **headers** and an optional **body**. For example `GET /products/42` with headers like `Host`, `Authorization` and `Accept`. The **response** has a **status code**, headers and a body, like `200 OK` with some JSON.

Request

```
GET /products/42 HTTP/1.1
Host: shop.example.com
Authorization: Bearer eyJhbGci…
Accept: application/json
```

method + path, then headers, then an optional body

Response

```
HTTP/1.1 200 OK
Content-Type: application/json
Cache-Control: max-age=3600
ETag: "v7"
{"id": 42, "name": "Lamp"}
```

status code, headers, then the body

Figure 1. One request and its response. Headers say who is calling, what format is wanted and how long the answer may be cached.

HTTP itself is **stateless**: each request stands alone. Cookies or tokens, sent on every request, carry the session, so any server behind a load balancer can handle the next one.

2 Methods: safe and idempotent

Safe means the request doesn't change anything on the server. **Idempotent** means repeating it has the same effect as sending it once. Idempotency is what makes a retry safe after a timeout, when you don't know if the first try worked.

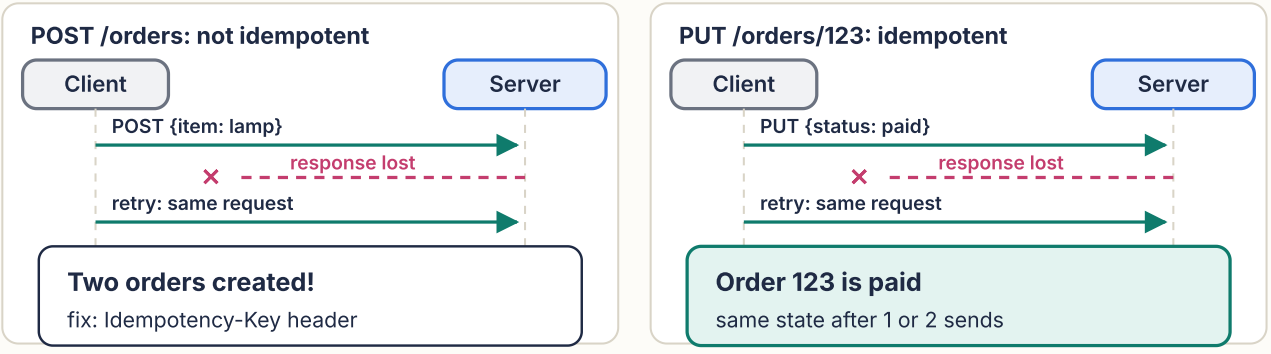


Figure 2. The response to the first send is lost, so the client retries. A second POST creates a second order; a second PUT leaves the same state.

Method	Meaning	Safe	Idempotent
GET	Read a resource	yes	yes
HEAD	Like GET, headers only (check size, ETag, existence)	yes	yes
OPTIONS	Which methods are allowed (used by CORS preflight)	yes	yes
PUT	Replace the resource with this full version	no	yes
DELETE	Remove the resource	no	yes
POST	Create a resource, or run an action	no	**no**
PATCH	Update part of a resource	no	not guaranteed

So a client can safely retry a `PUT`, but retrying a `POST` might create a duplicate order. `PATCH` depends on the change: “set status to paid” is idempotent, “add 1 to quantity” is not.

3 Status codes come in families

Family	Meaning	Common codes
2xx	Success	`200 OK`, `201 Created` (after a POST, with a `Location` header), `204 No Content`
3xx	Redirect, or use your cache	`301` / `302` redirects, `304 Not Modified`
4xx	The client made a mistake	`400 Bad Request`, `401`, `403`, `404 Not Found`, `429 Too Many Requests` (rate limited)
5xx	The server failed	`500 Internal Server Error`, `502 Bad Gateway` (a proxy got a bad answer from upstream), `503 Service Unavailable` (overloaded or down)

The family tells a client what to do. A 4xx won't succeed if repeated unchanged (except 429, after waiting). A 5xx or 429 may be worth retrying with backoff, if the method is idempotent.

4 401 vs 403

A classic interview question. **401 Unauthorized** really means **unauthenticated**: we don't know who you are, so log in or send a valid token (the response includes a `WWW-Authenticate` header saying how). **403 Forbidden** means we know exactly who you are, but you're **not allowed** to do this. A normal user opening an admin page gets a 403. Logging in again won't help.

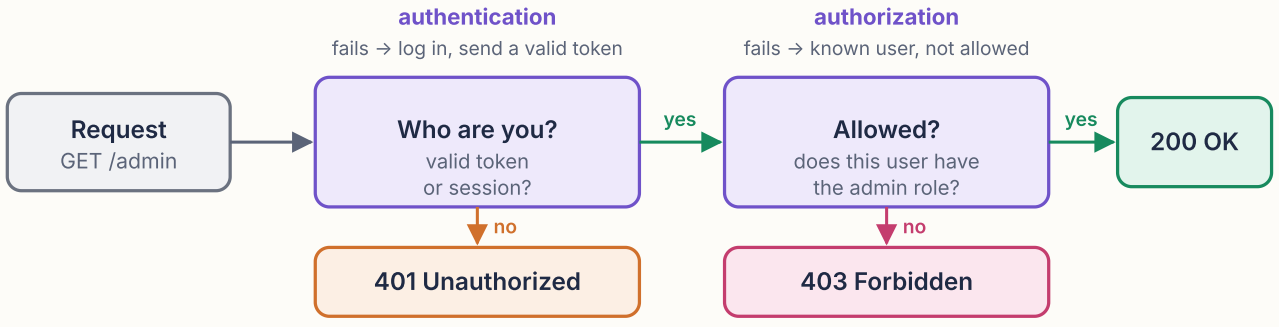


Figure 3. Authentication comes first (fails with 401), then authorization (fails with 403).

5 Caching headers and ETags

Caching headers let browsers and CDNs reuse responses. `Cache-Control` says how long a response stays fresh, like `max-age=3600` (one hour), or `no-store` for private data that must never be cached. **Content negotiation** uses headers too: the client sends `Accept: application/json` or `Accept-Encoding: gzip`, and the server picks the best format it can give and says which in `Content-Type` and `Content-Encoding`.

An **ETag** is a version tag for a response, like `ETag: "v7"`. Next time, the client sends `If-None-Match: "v7"`. If nothing changed, the server replies **304 Not Modified** with no body at all, and the client reuses its cached copy. If it did change, the server sends 200 with the new body and a new ETag.

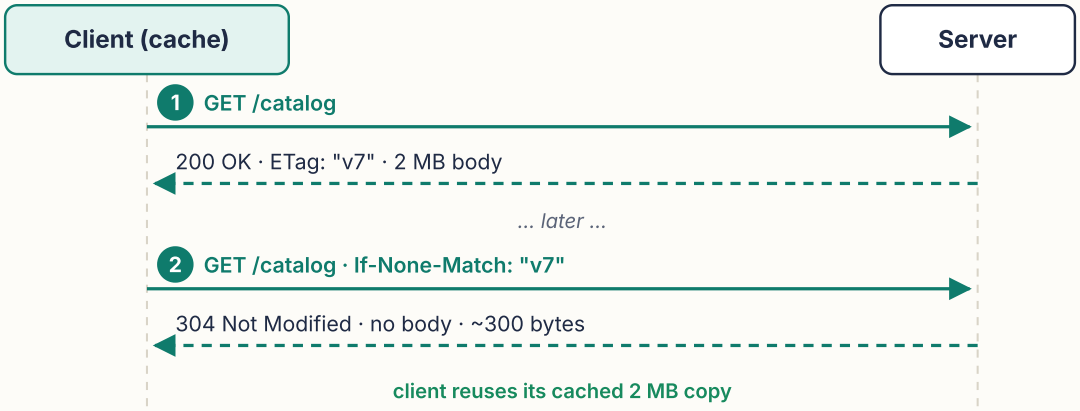


Figure 4. Revalidating with an ETag. The second exchange costs a few hundred bytes instead of 2 MB.

WORKED EXAMPLE: A PRODUCT CATALOG

The catalog response is **2 MB** and changes a few times a day. 100,000 app opens a day, each fetching it: **200 GB** of transfer. With an ETag, unchanged fetches become 304s of about **300 bytes**: 100,000 × 300 B ≈ **30 MB**, plus a few full downloads after each change. Add `Cache-Control: max-age=300` and most opens don't even ask for 5 minutes.

COMMON MISTAKES

- Automatically retrying POST on timeout without an idempotency key: duplicate orders and double charges.
- Returning 401 when the user is logged in but lacks permission (that's 403), or 200 with an error message in the body.
- Changing data in a GET handler. Crawlers, prefetchers and caches assume GET is safe.
- Using `no-cache` when you mean `no-store`. `no-cache` still stores the response, it just revalidates before every use.

6 Key terms

Stateless: each request stands alone

Safe method: doesn't change server state

Idempotent: repeating = same effect as once

401: not authenticated: who are you?

403: authenticated, but not allowed

Cache-Control: how long a response stays fresh

Content negotiation: Accept headers pick the format

ETag: version tag of a response

304 Not Modified: your copy is current, no body

7 Interview questions

Q1. Which HTTP methods are idempotent? BASIC

GET, HEAD, PUT, DELETE and OPTIONS. POST is not, so retries can create duplicates unless you add an idempotency key; PATCH is not guaranteed to be.

Q2. What's the difference between 401 and 403? BASIC

401 means you're not authenticated: send valid credentials. 403 means you're authenticated but not allowed: a normal user on an admin page. Re-logging in fixes a 401, not a 403.

Q3. What does it mean that HTTP is stateless? BASIC

The server keeps no memory between requests; each one carries everything needed, with cookies or tokens identifying the session. That's what lets any server behind a load balancer handle any request.

Q4. A payment POST times out. Can the client retry? SCENARIO

Only safely with an idempotency key: the client sends a unique `Idempotency-Key` header, the server stores the result under that key, and a retry with the same key returns the stored result instead of charging twice.

Q5. Your mobile app downloads a 2 MB catalog on every launch. How do you cut bandwidth? SCENARIO

Return an ETag (and a short `max-age`). The app sends `If-None-Match`; when nothing changed, the server answers 304 with no body, about 300 bytes instead of 2 MB. Also serve it gzip-compressed via `Accept-Encoding`.

Q6. Users get 502 from the load balancer, and 503 at peak. What do they tell you? SCENARIO

502: the load balancer got an invalid response from a backend (it crashed or closed the connection). 503: the service is overloaded, has no healthy backends, or is in maintenance; clients should back off, ideally honouring `Retry-After`.

Q7. DELETE is idempotent, but the second DELETE returns 404. Is that a contradiction? TRICKY

No. Idempotency is about the effect on server state, not the response. After one or two DELETEs the resource is gone either way; the status code can differ.

Q8. Is PATCH idempotent? TRICKY

Not guaranteed. A PATCH that sets fields (`{"status": "paid"}`) is idempotent; one that applies a relative change ("add 1 to quantity") is not. Treat PATCH like POST for retries unless your API defines it otherwise.

Next: SD-B-011 · HTTPS & TLS 1.3 backendroadmap.com

HTTPS & TLS 1.3 Handshake

ECDHE, forward secrecy and 0-RTT

Watch the 3-min video

In one line: HTTPS is HTTP inside TLS; the TLS 1.3 handshake agrees on fresh secret keys with ECDHE and proves the server's identity with a certificate, all in one round trip, then switches to fast symmetric encryption.

1 What TLS guarantees

Every padlock in the browser means a TLS handshake just happened, usually in tens of milliseconds. **HTTPS** is plain HTTP carried inside a **TLS** connection (port 443). TLS adds three guarantees:

- **Confidentiality:** nobody on the network can read the traffic.
- **Integrity:** nobody can change a byte without the receiver noticing.
- **Authentication:** the server proves who it is with a certificate (the next lesson).

2 The TLS 1.3 handshake, step by step

One round trip both agrees on keys and proves the server's identity:

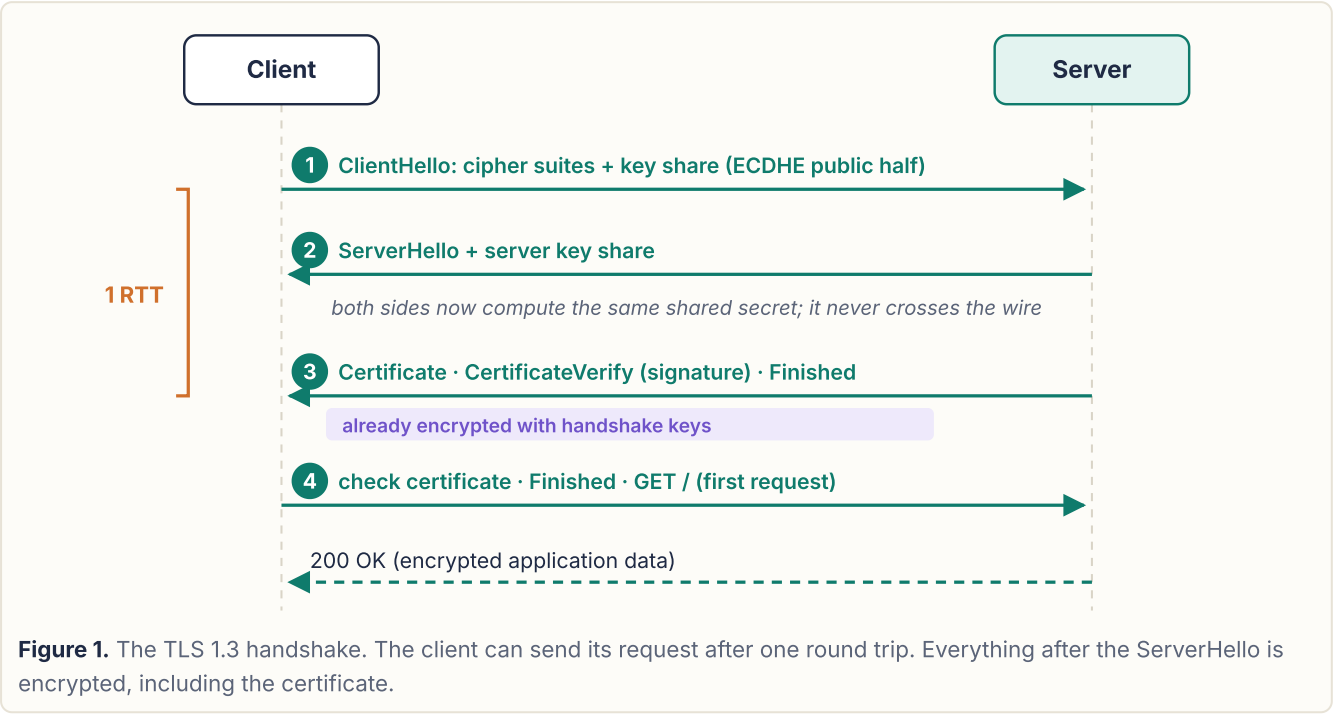


Figure 1. The TLS 1.3 handshake. The client can send its request after one round trip. Everything after the ServerHello is encrypted, including the certificate.

1. **ClientHello.** The client lists the cipher suites it supports and sends a **key share**: its public half of an **ECDHE** (elliptic-curve Diffie-Hellman, ephemeral) exchange, usually on curve X25519.
2. **ServerHello.** The server picks a suite and sends its own key share. From the two shares, each side computes the **same shared secret**. The secret itself is never sent.
3. **Server's encrypted flight.** Already encrypted, the server sends its **Certificate**, a **CertificateVerify** (a signature over the handshake made with its private key, proving it owns the certificate) and **Finished** (a MAC over the whole handshake, so nothing was tampered with).

4. **Client finishes.** The client checks the certificate and signature, sends its own Finished and, in the same flight, its first HTTP request.

3 Why TLS 1.3 is faster and safer than 1.2

TLS 1.2 needed **two round trips** before the first request, and still allowed old, weak options (RSA key exchange, CBC modes, RC4, SHA-1). TLS 1.3 needs **one**, removed the legacy algorithms (only five cipher suites remain, all AEAD, such as `TLS_AES_128_GCM_SHA256`), and made forward secrecy mandatory.



Figure 2. Time to the first response on a 100 ms round-trip link, counting the TCP handshake. TLS 1.3 saves 100 ms on every new connection; 0-RTT resumption saves another 100 ms.

Expensive once, cheap after. Public-key crypto (key exchange and signature) runs only during the handshake. Then both sides derive **session keys** from the shared secret and switch to fast **symmetric** encryption such as AES-GCM or ChaCha20-Poly1305, which modern CPUs do at gigabytes per second.

Resumption. After a handshake the server can hand the client a session ticket. Next time, the client offers it as a **pre-shared key (PSK)** and the server skips sending and checking the certificate. TLS 1.3 also allows **0-RTT**: the client sends *early data* (say, a GET) in its very first message. The catch: an attacker who copies that first message can **replay** it, and the server cannot tell. So accept 0-RTT only for **idempotent** requests such as GETs, never for "pay €50".

4 Forward secrecy

Suppose an attacker records all your encrypted traffic today and steals the server's private key next year. With the old **RSA key exchange**, the client encrypted the session secret with the server's long-term public key, so the stolen key unlocks every recording. With **ECDHE**, each session uses fresh **ephemeral** keys that are thrown away afterwards; the long-term private key only *signs* the handshake. Stealing it later decrypts nothing from the past. That property is **forward secrecy**.

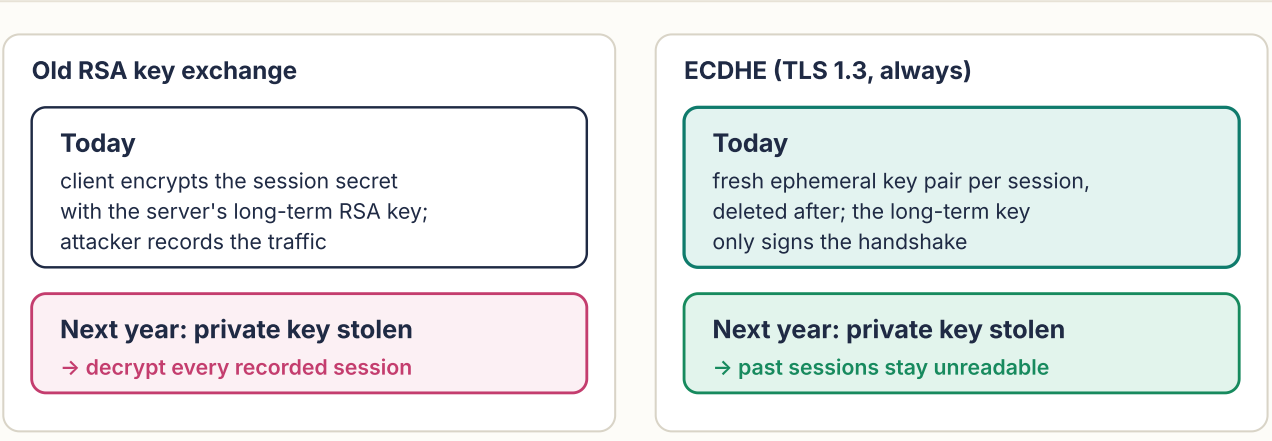


Figure 3. With RSA key exchange one stolen key exposes years of recorded traffic. With ECDHE, a stolen key lets an attacker impersonate the server from then on, but not read the past.

5 Where TLS ends in your architecture

Most often TLS **terminates at the load balancer**. It holds the public certificate, decrypts, and can route by path, header or cookie, add headers like `X-Forwarded-Proto`, and run a WAF. It then **re-encrypts** to the backends over a second TLS connection, so traffic stays encrypted inside your network too, as zero-trust and compliance rules (PCI DSS, HIPAA) often require. In **passthrough** mode the load balancer forwards raw encrypted bytes (layer 4) and the backends terminate TLS themselves: true end-to-end encryption, but the load balancer cannot see HTTP, so no path routing.

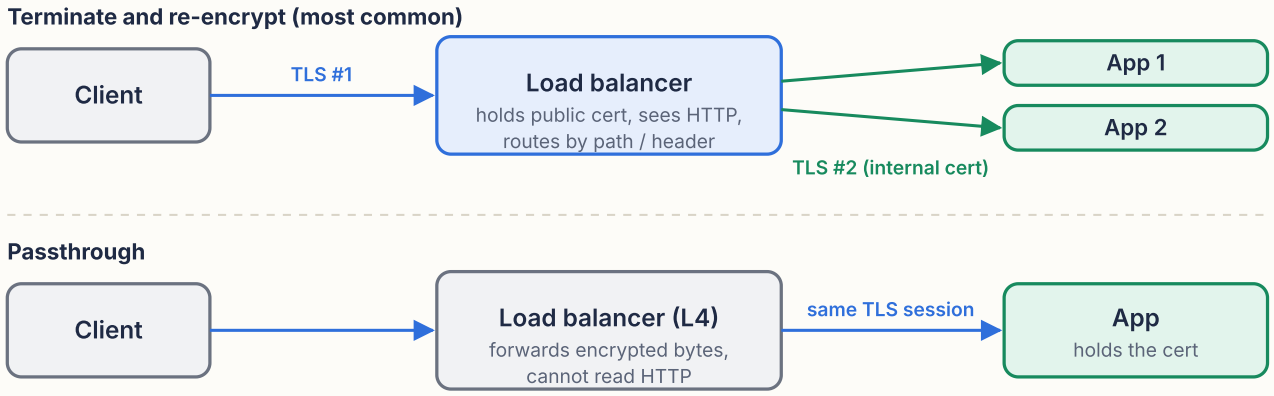


Figure 4. Top: two separate TLS connections, with the load balancer able to read and route HTTP. Bottom: one TLS session from client to app; the load balancer only sees bytes.

WORKED EXAMPLE

A mobile API: 100 ms round trip, users open a new connection on every app launch. TCP + TLS 1.2 + request = 4 round trips ≈ **400 ms** to the first byte. Moving to TLS 1.3: 3 round trips ≈ **300 ms**. Returning users with resumption skip certificate checks; enabling 0-RTT for the idempotent `GET /feed` brings it to ≈ **200 ms**. `POST /orders` still waits for the full handshake. (HTTP/3 folds the TCP step in too: lesson SD-B-013.)

COMMON MISTAKES

- Accepting 0-RTT early data for POSTs or payments: it can be replayed.
- Leaving TLS 1.0 / 1.1 or RSA key exchange enabled "for old clients": it removes forward secrecy for everyone who negotiates down.
- Terminating at the load balancer and sending plain HTTP inside, then assuming the internal network is safe.

6 Key terms

TLS: encrypts and authenticates a connection

CertificateVerify: signature proving the server holds the private key

Forward secrecy: a stolen long-term key can't decrypt past traffic

ECDHE: ephemeral elliptic-curve Diffie-Hellman key exchange

Finished: MAC over the handshake transcript

PSK / resumption: reuse a previous session, skip cert work

Key share: one side's public half of ECDHE

AES-GCM: fast symmetric AEAD cipher for bulk data

0-RTT: data in the first message; replayable

7 Interview questions

Q1. Walk through a TLS 1.3 handshake. BASIC

Client sends ClientHello with cipher suites and an ECDHE key share. Server replies with ServerHello and its key share; both derive the same secret. The server sends, encrypted, its certificate, a CertificateVerify signature and Finished. The client verifies, sends Finished and its first request: one round trip.

Q2. What is forward secrecy? BASIC

Each session uses ephemeral ECDHE keys that are deleted afterwards, and the long-term key only signs. So stealing the server's private key later cannot decrypt traffic recorded in the past. TLS 1.3 makes it mandatory.

Q3. Why use symmetric encryption after the handshake? BASIC

Public-key operations cost around a millisecond of CPU each; AES-GCM with hardware support encrypts several GB/s per core. So the handshake pays for asymmetric crypto once, then bulk data uses symmetric session keys.

Q4. Your app has 100 ms RTT and users complain about slow first loads. What does TLS 1.3 buy you? SCENARIO

One round trip less per new connection: about 400 ms to 300 ms counting TCP. Add session resumption, and 0-RTT for idempotent GETs, to reach about 200 ms. HTTP/3 would also merge the TCP handshake.

Q5. Security asks for encryption inside the data centre, but you route by URL path at the load balancer. What do you do? SCENARIO

Terminate TLS at the load balancer (it needs HTTP to route), then re-encrypt to the backends with internal certificates, or mTLS through a service mesh. Passthrough would block path routing.

Q6. Should you enable 0-RTT for your whole API? SCENARIO

No. Early data can be replayed by anyone who captured it, and the server can't detect it on its own. Allow it only for idempotent, safe requests (GETs) and reject or delay early data for POST, PUT and payments.

Q7. If TLS 1.3 encrypts the certificate, can a network observer still see which site you visit? TRICKY

Usually yes. The destination IP is visible, and the SNI hostname in the ClientHello is plaintext unless Encrypted Client Hello (ECH) is used. TLS 1.3 hides the certificate, not the destination.

Q8. An attacker steals your TLS private key today. What can they do with TLS 1.3? TRICKY

Impersonate your server to clients going forward (until the certificate is revoked and replaced), but not decrypt previously recorded sessions, thanks to ephemeral ECDHE. Rotate the key, revoke the certificate, and check for misuse.

Next: SD-B-012 · Certificates & PKI backendroadmap.com

Certificates & PKI

chain of trust, SNI, Let's Encrypt and mTLS

Watch the 3-min video

In one line: a certificate binds a name to a public key; browsers trust it by checking signatures up a chain to a root in their trust store, and short-lived certificates with automated renewal (ACME) end surprise expiry outages.

1 What a certificate is

Your HTTPS site works for 90 days. Then one morning every browser shows a red warning: the certificate expired and nobody renewed it. It is one of the most common outages there is.

A certificate is an **X.509** document that **binds a name to a public key**. It contains:

Field	What it holds
Subject Alternative Names (SAN)	The domain names it is valid for, e.g. `shop.example.com` or a wildcard `*.example.com` (one level only)
Public key	The key matching the server's private key, which never leaves the server
Issuer	The certificate authority (CA) that signed it
Validity	`notBefore` and `notAfter` dates
Signature	The issuer's digital signature over all of the above, so no field can be changed

2 The chain of trust

Who signs it? A **certificate authority (CA)**. Your server's **leaf** certificate is signed by an **intermediate CA**. The intermediate is signed by a **root CA**. Root certificates come pre-installed in the operating system and browser: the **trust store**. Roots are kept offline and only sign intermediates, so a compromised intermediate can be revoked without replacing roots on billions of devices.

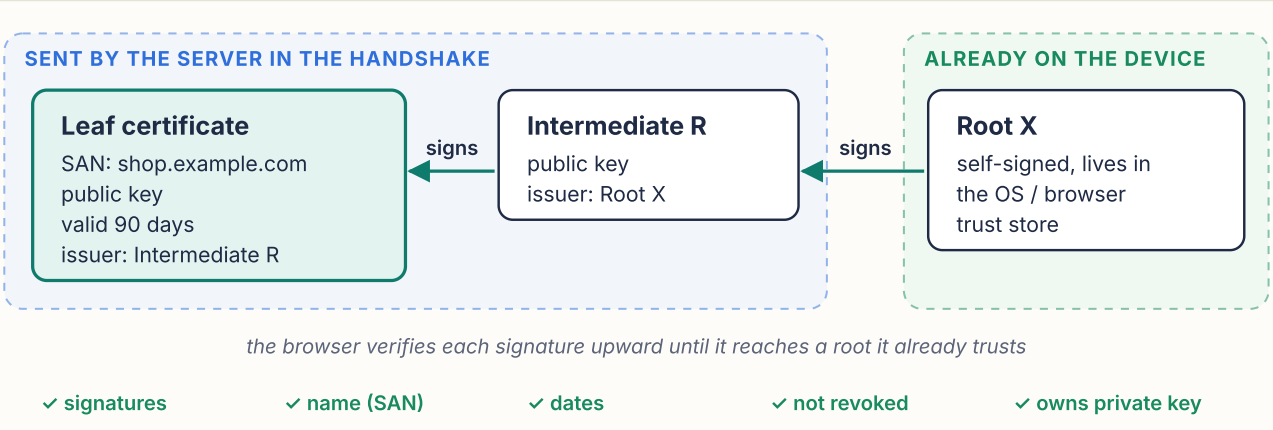


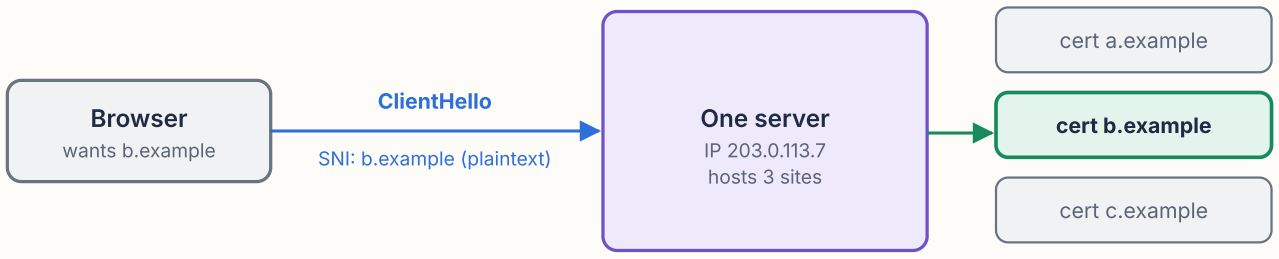
Figure 1. The server sends its leaf plus the intermediate. The root is never sent: the client must already have it. Every check must pass, or the browser shows the warning.

How a browser decides to trust a certificate:

1. Verify each signature up the chain until it reaches a root in its trust store.
2. Check that the site's hostname matches a name in the SAN.
3. Check that today is between the validity dates.
4. Check that the certificate has not been **revoked** (via CRLs, OCSP or the browser's own revocation lists).
5. In the handshake, the server **signs** with the matching private key (CertificateVerify), proving it owns the certificate and did not just copy it.

3 SNI: many sites, one IP address

One server or load balancer often hosts many domains on a single IP address. Which certificate should it send? **Server Name Indication (SNI)**: the client puts the hostname in its ClientHello, so the server picks the right certificate *before* encryption starts. Because it comes first, SNI is sent in plaintext (Encrypted Client Hello, ECH, is the newer fix).



the hostname arrives before encryption starts, so the server can choose the right certificate

Figure 2. SNI lets one IP serve many HTTPS sites, each with its own certificate. Without SNI the server would have to guess, and usually sends a default certificate with the wrong name.

4 Rotation: Let's Encrypt, ACME and cert-manager

Let's Encrypt issues free certificates that last **90 days**, using the **ACME** protocol. To get one you prove you control the domain with a **challenge**:

- **HTTP-01:** serve a token file at `http://your-domain/.well-known/acme-challenge/…` on port 80.
- **DNS-01:** create a TXT record at `_acme-challenge.your-domain`. Needed for wildcard certificates, and works for servers not reachable from the internet.

In Kubernetes, **cert-manager** automates it: it requests certificates, stores them as Kubernetes **Secrets** that ingresses use, and renews them well before they expire (by default when a third of the lifetime is left). Short lifetimes plus automation mean no surprise expiries, and a leaked key is useful for weeks, not years. The industry is pushing lifetimes even shorter, so manual renewal is no longer realistic.



renew early: 30 days leaves time for several failed attempts and for someone to notice the alert

Figure 3. A 90-day certificate renewed at day 60. Alert if any certificate has fewer than ~14 days left: that means automation has been failing for two weeks.

5 Mutual TLS (mTLS)

Normally only the server shows a certificate. With **mutual TLS**, the client presents one too, and the server checks it against a trusted CA. This is ideal for **service-to-service authentication**: the payments service accepts calls only from services holding a valid internal certificate, issued by your own private CA. Service meshes like **Istio** and **Linkerd** run a sidecar proxy next to each service that issues, presents and rotates these certificates automatically (often every 24 hours), so application code does not change.

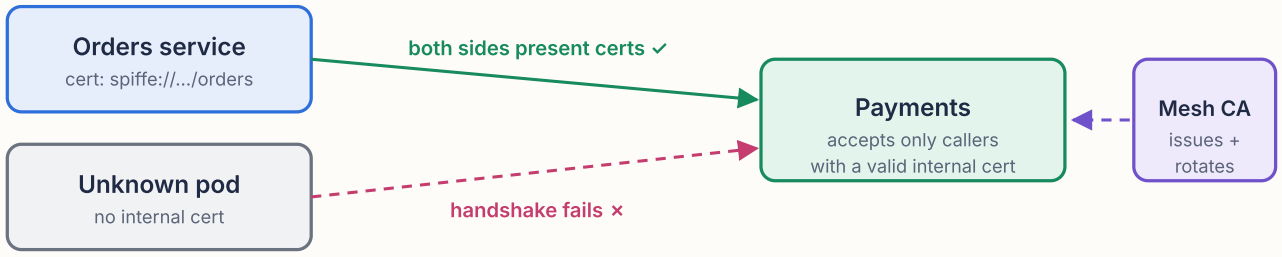


Figure 4. With mTLS the payments service knows which service is calling, from its certificate, before any request is read. A pod without a valid certificate cannot even finish the handshake.

WATCH OUT: CERTIFICATE PINNING

Some mobile apps **pin**: they hard-code which certificate or public key they will trust. If you rotate to a new key the app doesn't know, every installed copy stops connecting until users update. If you pin, pin a backup key too and plan rotation, or pin the CA's intermediate instead of your leaf.

WORKED EXAMPLE

You run 40 domains. With manual 1-year certificates that is about 40 renewals a year, each one a chance to forget. With ACME and 90-day certificates renewed at 30 days left, that is about 160 renewals a year, all automatic, each with a 30-day window to retry. Monitor `notAfter` for every endpoint, and page when any is under 14 days.

COMMON MISTAKES

- Renewing by hand and relying on someone's calendar.
- Serving the leaf without the intermediate: works in browsers that cached it, fails in curl, mobile apps and containers.
- Relying on the certificate's CN instead of SAN, or expecting `*.example.com` to cover `a.b.example.com`.
- Pinning a leaf key in a mobile app with no backup pin.

6 Key terms

X.509: the standard certificate format

CA: certificate authority, signs certificates

Leaf / intermediate / root: the three links of a chain

Trust store: roots pre-installed in OS and browser

SAN: the names a certificate is valid for

SNI: hostname in the ClientHello

ACME: protocol for automated issuance and renewal

cert-manager: automates ACME in Kubernetes

mTLS: both client and server present certificates

7 Interview questions

Q1. How does a browser trust a certificate? BASIC

It verifies the chain of signatures from the leaf, through intermediates, to a root CA in its trust store. It checks the hostname against the SAN, the validity dates and revocation. And in the handshake the server signs with the matching private key.

Q2. What is SNI and why is it needed? BASIC

Server Name Indication: the client sends the hostname in the ClientHello, so a server hosting many domains on one IP can present the matching certificate before encryption starts.

Q3. What is mTLS used for? BASIC

Mutual authentication, mostly between services: both sides present certificates, so the server knows exactly which service is calling. Meshes like Istio and Linkerd issue and rotate them automatically.

Q4. Your site went down because a certificate expired. How do you prevent it from happening again? SCENARIO

Automate with ACME (certbot, cert-manager, or managed certificates on the load balancer), renewing at 30 days left. Monitor notAfter for every endpoint and alert below 14 days, so failing automation is caught with two weeks to spare.

Q5. You need a certificate for *.internal.example.com on servers not reachable from the internet. Which ACME challenge? SCENARIO

DNS-01: create a TXT record at _acme-challenge.internal.example.com. HTTP-01 needs port 80 reachable from the internet and can't issue wildcards.

Q6. curl says 'unable to verify the first certificate' but the browser shows the padlock. Why? SCENARIO

The server isn't sending its intermediate certificate. The browser had cached it or fetched it on its own; curl and most libraries don't. Configure the server to send the full chain (leaf + intermediate).

Q7. Why do CAs use intermediates instead of signing leaves with the root? TRICKY

The root key stays offline in an HSM, so it's hard to steal. If an intermediate is compromised it can be revoked and replaced, without updating trust stores on billions of devices, which takes years.

Q8. Why are short certificate lifetimes considered more secure? TRICKY

Revocation checking is unreliable (browsers often soft-fail), so a stolen key stays usable until expiry. With 90-day or shorter certificates, the damage window is weeks, not years. It also forces automation, which removes human error.

Next: SD-B-013 · HTTP/1.1 vs HTTP/2 vs HTTP/3 backendroadmap.com

HTTP/1.1 vs HTTP/2 vs HTTP/3

how requests share the network

Watch the 3-min video

In one line: HTTP/1.1 sends one request at a time per connection; HTTP/2 multiplexes many streams over one TCP connection but still stalls on a lost packet; HTTP/3 runs on QUIC over UDP, so streams are independent, setup is faster and connections survive network changes.

A modern web page can fetch a hundred files: scripts, styles, images and API calls. How those requests share the network decides how fast the page feels. The meaning of HTTP (methods, status codes, headers) is the same in all three versions; what changes is how messages travel on the wire.

1 HTTP/1.1: one request at a time per connection

HTTP/1.1 is **text-based**. **Keep-alive** lets one TCP connection carry many requests, but only one at a time: the next request waits until the current response has fully arrived. A slow response blocks everything queued behind it. That is **head-of-line (HOL) blocking** at the HTTP level. (Pipelining tried to fix this, but responses still had to come back in order, and browsers never enabled it.)

So browsers open about **six connections per host**, each with its own TCP and TLS handshake, and still send the same bulky headers (cookies, user agent) on every request, often 500 bytes to 2 KB each.

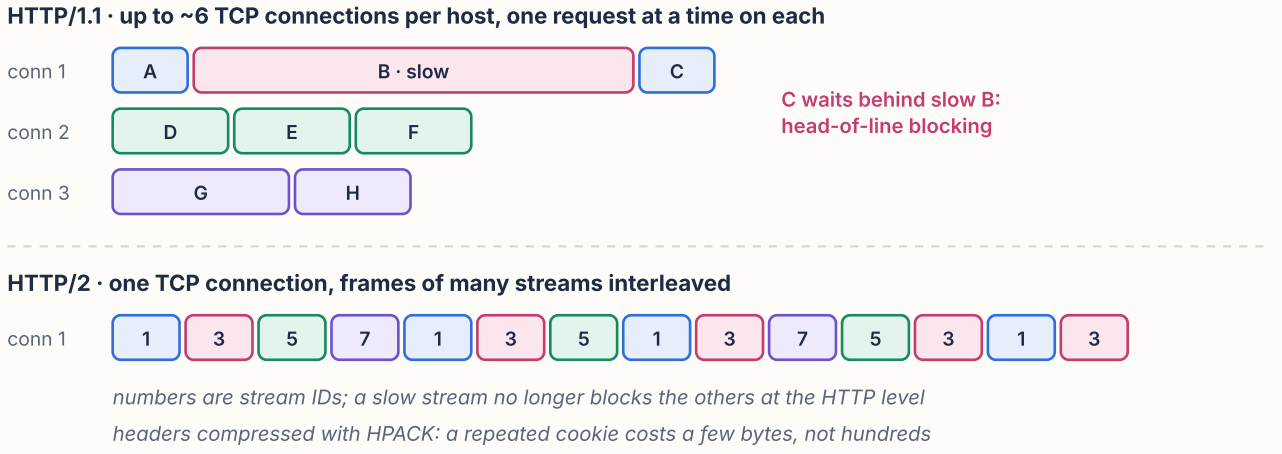


Figure 1. Top: six lanes at most, and a slow response blocks its lane. Bottom: HTTP/2 splits each request and response into frames and interleaves them on a single connection.

2 HTTP/2: binary frames and multiplexing

HTTP/2 switched to **binary framing**. Each request/response pair becomes a **stream** with an ID, cut into frames. Frames of many streams are **interleaved** on one TCP connection: that is **multiplexing**. No more waiting in line at the HTTP level, and no need for six connections (and six handshakes). Servers typically allow 100 or more concurrent streams per connection.

- **HPACK** header compression: both sides keep a table of headers already sent, so a repeated header becomes a small index of a few bytes.
- **Server push** let a server send files before they were asked for. It rarely helped and is now mostly abandoned (Chrome removed it in 2022).

- Browsers only use HTTP/2 over TLS; the version is agreed in the TLS handshake via **ALPN** (`h2`).

gRPC is built on HTTP/2. Every call is a stream, so one connection can carry thousands of concurrent calls, including long-lived **streaming calls in both directions**.

3 The catch: TCP head-of-line blocking

HTTP/2 still runs on **TCP**, and TCP delivers bytes strictly in order. If one packet is lost, every byte after it waits in the receiver's kernel buffer until the retransmission arrives, even bytes that belong to other streams. Lose a single packet and **every stream on that connection stalls**. On a lossy mobile network, one HTTP/2 connection can even be slower than six HTTP/1.1 connections, where a loss only stalls one of the six.

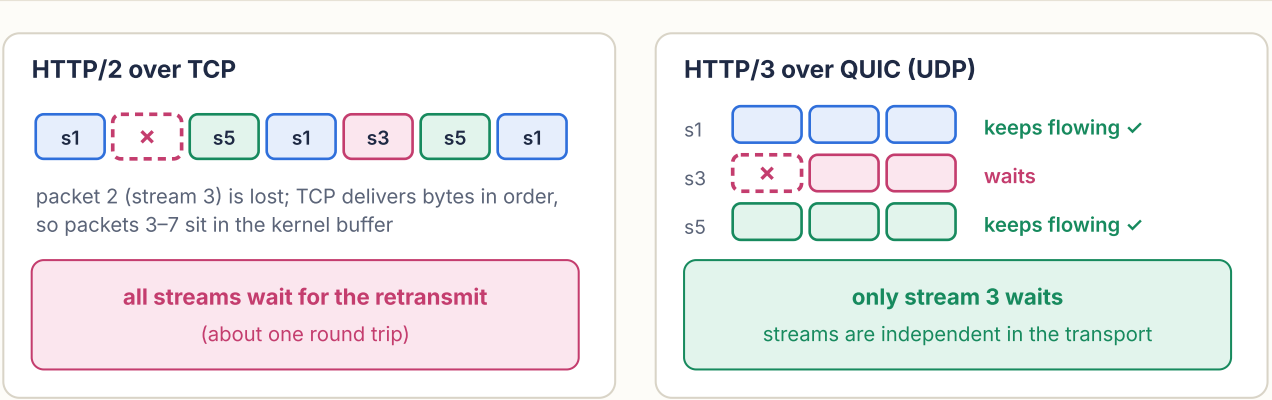


Figure 2. HTTP/2 moved head-of-line blocking from HTTP down to TCP. QUIC tracks ordering per stream, so a lost packet only delays the stream it belongs to.

4 HTTP/3: QUIC over UDP

HTTP/3 runs on **QUIC**, a transport protocol built on top of UDP. It keeps HTTP/2's ideas (streams, header compression, now called QPACK) and fixes the transport:

- **Independent streams.** QUIC handles ordering and retransmission per stream, so a lost packet only stalls its own stream.
- **Faster setup.** TLS 1.3 is built into QUIC, so the transport and encryption handshakes happen together in **one round trip**, or **zero** on resumption (0-RTT, idempotent requests only).
- **Connection migration.** A connection is identified by a **connection ID**, not by IP address and port. When your phone moves from Wi-Fi to 4G, the connection migrates instead of starting over.

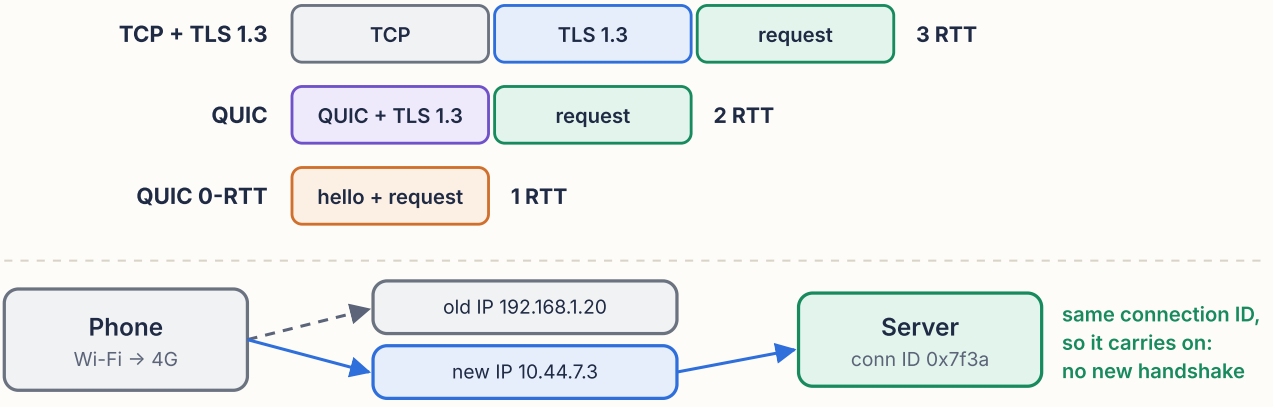


Figure 3. Top: round trips before the first response byte. QUIC merges the TCP and TLS handshakes. Bottom: the client's IP changes, but the connection ID doesn't, so the server keeps the same connection.

Why build on UDP instead of fixing TCP? TCP lives inside operating-system kernels, and countless routers, firewalls and middleboxes expect it to behave in a fixed way (it has **ossified**). Changing it would take decades. UDP passes through almost everywhere, and QUIC runs in **user space**, inside the browser or server, so it evolves as fast as they update. Some networks still block UDP, so browsers learn about HTTP/3 from an `Alt-Svc` header (or DNS) and fall back to HTTP/2 over TCP if QUIC fails.

	HTTP/1.1	HTTP/2	HTTP/3
Format	Text	Binary frames	Binary frames
Transport	TCP	TCP	QUIC over UDP
Requests per connection	One at a time	Many streams, multiplexed	Many independent streams
Head-of-line blocking	HTTP level	TCP level (packet loss)	Only within one stream
Headers	Repeated in full	HPACK	QPACK
Setup (with TLS 1.3)	2 RTT	2 RTT	1 RTT, or 0 on resume
Network change	Reconnect	Reconnect	Migrates (connection ID)

WORKED EXAMPLE

A page loads 60 files over a 100 ms round-trip mobile link with 2% packet loss. HTTP/1.1: 6 connections × (TCP + TLS) handshakes, and 60 requests queued about 10 deep per lane, so the page needs at least ~12 round trips (≈ 1.2 s) even with fast servers. HTTP/2: one handshake, all 60 requests sent at once, so far fewer round trips; but with 2% loss, each lost packet freezes all 60 streams for about a round trip. HTTP/3: one combined handshake (100 ms less), and a loss only delays the one file it belongs to.

COMMON MISTAKES

- Saying HTTP/2 removed head-of-line blocking completely: it moved it to TCP.
- Keeping HTTP/1.1 tricks on HTTP/2 (domain sharding, sprite sheets, giant bundles): extra domains mean extra connections and lose multiplexing.
- Assuming HTTP/3 always works: corporate firewalls may block UDP 443; keep the HTTP/2 fallback.
- Thinking a new version changes your API: methods, status codes and headers mean the same thing.

5 Key terms

Keep-alive: reuse one TCP connection for many requests

Multiplexing: interleaving frames of many streams

QUIC: UDP-based transport with built-in TLS 1.3

HOL blocking: one slow or lost item delays everything behind it

HPACK / QPACK: header compression for HTTP/2 / HTTP/3

Connection ID: identifies a QUIC connection, not IP + port

Stream: one request/response inside a connection

ALPN: TLS extension that picks h2 or http/1.1

Ossification: middleboxes freezing a protocol's behaviour

6 Interview questions

Q1. What problem does HTTP/2 solve? BASIC

HTTP/1.1 handles one request at a time per connection, so browsers open ~6 connections and a slow response blocks its lane. HTTP/2 multiplexes many streams over one connection and compresses headers with HPACK.

Q2. Why was HTTP/3 built on UDP? BASIC

To escape TCP's head-of-line blocking and its ossified, kernel-bound design. QUIC on UDP runs in user space, so it can offer independent streams, a combined 1-RTT handshake with TLS 1.3, and connection migration, and still evolve quickly.

Q3. What is head-of-line blocking? BASIC

When one slow or lost item delays everything queued behind it. In HTTP/1.1 it's a slow response on a connection; in HTTP/2 it's a lost TCP packet stalling all streams; HTTP/3 limits it to one stream.

Q4. Your mobile users on flaky networks see slow loads even after moving to HTTP/2. Why, and what helps? SCENARIO

Packet loss: every lost TCP packet stalls all streams on the single connection. Enable HTTP/3 at the CDN or load balancer (with HTTP/2 fallback); QUIC limits stalls to one stream and survives Wi-Fi/4G switches without reconnecting.

Q5. Why does gRPC use HTTP/2? SCENARIO

Streams: one connection carries thousands of concurrent calls, binary framing suits protobuf, and long-lived streams allow client, server and bidirectional streaming calls. It also gets HPACK and per-stream flow control.

Q6. You enabled HTTP/3 but some office users still use HTTP/2. Is something broken? SCENARIO

Probably not. Their firewall likely blocks UDP 443, so the browser falls back to TCP. Browsers discover HTTP/3 via Alt-Svc or DNS HTTPS records and use it only when QUIC actually connects.

Q7. Can HTTP/2 be slower than HTTP/1.1? TRICKY

Yes, on lossy links. With six HTTP/1.1 connections, a lost packet stalls one of them; with one HTTP/2 connection, it stalls every stream, and one connection also gets a single share of TCP congestion control.

Q8. Why can't we just fix head-of-line blocking in TCP? TRICKY

TCP is implemented in OS kernels and inspected by routers, firewalls and NATs that drop anything unexpected. Deploying a change takes many years. QUIC encrypts nearly all its headers, so middleboxes can't freeze its behaviour again.

Next: SD-B-014 · WebSockets backendroadmap.com

WebSockets

upgrade handshake, heartbeats and scaling

Watch the 3-min video

In one line: a WebSocket starts as an HTTP request, upgrades with `101 Switching Protocols`, and becomes one long-lived, full-duplex connection; the hard part is that every connection is state pinned to one server.

1 Why WebSockets

A chat app needs messages to appear instantly, in both directions. With plain HTTP the client must keep asking “anything new?”, and the server can only answer, never start a conversation. A **WebSocket** fixes that: one long-lived connection where **either side can send at any time**. Use `wss://` (WebSocket over TLS) in production.

2 The upgrade handshake

A WebSocket starts as a normal HTTP/1.1 request that asks to switch protocols. After one exchange, the connection belongs to WebSocket:

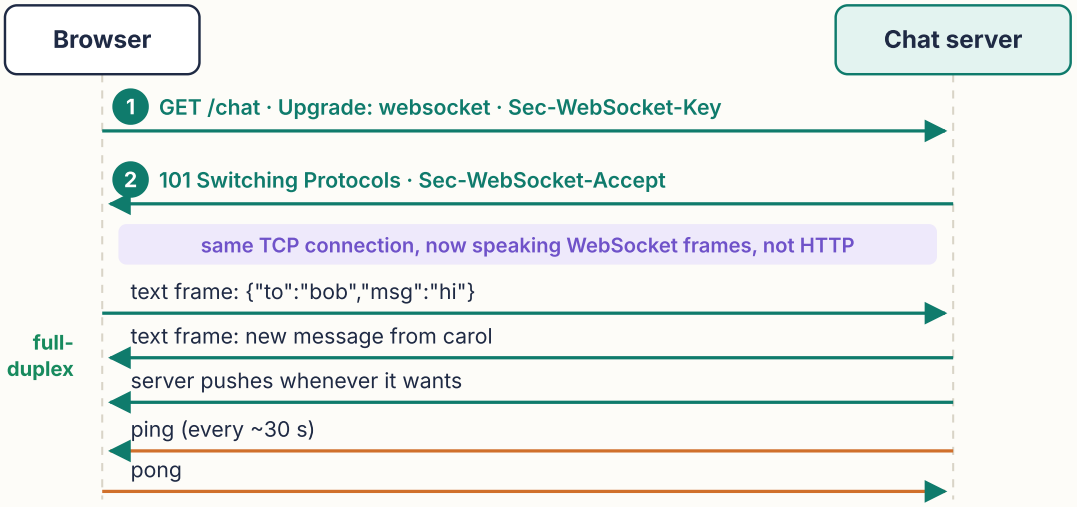


Figure 1. After the 101 response, both sides send frames whenever they like. Ping and pong are small control frames that keep the connection alive and prove the peer is still there.

On the wire, the client sends a random key and the server answers with a value derived from it:

```
GET /chat HTTP/1.1
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Sec-WebSocket-Version: 13
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+xOo=
```

The server proves it really speaks WebSocket by returning an **accept value derived from the key** (Base64 of SHA-1 of the key plus a fixed GUID). From then on, the **same TCP connection** carries WebSocket frames, not HTTP. Any other status (400, 401, 404, 200) means the upgrade failed and is debugged like any HTTP error.

3 Full-duplex frames and heartbeats

The connection is **full-duplex**: client and server both send whenever they want, without waiting for each other. Messages travel as **frames**, text (UTF-8) or binary, with just **2 to 14 bytes** of header each (client frames add a 4-byte mask), instead of hundreds of bytes of HTTP headers.

Connections live for hours, so you need **heartbeats**. **Ping** and **pong** control frames every ~30 seconds:

- prove the other side is still there (a phone that lost signal sends no close frame, so the socket looks open forever);
- stop load balancers and NAT gateways from closing a connection that looks idle (AWS ALB and nginx's `proxy_read_timeout` both default to 60 s);
- if pongs stop (say, two missed in a row), **close the connection and clean up**: remove it from the registry and free its memory. (Browsers answer pings automatically but JavaScript can't send them, so the server sends the pings.)

4 Example: a chat server pushing messages

Alice and Bob each hold a WebSocket to a **gateway** server, but not necessarily the same one. When Alice sends a message, her gateway stores it, looks up which gateway holds Bob's connection, and publishes it there; that gateway pushes it down Bob's socket. No polling at all.

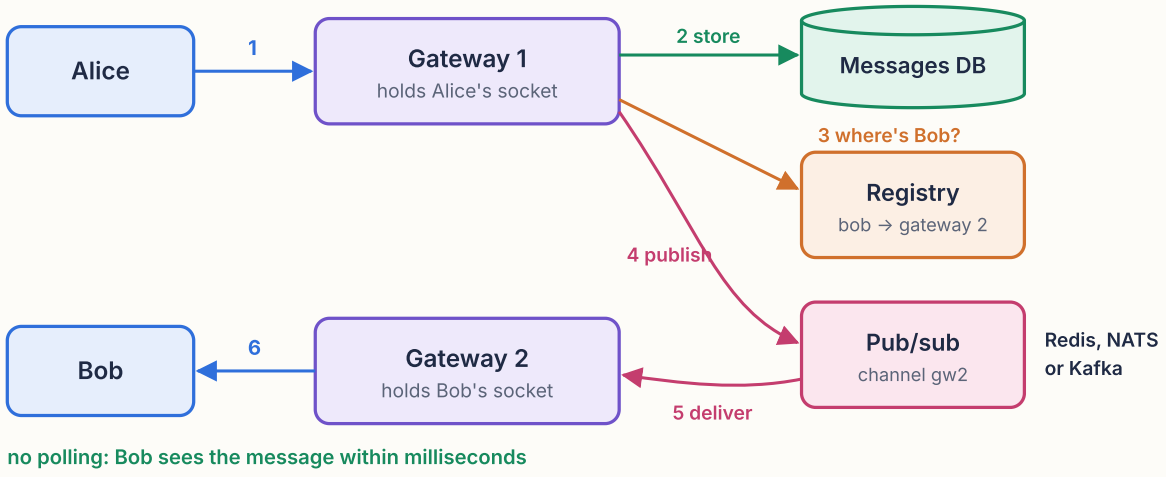


Figure 2. Gateways only hold sockets. A registry says who is connected where, and pub/sub carries messages between gateways. Storing first means Bob can still fetch the message if he is offline.

5 Scaling: every connection is state

An HTTP request lasts milliseconds; a WebSocket lasts hours, **pinned to one server**. That changes how you scale:

- **Load balancer** must support the upgrade (nginx needs `proxy_http_version 1.1` and the Upgrade/Connection headers set explicitly) and allow **long idle timeouts**, above the heartbeat interval.
- **Per-server limits** are memory and file descriptors, not CPU: each socket needs a file descriptor and some kilobytes of buffers. A tuned server (raised `ulimit -n`, async I/O) holds **hundreds of thousands** of mostly idle connections.

- **Scale out** with many gateway servers, a **registry** of who is connected where (e.g. Redis with a TTL refreshed by heartbeats), and **pub/sub** to route messages between servers.

WORKED EXAMPLE

Target: **2 million** concurrent users. At ~20 KB per idle connection, 2M × 20 KB ≈ **40 GB** of memory across the fleet. If one gateway safely holds 100,000 connections, you need 20 gateways; run **30** so that losing a server or draining one for a deploy doesn't overload the others. Heartbeats every 30 s mean 2M / 30 ≈ **67,000 pings per second** fleet-wide, tiny per server. Keep the load balancer idle timeout above 30 s, for example 120 s.

6 Plan for reconnection

When a server restarts, all its clients drop at once. If they all reconnect immediately, they **stampede** the next server (a thundering herd). So clients retry with **exponential backoff plus random jitter** (e.g. wait a random time up to 1 s, then up to 2 s, 4 s ... capped at 30 s), and **resume from the last message ID they saw** so nothing is lost. During deploys, **drain** servers gradually: stop sending new connections to a server, then close its sockets in batches with a "going away" close (code 1001), instead of killing them all at once.

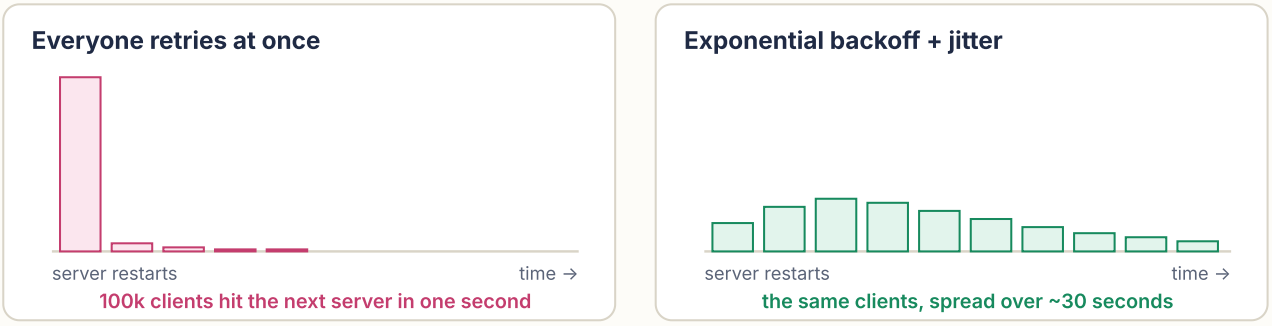


Figure 3. Left: a synchronized retry spike can knock over the server that was supposed to take the load. Right: random jitter spreads the same reconnects out.

COMMON MISTAKES

- No heartbeats: dead connections pile up, and idle-timeout proxies cut live ones at 60 s.
- Keeping the "who is connected" map only in one process's memory: messages for users on other servers vanish.
- Reconnecting instantly in a tight loop, with no backoff or jitter.
- Using WebSockets for one-way updates where server-sent events or plain HTTP would be simpler (next lesson).

7 Key terms

Upgrade: HTTP mechanism to switch protocols

Frame: unit of a WebSocket message (text, binary, control)

Connection registry: maps user → gateway

101 Switching Protocols: server agrees to switch

Ping / pong: control frames used as heartbeats

Backoff + jitter: growing, randomized retry delays

Full-duplex: both sides send at any time

Gateway: server that holds client sockets

Draining: closing connections gradually before shutdown

8 Interview questions

Q1. How does a WebSocket connection start? BASIC

As an HTTP/1.1 GET with Upgrade: websocket, Connection: Upgrade and a random Sec-WebSocket-Key. The server answers 101 Switching Protocols with a Sec-WebSocket-Accept derived from the key. Then the same TCP connection carries WebSocket frames.

Q2. Why do WebSockets need heartbeats? BASIC

To detect dead peers (a dropped phone sends no close frame) and to keep load balancers and NATs, which often cut idle connections after ~60 s, from closing healthy ones. Ping every ~30 s; close after missed pongs.

Q3. How do you scale to millions of WebSocket connections? SCENARIO

Many gateway servers (~100k connections each) behind a load balancer that supports upgrades and long idle timeouts; a registry of user → gateway; pub/sub between gateways; heartbeats; and clients that reconnect with backoff and jitter. For 2M users: about 20 gateways, plus spare capacity.

Q4. Users on gateway 1 don't receive messages sent by users on gateway 2. What's wrong? SCENARIO

Each gateway only knows its own sockets. Add pub/sub (Redis, NATS, Kafka) and a registry, so the sender's gateway can publish to the gateway that holds the recipient.

Q5. Every deploy causes a spike of errors and a CPU storm. Why, and how do you fix it? SCENARIO

Killing a server drops all its sockets and every client reconnects at once. Drain gradually (stop new connections, close in batches with code 1001), and make clients use exponential backoff with jitter and resume from their last message ID.

Q6. Connections drop exactly every 60 seconds when nobody is chatting. Why? TRICKY

A proxy or load balancer idle timeout (e.g. nginx proxy_read_timeout or ALB idle timeout, both 60 s by default). Send heartbeats more often than the timeout, e.g. every 30 s, and raise the timeout.

Q7. Can you use sticky sessions instead of pub/sub? TRICKY

Stickiness only decides where one client lands. Alice and Bob can still land on different servers, so messages still need to cross servers. Use a registry and pub/sub, or route everyone in a room to the same server by consistent hashing.

Next: SD-B-015 · SSE vs Long Polling vs WebSockets backendroadmap.com

SSE vs Long Polling vs WebSockets

choosing how the server pushes updates

Watch the 3-min video

> **In one line:** short polling asks on a timer, long polling holds each request until there is news, server-sent events keep one HTTP response open and stream events down it, and WebSockets give a two-way channel. Pick the simplest one that fits the direction of your data.

When you chat with an AI assistant, the answer appears word by word. That is usually **server-sent events (SSE)**: one HTTP response that stays open while the server keeps pushing tokens. Here is how it compares with polling, long polling and WebSockets.

1 Short polling and long polling

Short polling is the simplest: the client asks every few seconds, “anything new?”. Most answers are empty, which wastes requests, and an update can wait up to a full interval before anyone sees it (on average half of it).

Long polling is smarter. The client sends a request and the server **holds it open** until there is news, or until a timeout (say 30 seconds), then answers. As soon as a response arrives, the client sends the next request. Updates arrive almost instantly, and it works through any HTTP infrastructure, which makes it a great **fallback**. The cost: every message needs a full request and response, with headers, and there is a small gap while the next request is on its way.

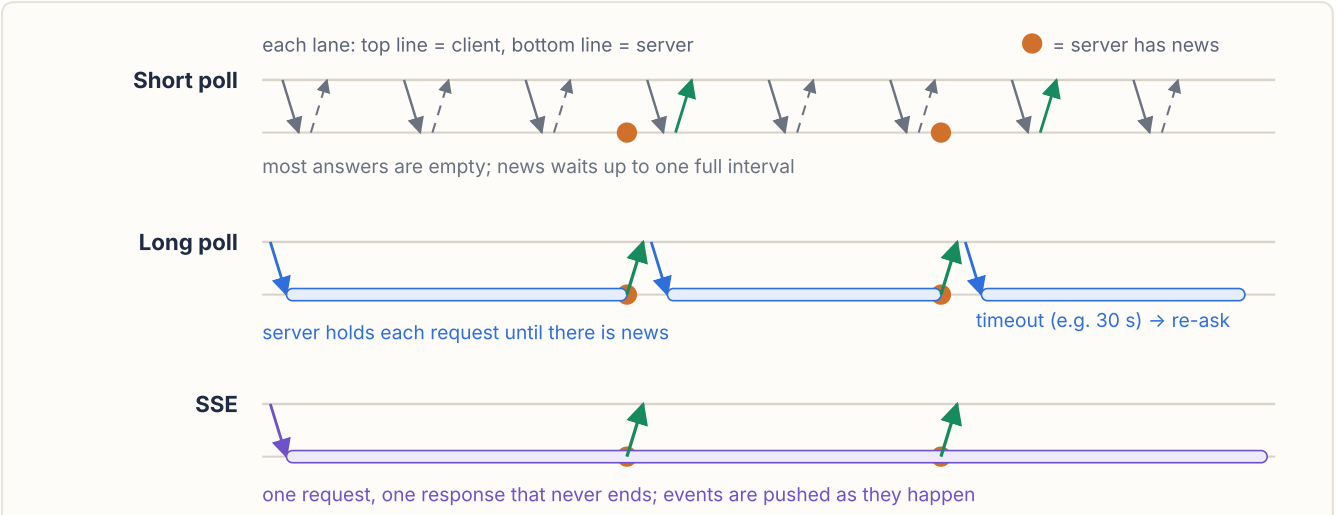


Figure 1. Three ways to get the two news events (orange) to the client. Short polling finds them late and asks many empty questions; long polling and SSE deliver them almost at once.

2 Server-sent events

With **SSE** the browser opens one request with the `EventSource` API. The server replies with `Content-Type: text/event-stream` and **never closes** the response. It writes events as they happen, as plain text: lines starting with `data:`, with a **blank line** ending each event.

```
id: 42
event: score
data: {"home": 2, "away": 0}

: keep-alive (a comment line, ignored)
```

Each event can carry an `id`. If the connection drops, the browser **reconnects automatically** (after a few seconds, or the `retry:` value the server set) and sends a `Last-Event-ID` header, so the server can **resume** from where it stopped.

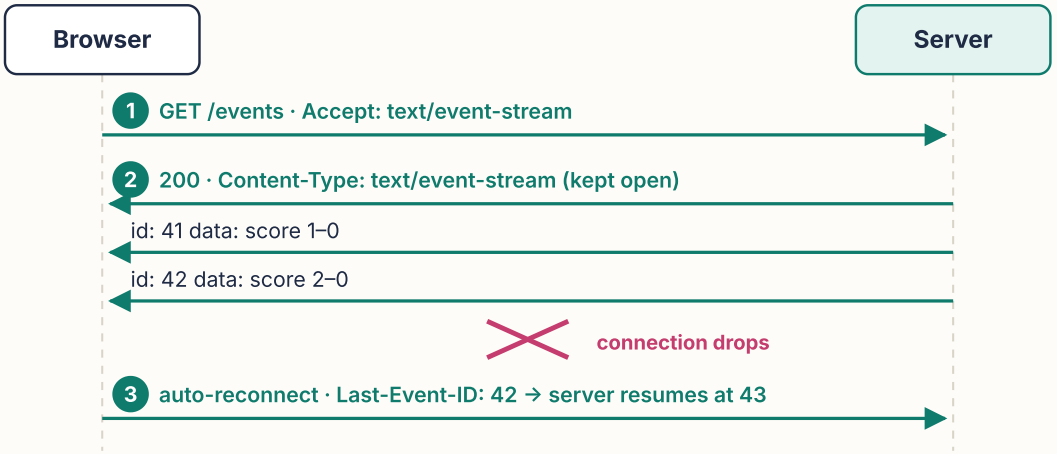


Figure 2. SSE recovery is built in: the browser reconnects by itself and tells the server the last event it saw. With WebSockets you write this logic yourself.

3 Example: streaming LLM tokens from FastAPI

The endpoint returns a `StreamingResponse` with the event-stream media type. An **async generator** yields each token as a `data:` line the moment the model produces it.

```
import json
from fastapi import FastAPI
from fastapi.responses import StreamingResponse
app = FastAPI()

@app.get("/chat/stream")
async def chat_stream(q: str):
    async def events():
        async for token in llm.stream(q):   # your model client
            yield f"data: {json.dumps(token)}\n\n"
        yield "data: [DONE]\n\n"
    return StreamingResponse(events(), media_type="text/event-stream",
                             headers={"Cache-Control": "no-cache", "X-Accel-Buffering": "no"})
```

The user sees the first words in a few hundred milliseconds instead of waiting ten seconds for the full answer. JSON-encoding each token keeps newlines inside a token from breaking the format, and `X-Accel-Buffering: no` stops nginx from buffering the stream.

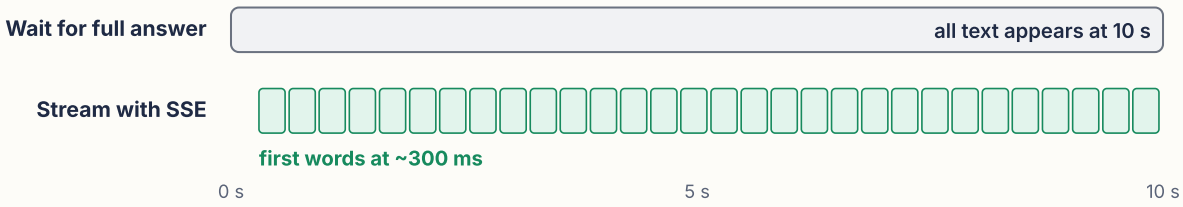


Figure 3. Same model, same total time. Streaming changes when the user starts reading: time to first token, not time to last token, is what feels fast.

4 How they compare

	Short polling	Long polling	SSE	WebSockets
Direction	Client pulls	Client pulls, server waits	Server → client	Both ways
Latency	Up to one interval	Near real time	Real time	Real time
Cost per message	Full request, often empty	Full request	A few bytes	2–14 bytes
Protocol	Plain HTTP	Plain HTTP	Plain HTTP	Upgrade to WebSocket
Reconnect	n/a	Next request	Built in, with Last-Event-ID	You build it
Infrastructure	Anything	Anything (long timeouts)	Anything that doesn't buffer	Upgrade-aware LBs, long timeouts

5 Choosing

- **Choose SSE** when updates flow mainly from server to client: notifications, live scores, dashboards, job progress and AI token streams. It is plain HTTP, so it works with HTTP/2, proxies, CDNs and your normal cookie or header authentication.
- **Choose WebSockets** when the client also sends frequent messages: chat, collaborative editing, multiplayer games.
- **Keep long polling** as a fallback for networks or proxies that break streaming, and short polling for rare, non-urgent checks.

CAVEAT: THE SIX-CONNECTION LIMIT

Over HTTP/1.1, browsers allow only about **six connections per host**, shared by all tabs. Each open EventSource uses one, so six tabs of your dashboard can freeze every other request to that host. Serve SSE over **HTTP/2**, where each stream is just one of ~100 multiplexed streams on a single connection.

WORKED EXAMPLE

A dashboard with 10,000 open users, and about 1 update per minute each. Short polling every 5 s: 10,000 / 5 = **2,000 requests/s**, about 98% of them empty, and updates arrive 2.5 s late on average. SSE: 10,000 idle connections and about 10,000 / 60 ≈ **170 small events/s**, delivered at once. Same features, roughly a tenth of the traffic.

COMMON MISTAKES

- A proxy or gzip layer buffering the stream, so events arrive in one lump at the end.
- Serving SSE over HTTP/1.1 and hitting the six-connection limit across tabs.
- Picking WebSockets for one-way updates and then building reconnection and auth by hand.

6 Key terms

Short polling: ask on a fixed interval

Long polling: server holds the request until news or timeout

SSE: one long HTTP response carrying events

EventSource: browser API for SSE

text/event-stream: SSE content type

Last-Event-ID: header sent on reconnect, to resume

StreamingResponse: FastAPI response fed by a generator

Time to first token: delay before the first words appear

Fallback: simpler transport used when the best one fails

7 Interview questions

Q1. When would you choose SSE over WebSockets? BASIC

When data flows one way, server to client, and you want plain HTTP, automatic reconnects with Last-Event-ID and simple infrastructure: notifications, dashboards, streaming LLM tokens.

Q2. How does long polling work? BASIC

The client sends a request that the server holds open until there is data or a timeout (e.g. 30 s). Then the client immediately sends another. Near real-time, works through any HTTP infrastructure, but costs a full request per message.

Q3. Design token streaming for an AI chat product. SCENARIO

POST the prompt, return a text/event-stream response, and yield each token as a data: line from an async generator; send a final [DONE] event. Disable proxy buffering, serve over HTTP/2, send keep-alive comments every ~15 s, and cancel generation if the client disconnects.

Q4. Users say live scores arrive all at once at the end. What's wrong? SCENARIO

Something buffers the response: nginx proxy_buffering, a gzip middleware, or a CDN. Set X-Accel-Buffering: no (or turn buffering off), don't compress the stream, and set Cache-Control: no-cache.

Q5. Your dashboard freezes when a user opens it in seven tabs. Why? SCENARIO

Over HTTP/1.1 the browser allows about six connections per host; six EventSource streams take them all and the seventh request waits. Serve over HTTP/2, or share one stream across tabs (e.g. a SharedWorker).

Q6. EventSource can't set an Authorization header. How do you authenticate SSE? TRICKY

Use cookies (sent automatically, withCredentials for cross-origin), or a short-lived token in the URL, or read the stream with fetch() instead of EventSource, which allows headers and POST. That is how most LLM APIs are consumed.

Q7. Is long polling obsolete? TRICKY

No. It needs nothing special from proxies or firewalls, so it is a reliable fallback; Socket.IO, for example, can start on long polling and upgrade to WebSockets. It is just less efficient per message than SSE.

Next: SD-B-016 · HTTP Streaming & Chunked Transfer backendroadmap.com

HTTP Streaming

chunked transfer, backpressure and gRPC streams

Watch the 3-min video

In one line: instead of building a huge response in memory, send it in pieces as you produce it (chunked transfer in HTTP/1.1, data frames in HTTP/2), read the data lazily, and let TCP backpressure keep memory flat.

1 Why stream, and how it works on the wire

A user clicks **Export** and asks for five million orders as a CSV file. If the server builds the whole file in memory first, it may need gigabytes of RAM. The user stares at a spinner for a minute, or the server simply runs out of memory and crashes. The fix is **streaming**: send the file in pieces while you are still producing it.

In **HTTP/1.1** a server normally sends a `Content-Length` header, so it must know the body size before it sends the first byte. With `Transfer-Encoding: chunked` it doesn't. The body goes out as a series of chunks; each one starts with its size in hexadecimal, and a **zero-length chunk** marks the end.

HTTP/2 and HTTP/3 don't use chunked encoding at all (it is not allowed there). A body is already a series of **DATA frames**, and the last one carries an end-of-stream flag. Same idea, built into the protocol.

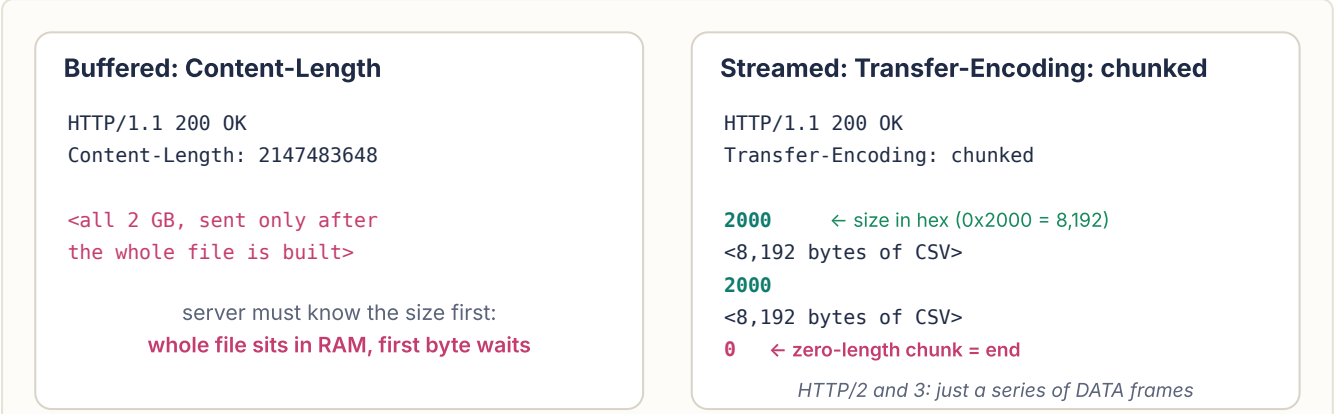


Figure 1. Left: with Content-Length the server must finish the whole body first. Right: chunked encoding sends each piece with its size, and a 0 chunk ends the body.

So the server can start sending before it has finished computing: the **first bytes leave in milliseconds**. **Memory stays flat**, because only one chunk is held at a time. And the client can **process data as it arrives**.

2 Example: a streaming CSV export in FastAPI

Read rows with a **server-side cursor**, in batches of about 1,000, so the database driver never loads the full result. A **generator** turns each batch into CSV lines and yields them. Return a `StreamingResponse` with media type `text/csv` and a `Content-Disposition` header so the browser saves it as a file.

WORKED EXAMPLE

```
async def rows(): async for batch in cursor_batches(size=1000): yield to_csv(batch)
return StreamingResponse(rows(), media_type="text/csv", headers={"Content-Disposition":
"attachment; filename=orders.csv"})
```

Five million rows at about 100 bytes each is roughly **500 MB** of CSV. Buffered, that is 500 MB or more in RAM per export; ten users at once is 5 GB. Streamed, each export holds one batch: 1,000 × 100 bytes ≈ **100 KB**, and the download starts immediately. (SQLAlchemy: `stream_results=True` + `yield_per(1000)`; psycopg: a named cursor.)

3 Backpressure: when the client reads slowly

What if the client downloads slowly, say on a weak mobile link? **TCP flow control** handles it. The client's receive window fills, then the server's send buffer fills. Once that buffer is full, the server's write call waits. In an async framework, the generator simply isn't asked for the next chunk until there is room.

So a slow reader automatically slows down the producer, instead of data piling up in memory. This only works if you produce lazily: a generator that builds a list of every row first has already lost.

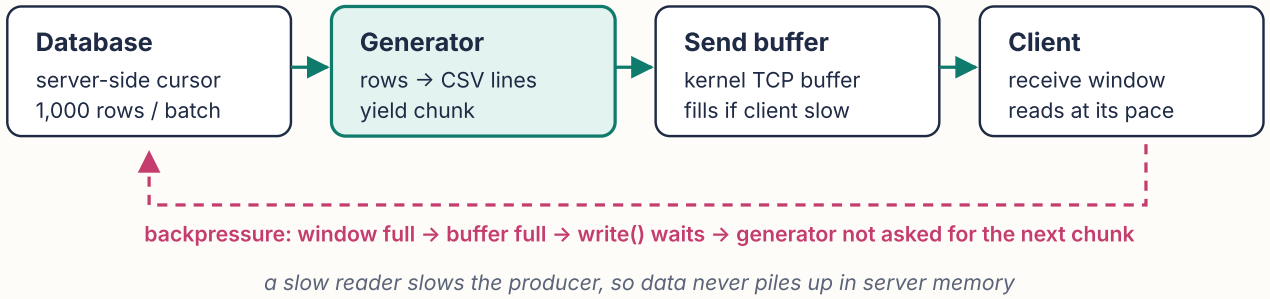


Figure 2. Data flows left to right; pressure flows back right to left. Each stage only pulls the next piece when the stage after it has room.

4 gRPC: streaming as a first-class idea

gRPC runs over HTTP/2 and builds streaming into its contract. A method in the `.proto` file can mark either side as a `stream`, giving four call types:

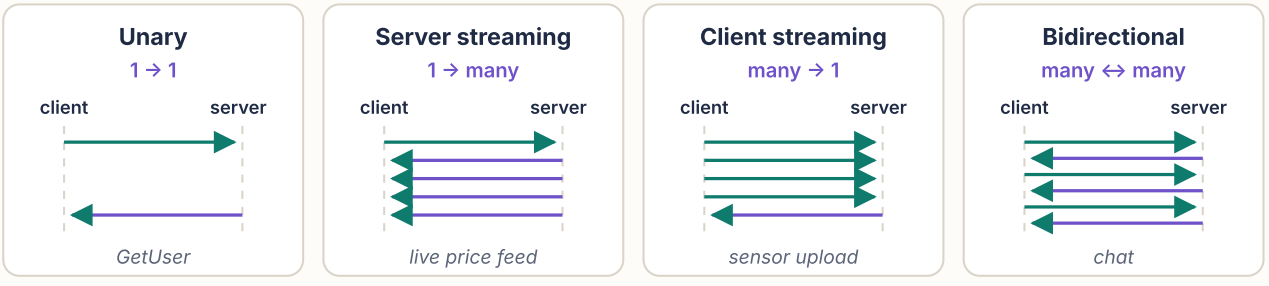


Figure 3. Black arrows are client messages, purple are server messages. In bidirectional mode both sides send independently, at any time.

5 Pitfalls

- **Buffering proxies.** A proxy may hold the whole response before forwarding it. In NGINX set `proxy_buffering off;` or send `X-Accel-Buffering: no`.

- **Errors mid-stream.** The status `200` was sent with the first chunk, so a failure halfway cannot change it. Add an error marker or a row count / checksum at the end. (gRPC sends its status in HTTP/2 trailers, after the data, so it handles this.)
- **Timeouts.** Set idle and total timeouts so a stuck client can't hold a connection open forever.

COMMON MISTAKES

- Using a `StreamingResponse` but fetching all rows first with `fetchall()`: memory is still huge.
- Testing locally (works) and deploying behind a buffering proxy (download starts only at the end).
- Assuming a 200 means the file is complete.

6 Key terms

Chunked transfer: body as size-prefixed chunks

DATA frame: HTTP/2 and 3 unit of body data

Backpressure: slow consumer slows the producer

Content-Length: body size, sent before the body

Server-side cursor: rows fetched in batches

Trailers: headers sent after the body

7 Interview questions

Q1. How do you stream a large HTTP response? BASIC

Send it in pieces as you produce it: `Transfer-Encoding: chunked` in HTTP/1.1, or DATA frames in HTTP/2 and 3. Yield rows lazily from a server-side cursor, so memory stays at one batch, not the whole file.

Q2. What does chunked transfer encoding look like on the wire? BASIC

Each chunk is its size in hex, CRLF, the bytes, CRLF. For example `2000` then 8,192 bytes. A chunk of size `0` ends the body, optionally followed by trailers. No Content-Length is sent.

Q3. What streaming modes does gRPC support? BASIC

Four: unary (1 request, 1 response), server streaming (1 request, many responses), client streaming (many requests, 1 response) and bidirectional streaming (both sides stream at once).

Q4. Streaming works locally, but in production the download starts only after 60 s. Why? SCENARIO

Something in between is buffering the whole response, usually NGINX with `proxy_buffering on` (the default) or a compression layer. Turn buffering off for that route or send `X-Accel-Buffering: no`.

Q5. A 1 Mbit/s client downloads a 500 MB export. Does server memory grow? SCENARIO

No, if the producer is lazy. TCP flow control fills the client's window and the server's send buffer, then the write waits and the generator pauses. But it takes 500 × 8 / 1 ≈ 4,000 s (over an hour), so set a total timeout.

Q6. The database fails at row 3 million of a streamed export. How does the client find out? TRICKY

The status 200 is already sent, so it can't change. Close the connection abnormally (in HTTP/1.1, no final 0 chunk; in HTTP/2, a stream reset) and/or write an explicit error marker, and include a row count or checksum at the end so clients can detect a truncated file.

Q7. Why can't you use Transfer-Encoding: chunked with HTTP/2? TRICKY

HTTP/2 forbids it: the body is already split into DATA frames with their own lengths, and the end-of-stream flag marks the end. Proxies convert between the two.

Next: SD-B-017 · Forward vs Reverse Proxy backendroadmap.com

Forward vs Reverse Proxy

egress control, TLS termination and NGINX

Watch the 3-min video

In one line: a forward proxy acts for clients, controlling and hiding their outbound traffic; a reverse proxy acts for servers, sitting in front of them, hiding the backends and taking work off the app.

1 Two machines in the middle

Two kinds of machines sit in the middle of web traffic, and both are called **proxies**. One works for the clients. The other works for the servers. Mixing them up is a classic interview slip.

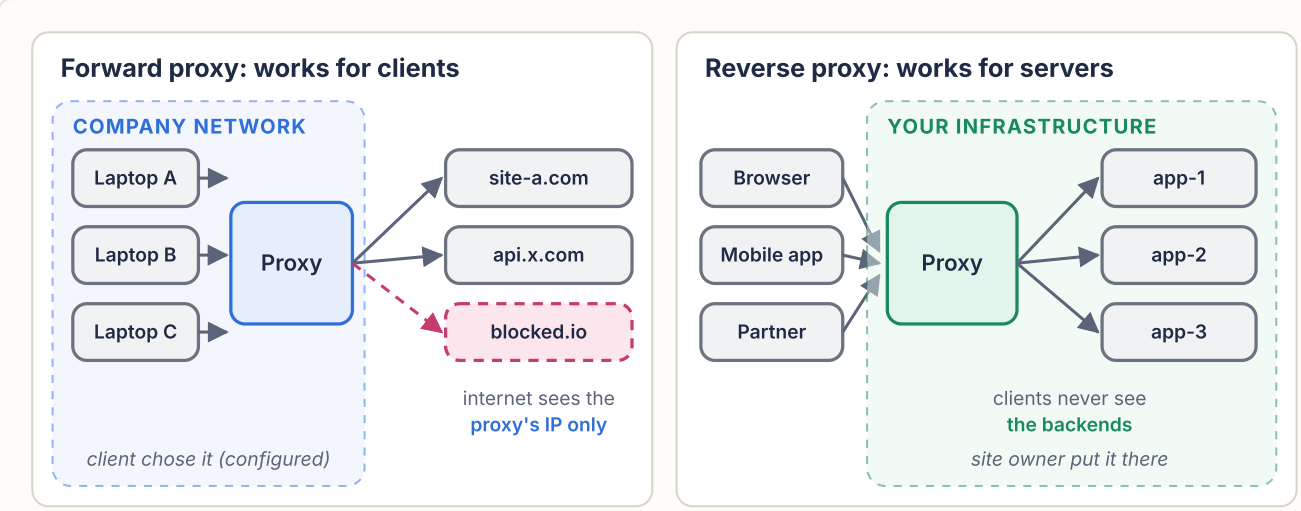


Figure 1. Left: laptops send outbound requests through a forward proxy, which can block sites. Right: every client hits the reverse proxy, which forwards to a backend the client never sees.

2 Forward proxy: egress control for clients

A **forward proxy** sits in front of clients. Laptops inside a company send their outbound requests to the proxy, and the proxy talks to the internet on their behalf. Websites see the proxy's address, not the employee's.

Companies use it for **egress control**: blocking certain sites, enforcing **allow-lists** so servers can only call approved APIs (a payment provider, a package mirror), **logging** who called what, and **caching** shared downloads. Squid is a classic example. In the cloud, an egress proxy or a firewall in front of a **NAT gateway** plays the same role: the NAT gives private servers one public address, and the firewall decides which domains they may reach.

3 Reverse proxy: the front door for servers

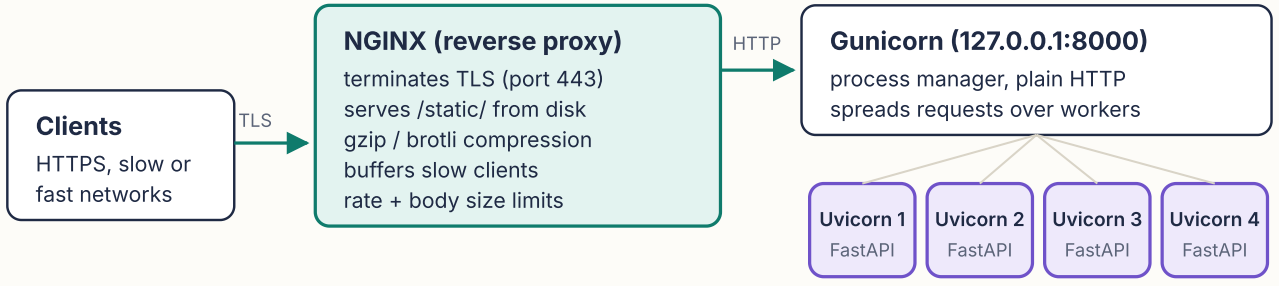
A **reverse proxy** sits in front of servers. Clients think they are talking to your site, but they are really talking to the proxy, which forwards each request to one of your backends. The client never sees the backends.

Ask	Forward proxy	Reverse proxy
Who put it there?	The client side (browser or OS setting, corporate policy)	The site owner; clients don't know it exists
Whose identity does it hide?	Hides the clients from servers	Hides the servers from clients
Typical jobs	Block sites, allow-list APIs, log, cache	TLS, routing, load balancing, caching, compression
Examples	Squid, corporate web proxy, cloud egress firewall	NGINX, HAProxy, Envoy, API gateway, cloud load balancer

A reverse proxy takes a lot of work off app servers:

- **TLS termination**: the proxy holds the certificates and decrypts HTTPS, so apps don't.
- **Routing** by host or path (`api.` to one service, `/images/` to another) and **load balancing** across instances.
- **Caching** static files and responses, and **compression** with gzip or brotli.
- **Protection**: rate limiting, request size limits, and **buffering slow clients**.

4 The classic setup: NGINX, Gunicorn, Uvicorn



Python workers only see fast, complete, plain-HTTP requests for dynamic pages

Figure 2. NGINX faces the internet and does the generic work. It forwards only dynamic requests to Gunicorn on a local port, which spreads them across Uvicorn worker processes running the FastAPI app.

WORKED EXAMPLE: WHY BUFFERING SLOW CLIENTS MATTERS

Your app runs 4 Uvicorn workers. A user on a weak mobile link uploads a 2 MB form at 100 KB/s: that is **20 seconds** of sending. Without a proxy, a worker holds that connection for the whole upload. NGINX reads the full request body first (`client_max_body_size` rejects anything too large), then hands Gunicorn the complete request over localhost in about a millisecond. On the way back, NGINX buffers the response and trickles it to the slow client, so the Python worker is free again almost at once.

Config sketch: `location /static/ { root /srv/app; }` and `location / { proxy_pass http://127.0.0.1:8000; proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for; }`

5 Common reverse proxies

NGINX, **HAProxy** and **Envoy** are the usual choices. Envoy adds features for microservices: retries, circuit breaking, rich metrics, and dynamic configuration, and it is the data plane behind service meshes such as **Istio**. **API gateways** and **cloud load balancers** (AWS ALB, Azure Application Gateway) are reverse proxies too, with extra features such as authentication and API keys on top.

COMMON MISTAKES

- Calling a load balancer a “forward proxy”: it acts for the servers, so it is a reverse proxy.
- Logging the proxy's IP as the client IP: read `X-Forwarded-For`, but only trust it from your own proxy.
- Exposing Gunicorn directly on the internet: no TLS, and slow clients tie up workers.
- Assuming a NAT gateway alone controls egress: it hides addresses but does not filter by domain.

6 Key terms

Forward proxy: acts for clients' outbound traffic

Egress control: limiting what inside systems can reach

X-Forwarded-For: header carrying the original client IP

Reverse proxy: acts for servers, in front of them

TLS termination: decrypt HTTPS at the proxy

Service mesh: sidecar proxies (Envoy) per service

7 Interview questions

Q1. What's the difference between a forward and a reverse proxy? BASIC

A forward proxy acts for clients: they send outbound traffic through it, and it controls and hides them. A reverse proxy acts for servers: it sits in front of them, receives all client traffic and hides the backends. Ask who put it there and whose identity it hides.

Q2. What can a reverse proxy offload from app servers? BASIC

TLS termination, routing by host or path, load balancing, caching, gzip or brotli compression, rate limiting, request size limits, and buffering slow clients.

Q3. Your servers must call only Stripe and your package mirror. How do you enforce it? SCENARIO

Put outbound traffic through an egress proxy or a cloud firewall with an allow-list of domains, block direct internet access in the network rules, and log every request. Deny by default; add domains by review.

Q4. Why put NGINX in front of Gunicorn and Uvicorn? SCENARIO

NGINX terminates TLS, serves static files from disk, compresses, enforces limits and buffers slow clients. With 4 workers, four slow uploads at 20 s each would otherwise block the whole app; with NGINX, each worker is busy for milliseconds.

Q5. After adding a reverse proxy, every request in your logs shows the same client IP. Why? SCENARIO

The app now sees the proxy's address. Have the proxy set `X-Forwarded-For` (or the standard `Forwarded` header) and configure the app to trust it only from the proxy's IP, so clients can't fake it.

Q6. Is a load balancer a forward or a reverse proxy? TRICKY

A reverse proxy. The site owner runs it, clients don't know the backends behind it, and it spreads requests across them. API gateways are reverse proxies too.

Q7. Can a corporate forward proxy read your HTTPS traffic? TRICKY

Normally no. The browser sends `CONNECT host:443` and the proxy only tunnels encrypted bytes, so it sees the hostname, not the content. It can read traffic only with TLS interception: the company installs its own root certificate on the laptops and the proxy re-encrypts.

Next: SD-B-018 · CDNs Explained backendroadmap.com

CDNs Explained

edge PoPs, cache keys, origin shield and purges

Watch the 3-min video

In one line: a CDN serves your content from edge servers near users, so handshakes and round trips are short and cache hits never reach the origin; after a deploy, change file names instead of purging.

1 The problem: content far from users

Your store's servers are in Virginia. A shopper in Mumbai loads the home page: forty product images and three JavaScript bundles, each crossing the world at around **200 ms per round trip**. The page crawls. A **content delivery network** (CDN) fixes this by bringing your content close to the user.

A CDN is a set of edge servers, called **points of presence** (PoPs), spread across the world. Your own servers are the **origin**. Users connect to the nearest PoP, often found through **anycast** or **DNS**. If that PoP has the file cached, it serves it right there, in a few milliseconds.

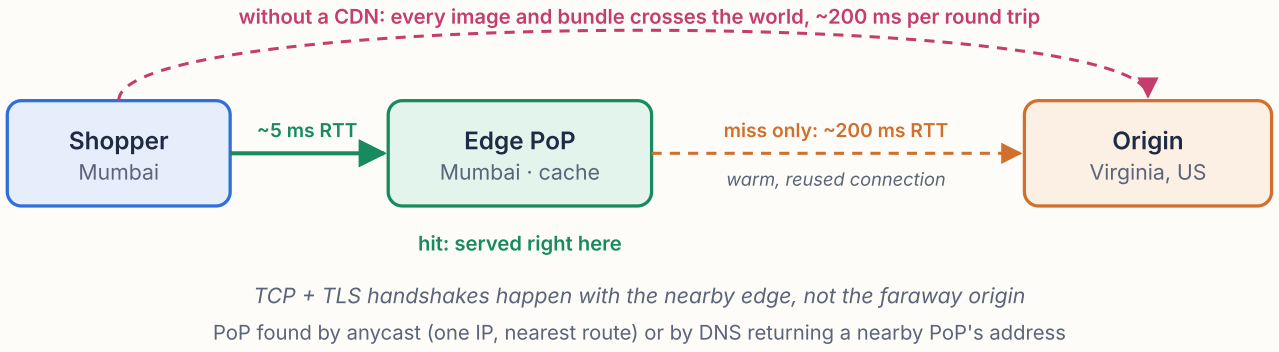


Figure 1. The edge sits a few milliseconds from the user. Only cache misses travel to the origin, over connections the CDN keeps open and warm.

2 How a CDN reduces latency

- **Shorter distance.** The TCP and TLS handshakes happen with a nearby edge, not the faraway origin.
- **Cache hits skip the long trip entirely.**
- **Even misses are faster.** The edge reuses warm, tuned connections back to the origin, so it skips new handshakes.
- **Bonus: offload.** The origin serves far less traffic and survives spikes.

WORKED EXAMPLE

A new HTTPS connection needs about **2 round trips** (TCP, then TLS 1.3) before the first request, and 1 more to get the response. From Mumbai to Virginia: 3 × 200 ms = **600 ms** before the first byte. To a Mumbai PoP at 5 ms: 3 × 5 = **15 ms**, plus a few ms to read from cache. At a **95% hit ratio**, 1 million requests an hour become only **50,000** at the origin (1 in 20).

3 Hit ratio, pull vs push, cache keys and shielding

The key metric is the **cache hit ratio**: the share of requests served from the edge. Most CDNs are **pull CDNs**: on a miss, the edge fetches the file from the origin, caches it and serves later requests itself. **Push CDNs** work the other way: you upload content ahead of time, which suits large, rarely changing files such as video libraries or software downloads.

Each cached object is stored under a **cache key**: usually the URL, plus any headers, cookies or query parameters you choose to include. Include too much, like a session cookie, and every user gets their own copy, so the hit ratio collapses. **Origin shielding** adds a middle cache layer: edge misses go to one shield PoP first, so the origin isn't hit by hundreds of edges at once.

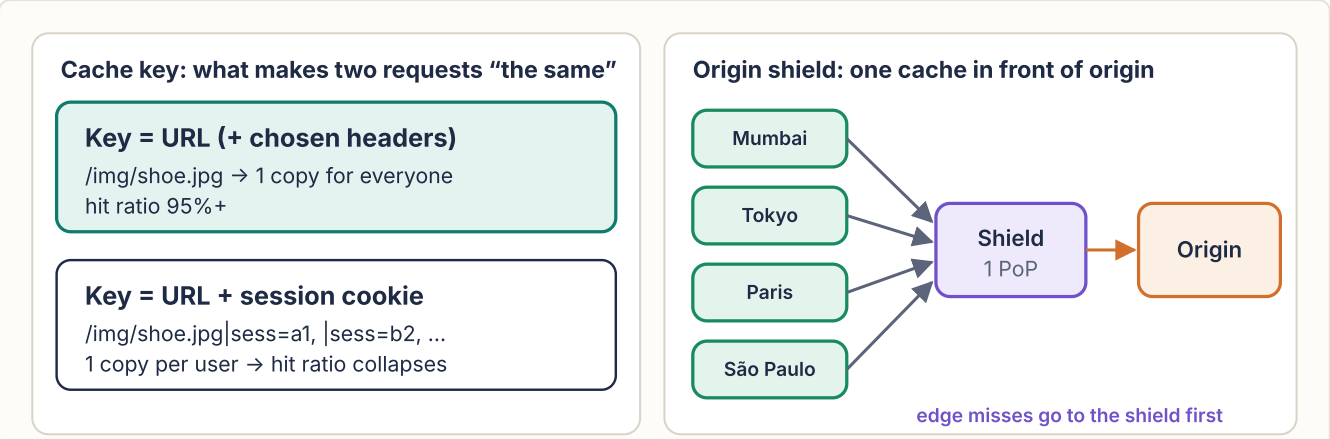


Figure 2. Left: keep the cache key small, or one image becomes thousands of copies. Right: with a shield, a new file costs the origin one request, not one per PoP.

4 Example: images and bundles on CloudFront or Azure Front Door

The build outputs files with a **content hash** in the name, like `app.3f9a.js`. They are served with `Cache-Control: public, max-age=31536000, immutable` (one year, and the browser need not even revalidate). The HTML page itself gets a **short TTL**, such as `max-age=60`, and after each deploy simply points to the new file names.

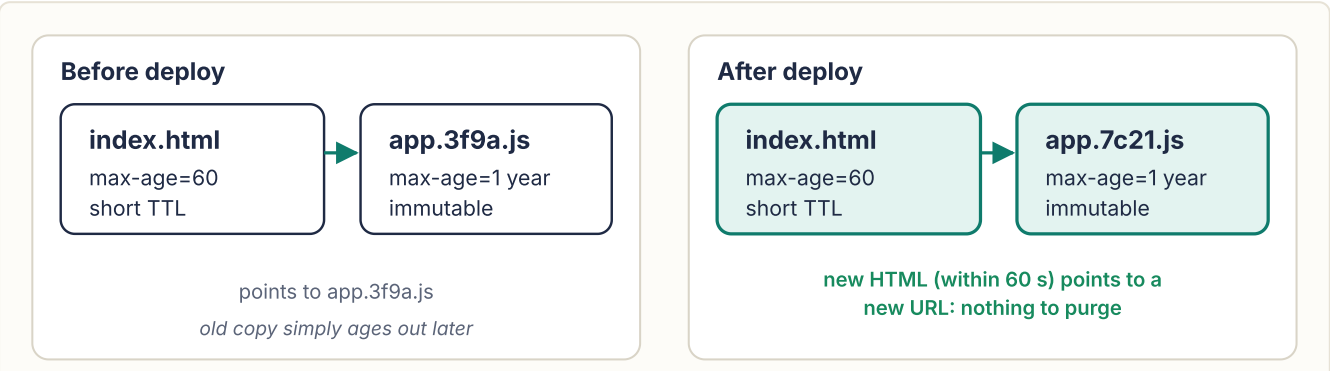


Figure 3. The content hash changes when the file changes, so a new version is a new URL. Only the short-lived HTML has to update.

5 Invalidating after a deploy

The best answer is: **you don't**. Versioned file names mean new content gets a new URL, and old copies simply age out. For things that must keep the same URL, like the HTML or an API response, use **short TTLs**, or issue a **purge**: an invalidation request for specific paths. Purges take time to spread to every PoP (seconds to minutes), and some CDNs charge per path (CloudFront, beyond a monthly free allowance), so they are the exception, not the plan.

COMMON MISTAKES

- Long TTLs on `app.js` without a hash in the name: users keep old code for days.
- Putting cookies or tracking query strings (`?utm_source=…`) in the cache key.
- Caching personalised API responses as `public`: one user's data shown to another.
- Relying on a purge in every deploy script.

6 Key terms

PoP: an edge location with cache servers

Cache hit ratio: share of requests served at the edge

Origin shield: middle cache in front of the origin

Origin: your own servers, the source of truth

Cache key: what identifies one cached object

Purge: invalidate specific cached paths

7 Interview questions

Q1. How does a CDN reduce latency? BASIC

It serves content from edge PoPs near users: handshakes and round trips are short (5 ms instead of 200 ms), cache hits skip the origin entirely, and misses reuse warm edge-to-origin connections.

Q2. Pull CDN vs push CDN? BASIC

Pull: the edge fetches from the origin on the first miss and caches it; easy, and the default for websites. Push: you upload content to the CDN ahead of time; good for large, rarely changing files like video libraries and downloads.

Q3. What is the cache hit ratio, and why does it matter? BASIC

The share of requests served from the edge. At 95%, the origin sees 1 request in 20; at 80%, 1 in 5, which is 4× more origin load. It drives both latency and origin cost.

Q4. How do you invalidate CDN content after a deploy? SCENARIO

Prefer versioned (content-hashed) file names with `max-age=31536000, immutable`, and keep the HTML short-lived (e.g. 60 s) so it points to new names. Purge only the few paths that must change in place.

Q5. Your hit ratio dropped from 95% to 30% after a marketing launch. What do you check? SCENARIO

The cache key. Likely new query parameters (`utm_*`) or a cookie are now part of it, so each visitor creates a new copy. Strip or ignore them in the key, and check `Vary` headers.

Q6. A new 50 MB video goes live and 300 PoPs miss at once. What protects the origin? SCENARIO

Origin shielding: all edge misses go to one shield PoP, which fetches the file once. The CDN also collapses concurrent misses for the same object into one origin request.

Q7. Can you put an API behind a CDN? TRICKY

Yes. Even uncached, it gains nearby TLS termination and warm connections. Cache only public, non-personalised GET responses with short TTLs; send `private` or `no-store` for per-user data.

Q8. Why is the HTML not given the same one-year TTL as the bundles? TRICKY

The HTML's URL never changes, so a long TTL would pin users to the old page, which points to old bundles. It is the entry point that must update, so it gets a short TTL or revalidation.

Next: SD-B-019 · Connection Pooling & Keep-Alive backendroadmap.com

Connection Pooling & Keep-Alive

pool sizing, leaks and idle timeouts

Watch the 3-min video

In one line: opening a connection costs DNS, TCP and TLS round trips, so keep connections open and share them through a pool; size it with Little's Law, and watch idle timeouts and leaks.

1 Why new connections are expensive

Your FastAPI service calls a payments API on every request. Under load, p99 latency jumps and connection errors appear. The culprit is often one line: **creating a new HTTP client inside every request handler**.

Every new HTTPS connection has a setup cost: a **DNS lookup**, a **TCP handshake** and a **TLS handshake**. That is two or three round trips before the first byte of your request. At 50 ms per round trip, that's over **100 ms of pure overhead**, plus CPU for the cryptography on both sides.

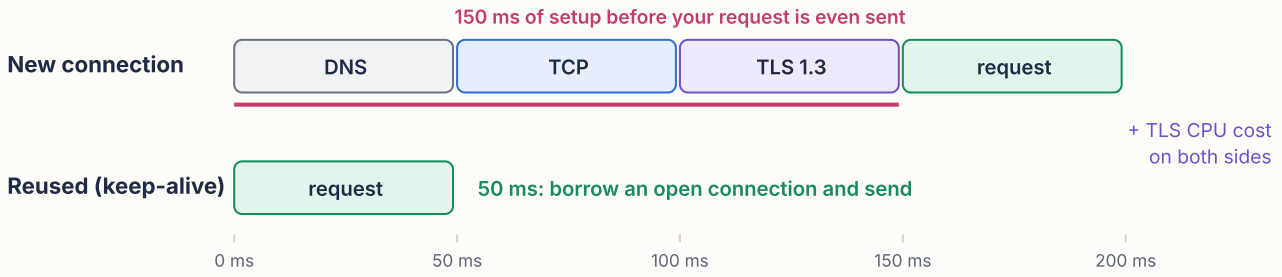


Figure 1. At 50 ms RTT a fresh connection spends ~150 ms on setup (DNS may be cached; TLS 1.2 adds a round trip). A reused one goes straight to the request.

2 Keep-alive and the connection pool

Keep-alive leaves the connection open after a response, so the next request can reuse it (the default in HTTP/1.1; HTTP/2 goes further and runs many requests over one connection). A **connection pool** manages a set of these open connections: you **borrow** an idle one, use it, and **give it back**. In Python that is `requests.Session` or an `httpx` client; database drivers and SQLAlchemy have their own pools. Without one, every call pays the full handshake again.

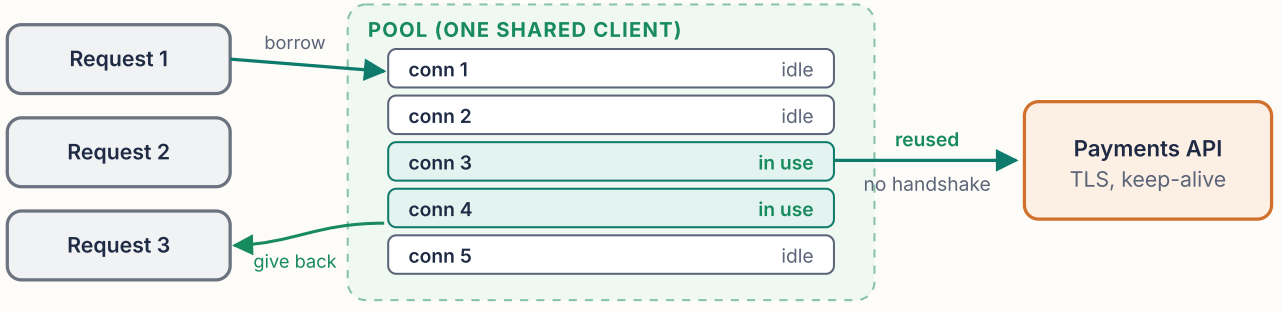


Figure 2. Requests borrow an idle connection, use it and return it. The pool keeps connections open, so calls skip the handshake.

Example: one shared client in FastAPI

Bad: a new `AsyncClient` inside the handler opens and tears down fresh connections on every request.

```
@app.get("/pay")
async def pay():
    async with httpx.AsyncClient() as c:   # new pool per request
        return (await c.post(URL, json=body)).json()
```

Fix: create one shared client at startup in FastAPI's `lifespan`, reuse it in every handler, and close it on shutdown.

```
@asynccontextmanager
async def lifespan(app):
    app.state.http = httpx.AsyncClient(timeout=5.0,
        limits=httpx.Limits(max_connections=30))
    yield
    await app.state.http.aclose()           # close on shutdown
app = FastAPI(lifespan=lifespan)
@app.get("/pay")
async def pay(request: Request):
    r = await request.app.state.http.post(URL, json=body)   # reused
    return r.json()
```

3 How big should the pool be?

Use **Little's Law**: connections needed = request rate × call duration.

```
200 calls/s × 0.1 s = 20 connections  (+ headroom, per worker or pod)
```

Too small, and requests queue for a free connection: latency climbs, then pool timeout errors appear, even though the downstream is perfectly healthy. **Too large**, and you overwhelm the downstream; big pools also waste memory and file descriptors.

WORKED EXAMPLE

Your service makes 200 payment calls per second, each taking 100 ms: about **20** connections busy at once. With ~50% headroom, aim for 30 in total. If that traffic is spread over 3 pods with 2 workers each (6 pools), each pool needs about 30 / 6 = **5**. Now a database with `max_connections = 100`, and 10 pods each holding a pool of 50 means up to **500**. The database starts refusing connections. Size each pool so the total across all pods fits, or put a pooler such as PgBouncer in front.

4 Two more traps: idle timeouts and leaks

Idle timeouts. If the server or load balancer closes idle connections after 60 s, but your pool keeps them for 90 s, you will reuse dead connections and see random resets. Keep the client's idle timeout **shorter** than the server's.

Leaks. If responses or streams aren't closed, their connections never return to the pool, until it is exhausted and every call waits for a pool timeout. Always use context managers (`async with client.stream(...)`, `with Session() as s`) so connections go back even on errors.

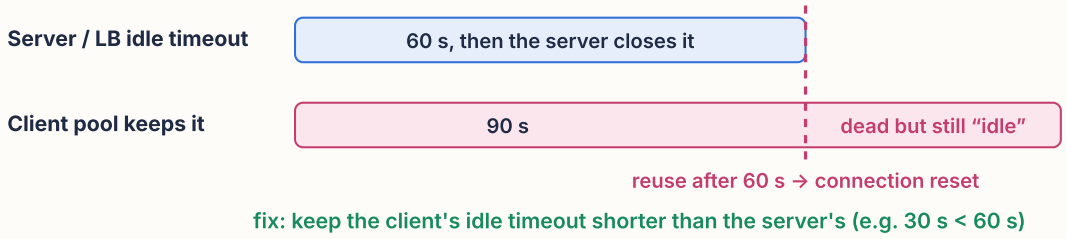


Figure 3. The server closed the connection at 60 s, but the client still thinks it is idle and usable. The next request on it fails.

COMMON MISTAKES

- A new client (or a new database engine) per request.
- Sizing each pod's pool on its own and forgetting to multiply by pods and workers.
- Raising the pool when the real problem is a leak or a slow downstream.

5 Key terms

Keep-alive: leave a connection open for reuse

Pool timeout: max wait to borrow a connection

Connection leak: borrowed, never returned

Connection pool: set of open connections to borrow

Idle timeout: close a connection unused this long

PgBouncer: external pooler in front of Postgres

6 Interview questions

Q1. Why reuse connections? BASIC

To skip DNS, TCP and TLS setup on every call. At 50 ms RTT that is about 100–150 ms and real CPU per call saved, which cuts latency and load on both sides.

Q2. What goes wrong when a pool is too small or too large? BASIC

Too small: requests queue for a connection and hit pool timeouts while the downstream is healthy. Too large: you exceed downstream limits like database max connections, and waste memory and file descriptors.

Q3. p99 jumped after traffic doubled, and logs show pool timeouts, but the payments API is healthy. What happened? SCENARIO

In flight doubled (say 20 → 40) and the pool stayed at 20, so requests wait for a connection. Raise the pool to rate × duration + headroom, and check for leaks (connections not returned).

Q4. You see random `connection reset` errors, mostly after quiet periods. Cause? SCENARIO

The pool reuses connections the server or load balancer already closed as idle (say at 60 s) while the client keeps them for 90 s. Set the client's keepalive expiry below the server's, e.g. 30 s.

Q5. Does HTTP/2 remove the need for a pool? TRICKY

Mostly for HTTP: one HTTP/2 connection multiplexes many concurrent requests, so you need few connections. You still reuse them, and database connections still need a pool.

Q6. The pool is exhausted at low traffic. Why? TRICKY

A leak: responses or streams not closed. Connections never return, so a small rate still drains it. Use context managers and watch the in-use count.

Next: SD-B-020 · Network Timeouts backendroadmap.com

Network Timeouts

connect, read, deadlines and the random 502

Watch the 3-min video

In one line: every outbound call needs a connect, read and pool timeout plus an overall deadline; pass the remaining budget downstream, make inner timeouts shorter than outer ones, and keep the app's keep-alive longer than the load balancer's.

1 Why timeouts matter

A downstream service hangs. It doesn't fail, it just stops answering. Without timeouts, every request that calls it **waits forever**. Threads pile up, connection pools drain, and soon your whole service is down too. Timeouts stop one slow dependency from taking everything else with it.

2 The kinds of timeout

Timeout	Limits how long you wait to...	Typical value
Connect	establish the TCP and TLS connection	1–2 s
Read	receive the *next bytes* of the response (not the whole response)	a little above the dependency's p99
Write	send a large request body	a few seconds
Pool	borrow a connection from the pool	short, e.g. 0.5–1 s
Idle	keep an unused connection open before closing it	shorter than the server's

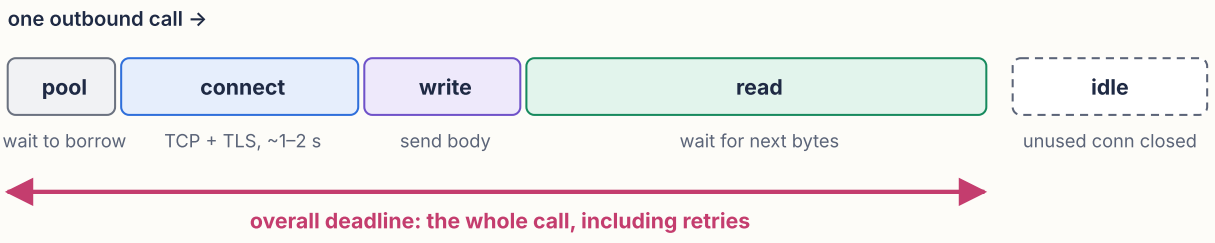
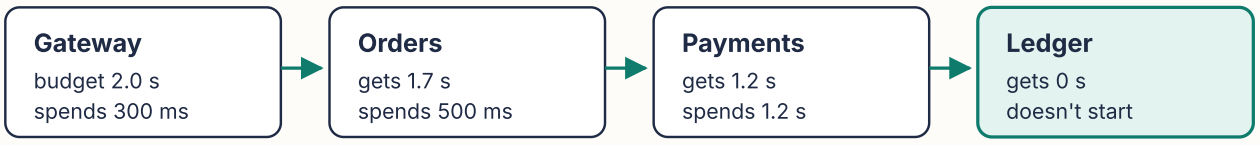


Figure 1. Each phase of a call has its own timeout. The overall deadline caps the sum, including any retries.

3 Per-operation timeouts are not enough

A read timeout of 5 seconds still allows a response that trickles in one byte every 4 seconds, forever: each read finishes in time. So also set a **total deadline** for the whole call, including retries. If the user-facing request has 2 seconds, no single call inside it should be allowed to take 10.

In **microservices, propagate the deadline**. The gateway receives a request with a 2-second budget, spends 300 ms, and passes the remaining 1.7 s to the next service, and so on. gRPC does this natively with deadlines; with HTTP you pass a header. If the budget is already spent, the downstream service doesn't even start the work.



pass the remaining budget down: gRPC deadlines natively, a header (e.g. X-Request-Deadline) over HTTP

if the budget is already spent, the downstream service returns at once instead of doing useless work

Figure 2. Each hop subtracts what it used. By the time the budget reaches zero, nobody downstream wastes effort on an answer no one will wait for.

4 Shrink timeouts inward, and the classic 502

Timeouts should get **shorter as you go deeper**. If the load balancer allows 60 s, the app's request timeout might be 55 s, and the database query timeout 50 s. Then the innermost call fails first with a clear error, and the caller still has time to retry or return a fallback, instead of the outer layer cutting the connection mid-work.

Now the classic bug. The load balancer has a **60-second idle timeout**, but the app server's keep-alive timeout is only **5 seconds** (Uvicorn's default). The app quietly closes an idle connection at the same moment the load balancer reuses it to send a request. The result: random **502 Bad Gateway** errors, often under *light* traffic, when connections sit idle. The fix: make the app's keep-alive longer than the load balancer's, say **75 s**, so the load balancer always closes first.

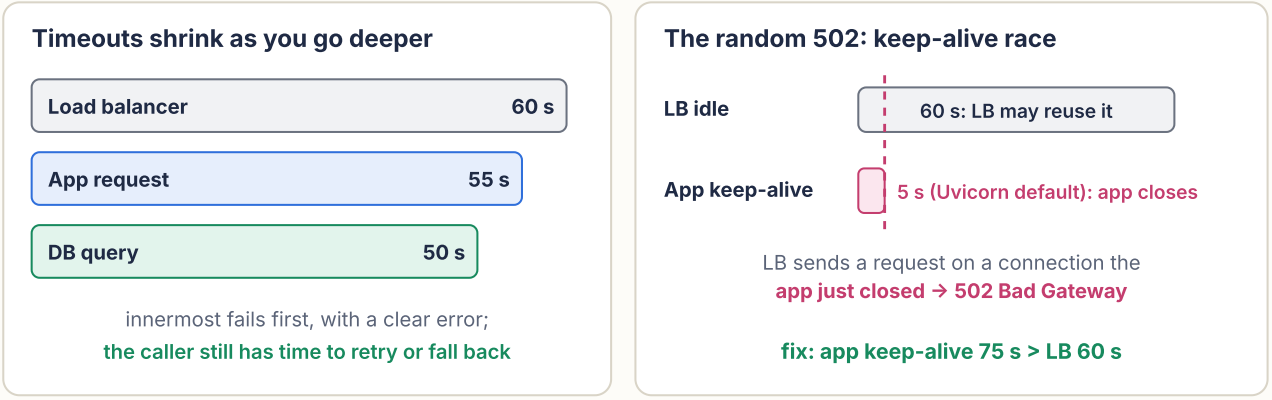


Figure 3. Left: outer layers wait longer than inner ones. Right: when the app closes idle connections before the load balancer does, they race and the loser is a 502.

WORKED EXAMPLE: PRACTICAL DEFAULTS

A payments API has a p99 of 400 ms; your endpoint promises 2 s. Use connect 1 s, read 1 s, pool 0.5 s, and an overall deadline of 1.5 s, leaving 0.5 s for your own work. That allows one retry with backoff, and only because the call carries an idempotency key. In httpx: `httpx.Timeout(1.0, connect=1.0, pool=0.5)` plus `asyncio.timeout(1.5)` around the call. Behind a 60 s load balancer, run `uvicorn --timeout-keep-alive 75`.

Base timeouts on the dependency's **real p99 latency**, not on guesses. Add retries with backoff **only when the operation is idempotent**. And never trust the library default, which is sometimes **no timeout at all**, as with Python's `requests`.

COMMON MISTAKES

- Calling `requests.get(url)` with no `timeout=`: it can wait forever.
- Retries without a total deadline: 3 tries × 5 s read timeout turns a 2 s budget into 15 s.
- An inner timeout longer than the outer one, so the outer layer cuts the work off with no clear error.
- App keep-alive shorter than the load balancer's idle timeout.

5 Key terms

Connect timeout: max time to set up TCP + TLS

Pool timeout: max wait to borrow a connection

Deadline propagation: pass the remaining budget downstream

Read timeout: max gap waiting for the next bytes

Deadline: absolute time budget for the whole call

502 Bad Gateway: proxy got no valid upstream response

6 Interview questions

Q1. What timeouts should every outbound call have? BASIC

A connect timeout (1–2 s), a read timeout (above the dependency's p99), a pool or acquire timeout, and an overall deadline that covers retries.

Q2. Why isn't a read timeout enough on its own? BASIC

It limits the gap between bytes, not the whole response. A server trickling one byte every 4 s never trips a 5 s read timeout, so add a total deadline.

Q3. A dependency hangs and your whole service goes down. Why, and how do you prevent it? SCENARIO

Without timeouts, each request waits forever, holding a thread and a connection; pools drain and everything queues. Set timeouts and a deadline on every call, and add a circuit breaker to fail fast while it is down.

Q4. Your gateway has a 2 s budget and calls three services in a chain. How do you set their timeouts? SCENARIO

Propagate the deadline: each hop gets what's left (2.0 → 1.7 → 1.2 s...), via gRPC deadlines or a header. A service that receives an already-expired budget returns at once without starting work.

Q5. You see random 502s from the load balancer, mostly at night with little traffic. Cause? SCENARIO

A keep-alive race: the LB keeps idle connections for 60 s, the app (Uvicorn's default 5 s) closes them sooner, and the LB reuses one the app just closed. Set the app's keep-alive above the LB's, e.g. 75 s.

Q6. Why should a client timeout be shorter than the server's? TRICKY

So the inner side gives up first. A client that closes idle connections before the server never reuses a dead one, and inner calls fail with a clear error while the caller still has time to retry or fall back.

Q7. Is it safe to retry every timed-out call? TRICKY

No. A timeout doesn't tell you whether the server did the work. Retry only idempotent operations (GET, or POST with an idempotency key), with backoff and jitter, and inside the overall deadline.

Next: SD-C-001 · REST Principles backendroadmap.com